PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA INFORMÁTICA

Sistema de Optimización de Recolección Colaborativa de Residuos Domiciliarios (RESCOL)

Alonso M. Robledo Álamos

Profesora Guía: **Leslie Pérez Cáceres**
Asignatura: **MII907 - Tesis de Grado**
Fecha de entrega: **15 de enero de 2024**

Enero de 2024

Índice

Resumen ..... **I**
Palabras clave I

Resumen ..... **I**
Keywords I

Lista de Figuras ..... **II**

Lista de Tablas ..... **IV**

Introducción **1**

1 Objetivos **4**

2 Estado del Arte **5**
2.1 Problemas Relacionados 5
2.1.1 Vehicle Routing Problem (VRP) y Variantes 5
2.1.2 Arc Routing Problem (ARP) y Variantes 7
2.1.3 Chinese Postman Problem (CPP) y Variantes 9
2.2 Recolección de Residuos Domiciliarios 10
2.2.1 Implementación basada en ARP 10
2.2.2 Implementación basada en VRP 11
2.2.3 Recolección de residuos domiciliarios en Argentina 12
2.2.4 Aplicación de la Tecnología Blockchain en la Gestión de Residuos 14
2.2.5 Tecnologías y Sistemas Computacionales Aplicados al Manejo de Residuos en la India 15
2.3 Métodos de Resolución de Problemas de Optimización 16
2.3.1 Métodos de Resolución Exactos 16
2.3.2 Métodos de Resolución Aproximados 16
2.3.3 Ant Colony Optimization 17

3 Definición del Problema **24**
3.1 Grafo de la Red Vial 24
3.2 Modelo Matemático 28
3.3 Instancias 29
3.3.1 Instancias Sintéticas 31
3.3.2 Casos Reales 35

4 Propuestas de Solución **38**
4.1 Métodos Exactos 38
4.2 Metaheurísticas: Ant Colony Optimization 39
4.2.1 Feromonas: tipos e inicialización 42
4.2.2 Construcción de rutas 42
4.2.3 Deposito global de feromonas 43

4.2.4 Evaporación de feromonas 43
4.2.5 Mecanismos nuevos 43
4.2.6 Parámetros 44
4.2.7 Estructuras de Datos 45
4.3 Finetuning o Sintonización Fina 46

5 Experimentos 48
5.1 Setup Experimental 48
5.1.1 Consideraciones en las Pruebas 50
5.1.2 Metodología del Finetuning 51
5.1.3 Proceso y Ejecución del Finetuning 53
5.2 Resultados y Discusión 54
5.2.1 Instancia A1 54
5.2.2 Instancia CasoRealPequeño 58
5.2.3 Instancia CasoRealMediano 62
5.2.4 Instancia CasoRealGrande 64
5.2.5 Comparación Entre Resultados de ACO 66
5.2.6 Resultados del Finetuning 72

6 Metodología de Trabajo 76

Conclusión y Trabajo Futuro 78

Bibliografía 80

Anexos A1

Resumen

La recolección de desechos es un problema que surgió cuando los seres humanos comenzaron a vivir en comunidades, y con el paso del tiempo, el volumen de residuos ha aumentado considerablemente. En tiempos modernos, la optimización de rutas para la recolección de basura puede generar ahorros en combustible y tiempo, además de asegurar la recogida de desechos en todas las calles del sector. En este trabajo, se investiga el estado del arte en problemas de ruteo y se propone un modelo preliminar que considera aspectos clave del contexto de recolección de basura, como el sentido de las calles y las normas de tránsito. En otros trabajos afines se han explorado modelos basados en Vehicle Routing Problem, Arc Routing Problem y Chinese Postman Problem, y se han utilizado técnicas como Programación de Enteros para instancias pequeñas o algoritmos metaheurísticos para resolver problemas de gran tamaño. En esta tercera iteración, se logró una versión estable y mejorada de la implementación de una metaheurística basada en el comportamiento de las hormigas que nos sirvió como base para seguir refinando este algoritmo, tomando como referencia métricas obtenidas de la resolución de instancias que representan redes viales de comunas de la Región Metropolitana. Un aspecto clave fue la utilización del configurador automático de algoritmos irace, que permitió obtener la combinación óptima de los parámetros del algoritmo para un conjunto determinado de instancias similares entre si. Esto a su vez nos da una visión general del comportamiento de dichos parámetros bajo ciertas situaciones.

Palabras clave: *Recolección de desechos, optimización de rutas, Arc Routing Problem, Vehicle Routing Problem, Chinese Postman Problem, Metaheurística, Ant Colony Optimization*

Abstract

Waste collection is a problem that arose when humans began to live in communities, and over time, the volume of waste has increased considerably. In modern times, optimizing routes for garbage collection can generate savings in fuel and time, in addition to ensuring waste collection on all streets in the sector. In this work, the state of the art in routing problems is investigated and a preliminary model is proposed that considers key aspects of the garbage collection context, such as street direction and traffic regulations. In other related works, models based on the Vehicle Routing Problem, Arc Routing Problem and Chinese Postman Problem have been explored, and techniques such as Integer Programming for small instances or metaheuristic algorithms have been used to solve large problems. In this third iteration, a stable and improved version of the implementation of a metaheuristic based on the behavior of ants was achieved, which served as a basis to continue refining this algorithm, taking as reference metrics obtained from the resolution of instances that represent road networks. of communes of the Metropolitan Region. A key aspect was the use of the automatic algorithm configurator irace, which made it possible to obtain the optimal combination of algorithm parameters for a given set of similar instances. This in turn gives us a general view of the behavior of said parameters under certain situations.

Palabras clave: *Garbage Collection, Route Optimization, Arc Routing Problem, Vehicle Routing Problem, Chinese Postman Problem, Metaheuristics, Ant Colony Optimization*

Índice de figuras

Figura 2.1 Ejemplo de VRP. Fuente: Wikimedia Commons. 6
Figura 2.2 Ejemplo de CPP. Fuente: Wikimedia Commons 9
Figura 2.3 Depósito de feromonas. Fuente: Wikimedia Commons 21
Figura 3.1 Ejemplo de Grafo de una red vial. 25
Figura 3.2 Transformación de nodos de intersecciones. 26
Figura 3.3 Grafo transformado. 27
Figura 3.4 Visualización de instancia A1 32
Figura 3.5 Visualización de instancia AvenidaTest 33
Figura 3.6 Frecuencia pesos de las calles de una instancia generada con una distribución normal. 34
Figura 3.7 Frecuencia pesos de las calles de una instancia generada con una distribución gamma. 35
Figura 3.8 Visualización de instancia CasoRealPequeño 36
Figura 3.9 Visualización de instancia CasoRealMediano 37
Figura 5.1 Ruta generada por Backtracking - A1. 55
Figura 5.2 Comparación de costos - A1. 56
Figura 5.3 Comparación del largo total del tour - A1. 56
Figura 5.4 Comparación de tiempo de ejecución - A1. 57
Figura 5.5 Ruta generada por Backtracking - CasoRealPequeño. 58
Figura 5.6 Comparación de costos - CasoRealPequeño. 59
Figura 5.7 Comparación del largo total del tour - CasoRealPequeño. 59
Figura 5.8 Comparación de tiempo de ejecución - CasoRealPequeño. 60
Figura 5.9 Comparación de costos - CasoRealMediano. 62
Figura 5.10 Comparación del largo total del tour - CasoRealMediano. 62
Figura 5.11 Comparación de tiempo de ejecución - CasoRealMediano. 63
Figura 5.12 Comparación de costos - CasoRealGrande. 64
Figura 5.13 Comparación del largo total del tour - CasoRealGrande. 65
Figura 5.14 Comparación de tiempo de ejecución - CasoRealGrande. 65
Figura 5.15 Gráfico de convergencia - Ant System - CasoRealMediano. 66
Figura 5.16 Gráfico de convergencia - MAX-MIN Ant System - CasoRealMediano. 67
Figura 5.17 Gráfico de convergencia - Ant System con matriz de salida - CasoRealMediano. 67
Figura 5.18 Gráfico de convergencia - MAX-MIN Ant System con matriz de salida - CasoRealMediano. 68
Figura 5.19 Gráfico de convergencia - Ant System con beta0 - CasoRealMediano. 68
Figura 5.20 Gráfico de convergencia - MAX-MIN Ant System con beta0 - CasoRealMediano. 69
Figura 5.21 Gráfico de convergencia - Ant System con limitador - CasoRealMediano. 69
Figura 5.22 Gráfico de convergencia - MAX-MIN Ant System con limitador - CasoRealMediano. 70
Figura 5.23 Gráfico de convergencia - Ant System con oscilador - CasoRealMediano. 70
Figura 5.24 Gráfico de convergencia - MAX-MIN Ant System con oscilador - CasoRealMediano. 71

Figura 5.25 Gráfico de violín - Mejores configuraciones Validación. 73
Figura 5.26 Gráfico de violín - Mejores configuraciones Costo Validación. 73
Figura 5.27 Gráfico de violín - Mejores configuraciones - Entrenamiento. 74
Figura 5.28 Gráfico de violín - Mejores configuraciones Costo - Entrenamiento. ... 75
Figura 6.1 Carta Gantt Tesis RESCOL. 77
Figura 6.2 Gráfico de convergencia - A1 - Ant System A9
Figura 6.3 Gráfico de convergencia - A1 - Ant System con beta0 ..... A10
Figura 6.4 Gráfico de convergencia - A1 - Ant System con limitador A11
Figura 6.5 Gráfico de convergencia - A1 - Ant System con matriz de salida A12
Figura 6.6 Gráfico de convergencia - A1 - Ant System con oscilador A13
Figura 6.7 Gráfico de convergencia - A1 - MAX-MIN Ant System A14
Figura 6.8 Gráfico de convergencia - A1 - MAX-MIN Ant System con beta0 ..... A15
Figura 6.9 Gráfico de convergencia - A1 - MAX-MIN Ant System con limitador . A16
Figura 6.10 Gráfico de convergencia - A1 - MAX-MIN Ant System con matriz de salida A17
Figura 6.11 Gráfico de convergencia - A1 - MAX-MIN Ant System con oscilador . A18
Figura 6.12 Gráfico de convergencia - CasoRealGrande - MAX-MIN Ant System con matriz de salida, beta0 y limitador A19
Figura 6.13 Gráfico de convergencia - CasoRealGrande - Ant System con matriz de salida, beta0 y limitador A19
Figura 6.14 Gráfico de convergencia - CasoRealPequeño - Ant System A20
Figura 6.15 Gráfico de convergencia - CasoRealPequeño - Ant System con beta0 . A21
Figura 6.16 Gráfico de convergencia - CasoRealPequeño - Ant System con limitador A22
Figura 6.17 Gráfico de convergencia - CasoRealPequeño - Ant System con matriz de salida A23
Figura 6.18 Gráfico de convergencia - CasoRealPequeño - Ant System con oscilador A24
Figura 6.19 Gráfico de convergencia - CasoRealPequeño - MAX-MIN Ant System A25
Figura 6.20 Gráfico de convergencia - CasoRealPequeño - MAX-MIN Ant System con beta0 ..... A26
Figura 6.21 Gráfico de convergencia - CasoRealPequeño - MAX-MIN Ant System con limitador A27
Figura 6.22 Gráfico de convergencia - CasoRealPequeño - MAX-MIN Ant System con matriz de salida A28
Figura 6.23 Gráfico de convergencia - CasoRealPequeño - MAX-MIN Ant System con oscilador A29
Figura 6.24 Visualización de coordenadas paralelas - Mejores soluciones A30
Figura 6.25 Visualización de coordenadas paralelas - Todo el espacio de búsqueda A31
Figura 6.26 Carta Gantt A32

Índice de tablas

Tabla 5.1 Parámetros base 49
Tabla 5.2 Parámetros MAX-MIN Ant System 49
Tabla 5.3 Recursos Algorítmicos 49
Tabla 5.4 Parámetros irace 52
Tabla 5.5 Recursos Algorítmicos irace y ACO 53
Tabla 5.6 Valores de las 6 mejores configuraciones 72
Tabla 5.7 Top 3 configuraciones de validación 74
Tabla 5.8 Top 6 configuraciones de entrenamiento 75
Tabla 6.1 Resultados Gurobi + Backtracking - A1 ..... A1
Tabla 6.2 Resultados Ant System - A1 ..... A2
Tabla 6.3 Resultados MAX-MIN Ant System - A1 ..... A3
Tabla 6.4 Resultados Gurobi + Backtracking - CasoRealPequeño A4
Tabla 6.5 Resultados Ant System - CasoRealPequeño A5
Tabla 6.6 Resultados MAX-MIN Ant System - CasoRealPequeño A6
Tabla 6.7 Resultados Ant System - CasoRealMediano A7
Tabla 6.8 Resultados MAX-MIN Ant System - CasoRealMediano A8

Introducción

La recolección de desechos ha sido un problema que ha existido desde que el ser humano vive en sociedad. Se considera que el primer vertedero que se conoce data del año 3000 AC en la ciudad de Knossos, capital de Creta, donde los pobladores de la civilización Minoica vertían sus desechos sólidos [8]. En la ciudad-estado de Atenas, alrededor del año 500 AC. se promulgó una ley que requería a todos sus ciudadanos botar su basura al menos dos kilómetros a las afueras de la ciudad [30]. Hoy en día se generan aproximadamente 2 billones de toneladas de desechos sólidos, y se proyecta que esta cantidad aumente a 3.4 billones de toneladas para el año 2050 [64]. Independientemente de la cantidad o de la manera en que los desechos serán tratados, primero es necesaria su recolección.

Según el Reporte del Estado del Medio Ambiente 2022 del Gobierno de Chile, en 2020 se generaron poco más de 18 millones de toneladas de residuos. De esta cantidad, 7.6 millones de toneladas corresponden a desechos municipales, donde el 91.5 % se destina a rellenos sanitarios, mientras que el resto va a vertederos, basurales u otros tipos de depósitos [41]. El principal problema es la recolección de estos desechos. En nuestro país, lo que se suele hacer, es que las personas dejan sus contenedores o bolsas con basura afuera de sus casas o las llevan a una calle más central. Luego pasa un camión recolector con operarios que recogen manualmente los desechos y los introducen al camión, que compacta la basura, para luego llevarla a un depósito temporal o definitivo. La ruta que siguen los camiones suele ser una dirección general que les da la municipalidad para la cual prestan servicios. Por ejemplo, a cada camión se le asigna un sector dentro de la comuna o una calle principal que deben recorrer en cierto sentido [58], sin embargo, no les dicen cómo recorrer las calles interiores, esto queda a criterio del conductor en base a su experiencia. Si bien, esto ha funcionado adecuadamente, es posible optimizarlo, lo que podría ayudar a reducir costos y horas de trabajo, especialmente en centros urbanos grandes o con redes viales complejas.

La manera óptima de recoger la basura de una zona determinada es un problema extremadamente complejo. Esto se debe a que es necesario resolver una serie de problemas que conjuntamente dan una solución a nuestro problema inicial. Esto involucra encontrar la mejor forma de sectorizar la comuna o ciudad y buscar el conjunto de rutas óptimas dentro de cada sector. Además, se debe tener en consideración la cantidad y capacidad de los camiones de recolección, sentido de las calles, señaléticas, costo asociado a transitar las calles con y sin carga, ubicación de los depósitos, entre otros. Modelar este problema considerando tantas variables puede resultar contraproducente, ya que, añade mucha complejidad y costo computacional a cambio de una solución ligeramente mejor, por lo que se debe encontrar un equilibrio entre la complejidad del modelo y la calidad de la solución, además del tiempo que toma encontrarla.

Durante los últimos 60 años, los problemas de ruteo han ganado mucha popularidad y atención de investigadores. Esto se debe a su potencial impacto económico en todo tipo de industrias que requieran visitar clientes o lugares para prestar un servicio [21]. Se han estudiado muchas variaciones de estos problemas, sin embargo, la que más se acerca a este

contexto es el problema de ruteo de arcos o ARP [21] por sus siglas en inglés. En términos simples, este problema busca encontrar la ruta más corta para visitar todos los arcos en una red, como las calles de una ciudad, teniendo en cuenta ciertas restricciones u objetivos, como distancia o tiempo. En términos más técnicos, el ARP consiste en encontrar la ruta más corta que recorra todas las aristas o bordes de un grafo, también considerando esas restricciones y objetivos. Existen otras variantes como el Capacitated ARP [33] que modela el problema de una manera mas realista considerando la capacidad de los vehículos.

El ARP y sus variantes han sido resueltos mediante una variedad de técnicas. Cuando el conjunto de soluciones es pequeño, es posible utilizar técnicas exactas para evaluar todas las soluciones para determinar la óptima global. Para esto, se han propuesto algoritmos tipo Branch and Bound, que han sido capaces de resolver instancias de hasta 30 arcos obligatorios [61, 69, 45, 52], estos algoritmos proponen distintas maneras de calcular los límites inferiores y superiores, que son componentes usado en esta técnica para eliminar conjuntos de soluciones infactibles. Además, en investigaciones mas recientes[12, 68] han usado programación matemática para resolver problemas de ruteo de arcos en contextos reales, como la recolección de desechos municipales, que es el problema que aborda este trabajo. Por otra parte, cuando el tamaño del problema aumenta, los métodos exactos ya no son viables dado el tiempo requerido para calcular la solución óptima global. Para estos casos, es necesario usar técnicas que entreguen soluciones aproximadas o sub-óptimas en un tiempo razonable. Este tipo de técnicas se conocen como metaheurísticas, y son especialmente útiles cuando la complejidad del problema impide que su conjunto de soluciones sea explorado por completo. Algunas metaheurísticas que han sido utilizadas para resolver problemas de ruteo como el ARP se destacan los algoritmos genéticos [47, 48], búsqueda tabú [13] y algoritmos basados en colonias de hormigas [60].

En este proyecto se aborda el diseño de un algoritmo para optimizar la recolección de residuos domiciliarios, modelándolo como un Arc Routing Problem. Se busca desarrollar el algoritmo teniendo en consideración variables como las leyes del tránsito, un costo de transitar y recorrer las calles para un sector en particular. El diseño se centra en el uso de técnicas aproximadas, específicamente metaheurísticas basadas en el comportamiento de las hormigas como Ant Colony Optimization.

Este documento se divide en secciones y subsecciones, las cuales están estructuradas de la siguiente manera: La sección 1 describe los objetivos de este proyecto; seguida por la sección 2 la cual provee una revisión del estado del arte, que está dividida en una subsección para describir los problemas relacionados al nuestro, otra para describir estos problemas aplicados al mundo real y como fueron abordados, y una tercera, para explorar métodos de resolución de problemas de optimización, estas corresponden a las subsecciones 2.1, 2.2 y 2.3. Después, en la sección 3 se define el problema en su representación como grafo, junto con el modelo matemático y formato de instancias que serán utilizados posteriormente, esto en las subsecciones 3.1, 3.2 y 3.3 respectivamente. Luego viene la sección 4 donde se explican las propuestas de solución, tanto para métodos exactos en 4.1, como para métodos aproximados en 4.2. En la sección 5 de experimentos, se definen y muestran una secuencia de experimentos incrementales en términos de desarrollo del algoritmo. Primero se muestra

un setup experimental inicial en la subsección 5.1, luego se muestran los resultados de estos experimentos en 5.2, para luego explicarse la subsección del finetuning 4.3, proceso crucial en el desarrollo del algoritmo. Después se explica la metodología utilizada junto con una carta Gantt en la sección 6. Finalmente se exponen las conclusiones obtenidas en este trabajo en la sección 6.

1. Objetivos

Objetivo General: Diseñar un algoritmo para generar rutas de recolección de residuos domiciliarios automáticamente tomando en cuenta ciertos criterios de optimización, con la finalidad de mejorar el sistema de recolección de basura en dos comunas de la región Metropolitana en Chile.

Para lograr esto, se proponen los siguientes objetivos específicos:

- Investigar sobre los distintos tipos de problemas de ruteo, tanto en el ámbito académico y como práctico.
- Definir y modelar el problema en particular al cual se quiere buscar una solución.
- Plantear distintas alternativas de solución para el problema de ruteo identificado, evaluando su viabilidad y eficiencia.
- Desarrollar un sistema que funcione a partir de la solución escogida, implementando el algoritmo propuesto.
- Definir un conjunto de casos de pruebas e instancias para probar el algoritmo desarrollado y medir su eficiencia.

Al cumplir con estos objetivos, no solo permitiría una mejora en el actual sistema de recolección de basura en las comunas que participan del proyecto, sino también sentaría un precedente para que otras comunas adopten un sistema similar y puedan mejorar los servicios de recolección. Siempre y cuando la solución sea exitosa y provoque un impacto positivo, ya sea ahorro de combustible de los camiones o esfuerzo por parte de los trabajadores.

2. Estado del Arte

Los problemas de ruteo son clásicos en la literatura sobre investigación de operaciones, probablemente uno de los mas conocido es el Travelling Salesman Problem [23], aunque existen otros como el Arc Routing Problem [21] o el Vehicle Routing Problem [49] Estos problemas han sido objeto de estudio desde la década de los 50 hasta nuestros días. Algunas de las primeras publicaciones académicas sobre problemas de ruteo fueron, Dantzig et al. (1954) [23], Dantzig & Ramser (1959) [24] y Guan (1962) [34]. La optimización de las rutas de recolección de desechos es una aplicación practica de estos problemas de ruteo, una de las primeras publicaciones en este contexto fue realizada en 1974 [7], en la cual se busca optimizar las rutas de los camiones recolectores en la ciudad de Nueva York mediante metaheurísticas y programación matemática.

En esta sección se revisa el estado del arte relacionado a problemas de ruteo como ARP, VRP y CPP, tanto sus definiciones, principales características, variantes e implementaciones en el mundo real. Sobre este ultimo punto, se revisa como fueron resueltos estos problemas desde el punto de vista técnico. Se explicará también sobre la teoría de los métodos de resolución exactos y aproximados.

2.1. Problemas Relacionados

El problema de la optimización de las rutas que siguen los camiones recolectores puede modelarse de distintas maneras. En esta sección se revisa el estado del arte de tres problemas clásicos en investigación de operaciones, que pueden aplicarse en la situación estudiada. Estos son, Vehicle Routing Problem (VRP), Arc Routing Problem (ARP) y Chinese Postman Problem (CPP), los cuales, dependiendo de sus restricciones u objetivos, dan origen a sus distintas variantes. En general, las rutas o soluciones que se buscan en estos problemas se encuentran seleccionando un conjunto de calles y generando las rutas dentro de este conjunto. Además, se pueden considerar elementos como cantidad, capacidad y velocidad de los camiones, tiempo de recolección, ventanas de tiempo, cantidad y ubicación de depósitos o rellenos sanitarios, sentido de las calles, leyes del transito, entre otros.

2.1.1. Vehicle Routing Problem (VRP) y Variantes

El problema de ruteo de vehículos (VRP), propuesto en 1959 [24], es un desafío de optimización de complejidad computacional NP-hard que busca encontrar un conjunto de rutas óptimas para satisfacer las necesidades de un conjunto de clientes, minimizando el costo total al optimizar factores como la distancia recorrida, el tiempo de viaje y otros aspectos relacionados con una flota de vehículos. Este problema no se centra en un solo vehículo, sino en una flota que sigue diferentes rutas, a diferencia de problemas clásicos de ruteo como el problema del vendedor viajero (TSP).

La figura 2.1 describe visualmente un problema de tipo Vehicle Routing Problem.

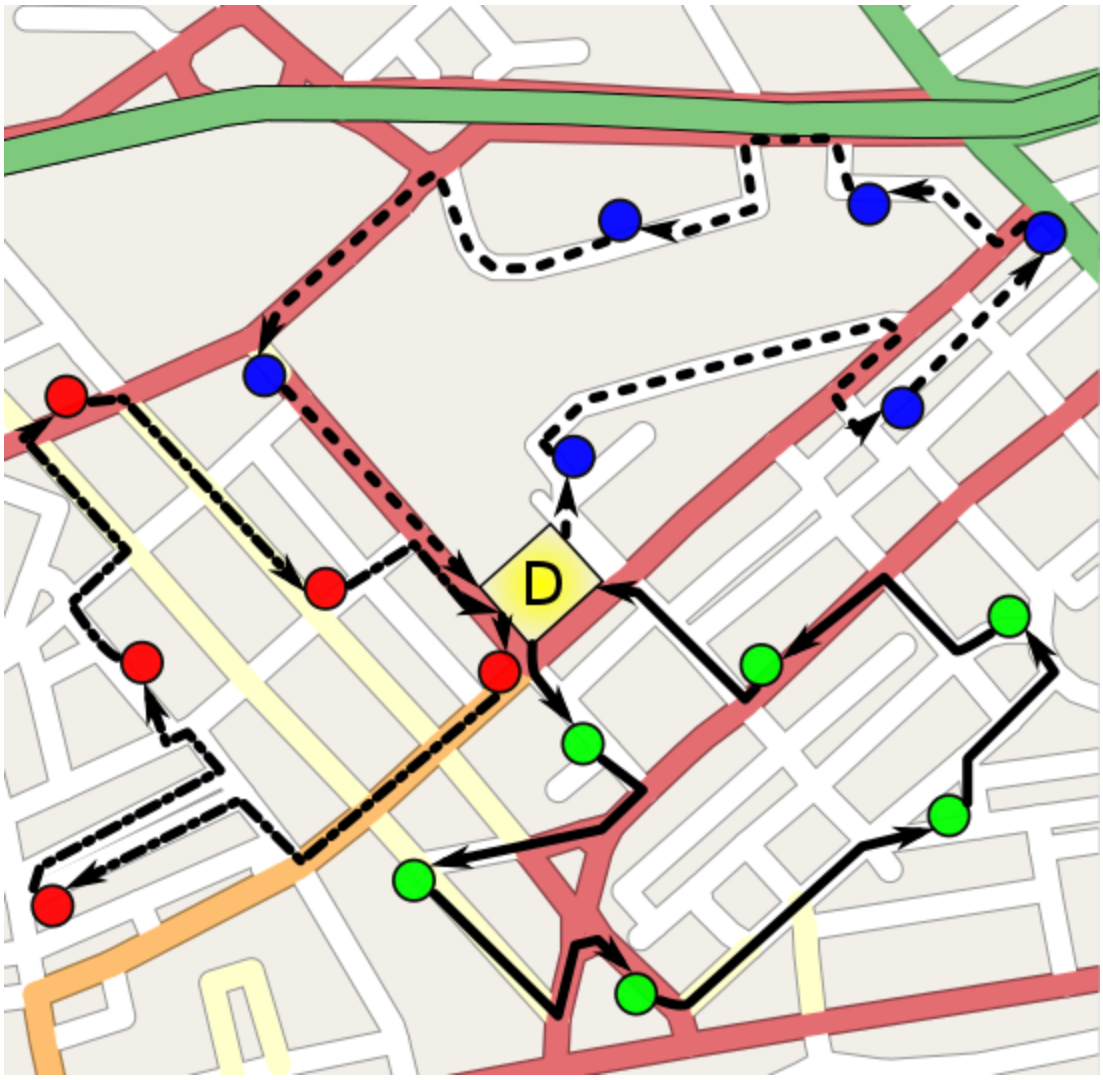


Figura 2.1: Ejemplo de VRP. Fuente: Wikimedia Commons.

En la figura, existen 3 vehículos, identificados con los colores rojo, azul y verde, donde cada uno realiza rutas de manera independiente y sirven a sus respectivos clientes representados por los puntos de colores. Además, se indican las rutas que siguen estos vehículos, las cuales comienzan y terminan en el mismo punto D.

La definición matemática del VRP clásico se define de la siguiente manera [46]: Sea $G = (V, A)$ un grafo dirigido donde $V \in \{0, \ldots, n\}$ es el conjunto de vértices y $A \in \{(i, j) : i, j \in V, i \neq j\}$ es el conjunto de arcos. En el conjunto V, el elemento 0 corresponde al depósito de los vehículos, los cuales deben estar definidos previamente como un conjunto de K vehículos idénticos con capacidad Q que sirven a cada cliente i . Cada arco del grafo está sujeto a un costo c_{ij} y una variable x_{ijk} que decide si se pasa o no, por ese arco con el vehículo k. La función objetivo de este problema trata de minimizar el costo total de viaje de todos los vehículos y está dada por:

$$\text{mín} \sum_{i=0}^{N} \sum_{j=0}^{N} \sum_{j \neq i, k=1}^{K} c_{ij} x_{ijk}$$

La función anterior está sujeta a un conjunto de restricciones que modelan el problema, tratando de minimizar el costo total de viaje de todos los vehículos. Considerando también, que la cantidad de tours y cantidad máxima de arcos salientes del depósito o nodo inicial sea un número K. También se asegura que existe un recorrido completo para cada vehículo. Estas restricciones no serán expuestas de manera formal en este documento, sin embargo se comentan de todas maneras para complementar la descripción de este problema de ruteo.

En cuanto al estudio de las variantes, la cantidad de publicaciones asociadas a estos ha ido creciendo a una tasa de 6 % cada año [28], lo que demuestra un interés en la materia. Algunas de las variantes que se han propuesto son el Capacitated VRP [50], que considera una capacidad máxima para los vehículos, Periodic VRP [15], que considera una frecuencia periódica en que operan los vehículos, VRP con múltiples depósitos [56], con distintos tipos de entregas [3], entre otros. También existen otros estudios y revisiones de las múltiples variantes del VRP [11].

2.1.2. Arc Routing Problem (ARP) y Variantes

El Arc Routing Problem (ARP) es un problema de optimización combinatoria de complejidad NP-hard. Busca visitar cada arista de un grafo al menos una vez, minimizando los costos asociados a estas. El ARP presenta distintas formulaciones, restricciones y consideraciones según el problema en cuestión. Algunas de las restricciones comunes incluyen la capacidad de los vehículos [33], el uso de ventanas de tiempo o plazos [9], uso de combustible [66], entre otros objetivos. Aunque el ARP y sus variantes puedan modelar situaciones del mundo real como limpieza de calles, recolección de basura, barrido de nieve, inspección de lineas férreas, entre otros [21], no siempre se consideran situaciones inesperadas o dinámicas que se encuentran en la práctica. Para abordar esta limitación, se propuso el Uncertain Capacitated Arc Routing Problem (UCARP) [55], que incorpora cuatro parámetros estocásticos o aleatorios: presencia y demanda de tareas, presencia de caminos y costos asociados a ellos, además de una capacidad máxima del vehículo. El UCARP es una variación del ARP que tiene en cuenta la incertidumbre en el problema, lo que lo hace más aplicable a situaciones reales.

En 2022 se utilizó el UCARP como base para una tesis de doctorado [2], argumentando que un UCARP no puede resolverse como un CARP estático utilizando técnicas tradicionales como la programación matemática y metaheurísticas. En su lugar, el autor propone una hiperheurística basada en programación genética con optimización en la transferencia de conocimiento. Esta hiperheurística permite abordar de manera más efectiva la incertidumbre presente en el UCARP y encontrar soluciones más adecuadas para problemas de enrutamiento en el mundo real. La técnica propuesta por el autor de esa tesis doctoral involucra una serie de enfoques y componentes clásicos de los algoritmos genéti-

cos, aplicados de manera novedosa para lograr mejores resultados que los que ya han sido reportados en la literatura. Esta técnica implica aumentar la diversidad de la población después de la transferencia de conocimiento para contrarrestar la perdida de diversidad cuando ocurren duplicados durante la transferencia, o bien, eliminar completamente los duplicados para lograr una población inicial de mejor calidad. Un algoritmo transfiere el conocimiento o las características de las políticas de enrutamiento (poblaciones), las cuales son optimizadas mediante una búsqueda tabú, prohibiendo al algoritmo genético recrear las poblaciones ya revisadas con el fin de aumentar la exploración del conjunto solución. También existe un mecanismo de transferencia de conocimiento o de soluciones únicas y de alta calidad entre dos poblaciones, una principal y una auxiliar, las cuales van evolucionando en paralelo. El principal desafío al que se enfrentó con este algoritmo genético fue la presencia de duplicados en las poblaciones iniciales, ya que, disminuyen la diversidad genética de las poblaciones y por lo tanto, su calidad. Los algoritmos y técnicas propuestas mejoraron efectivamente los resultados obtenidos, aunque también es importante considerar las poblaciones iniciales o fuentes de conocimiento, que provienen de otras instancias ya resueltas del mismo problema. También se debe destacar que las soluciones encontradas no corresponden a rutas, sino a políticas de enrutamiento, es decir, lineamientos o reglas que deben seguir las rutas para ser consideradas óptimas.

En esta otra publicación [21], se revisa exhaustivamente la literatura actual sobre el ARP hasta el año 2021, donde se identifican distintas variantes. A continuación se describen algunas de ellas:

- ARP periódico: Esta variante considera un recorrido periódico de las aristas o conjunto de aristas de un grafo, siguiendo un programa o frecuencia especifica. Las tareas de mantenimiento o inspección de lineas eléctricas o redes de vías férreas pueden ser modeladas de esta manera.
- ARP con ganancias: Para esta variante cada arista del grafo tiene asignado un valor o ganancia. Considerando una capacidad máxima o tiempo disponible, se trata de maximizar el valor obtenido al recorrer los arcos. Este modelo es útil cuando se trata de recolectar bienes en cada arco.
- ARP suficientemente cercano: Este problema considera un umbral de proximidad que tiene que cumplirse cuando se visita un arco, por lo que no es necesario visitar cierto conjunto de arcos si estos se encuentran suficientemente cerca de un nodo o del agente que recorre el grafo. Este modelo puede utilizarse para la lectura inalámbrica de sensores en cada arco. Por ejemplo, si existen tres calles paralelas y el vehículo recorre la del medio teniendo en rango la lectura de los sensores de las calles adyacentes, no es necesario pasar por ellas.
- ARP localizado: Esta variante considera un subconjunto de arcos dentro del grafo principal. Se utiliza cuando es necesario enfocarse en una zona especifica dentro de un grafo mas grande.
- ARP con drones: En este caso se utilizan drones en vez de vehículos terrestres, esto trae consigo ventajas y desventajas. Por una parte, los drones no están sujetos a calles o caminos preestablecidos, tampoco se considera el estado de estos caminos o tráfico. Sin embargo, existen otras restricciones como duración de la batería, alcance

y velocidad del drone.

2.1.3. Chinese Postman Problem (CPP) y Variantes

El Chinese Postman Problem es un caso particular del ARP, consiste en encontrar el camino mas corto, para un único agente que recorre todas las aristas de un grafo y vuelve al punto de inicio, es decir, hace un ciclo cerrado. Este problema se puede resolver en tiempo polinomial, usando métodos exactos, cuando el grafo es dirigido, no dirigido y mixto [20]. Un grafo se considera dirigido cuando solo se puede recorrer una arista en una dirección, es no dirigido cuando se pueden recorrer en ambas direcciones y mixto es un grafo que tiene ambos tipos de aristas.

La figura 2.2 describe visualmente un problema de tipo Chinese Postman Problem y también sirve para describir el problema ARP expuesto en la sección 2.1.2.

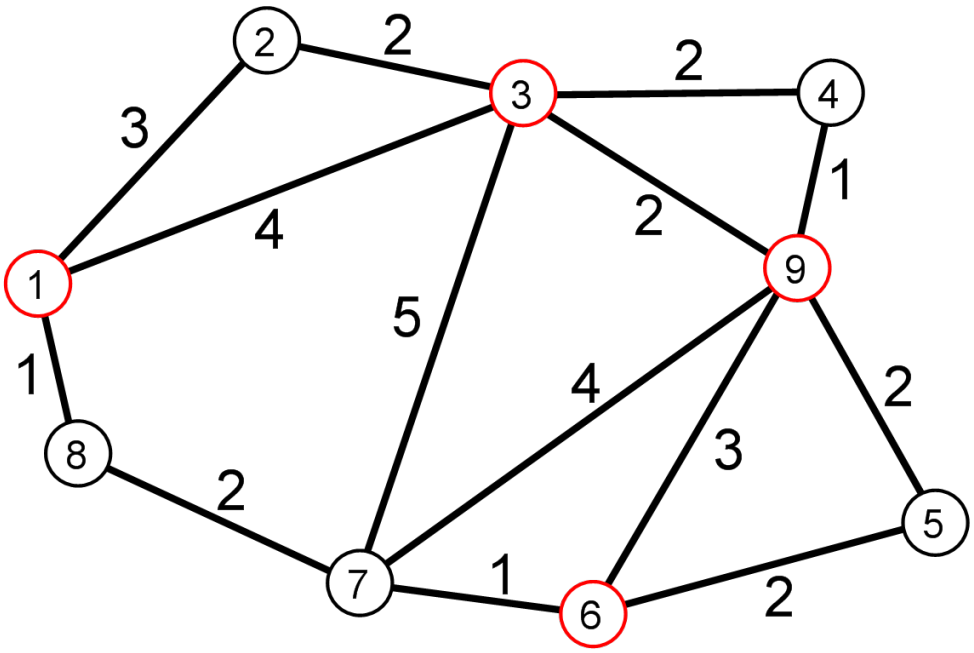


Figura 2.2: Ejemplo de CPP. Fuente: Wikimedia Commons

En la figura se muestra un grafo no dirigido que representa una instancia de Chinese Postman Problem. Los números de los arcos representan el peso de cada uno, mientras que los números de los nodos es un identificador. Los nodos destacados en rojo no tienen un significado especial para describir la instancia.

A pesar de que este problema fue propuesto por primera vez en 1962 por Guan [34], sigue siendo estudiado hoy en día en otras de sus variantes [43]. En este ultimo trabajo, se plantea un algoritmo heurístico matemático para resolver una variante del problema llamada Windy CPP, donde además en este caso, existen jerarquías de aristas. En el Windy

CPP, cada arista tiene un costo distinto según el sentido en que se recorra, y permite modelar situaciones como el trafico de una carretera o una ruta con viento. Cuando se agregan jerarquías, las aristas tienen un nivel de importancia y deben ser visitadas primero. La técnica utilizada por Keskin [43] en su trabajo se basa en una adaptación del algoritmo heurístico por capas propuesto por Dror et al. [27]. La adaptación reemplaza el algoritmo Blossom [63] que busca la ruta mas corta por una formulación matemática lineal entera no heurística. La razón de este cambio es la geometría asimétrica propia del problema, el algoritmo Blossom original requiere que el grafo tenga arcos opuestos con diferentes costos, es decir, que sea simétrico.

Existen algunas variantes simples como el Open CPP [67], que elimina la restricción de terminar en el mismo nodo inicial. Hay otras mas complejas como el General Routing problem, que permite modelar situaciones mas complejas y adaptar el problema a situaciones concretas del mundo real, esta variante permite modelar una flota de distintos tipos de vehículos, con distintos horarios, capacidad, servicios o localizaciones de los depósitos [31].

2.2. Recolección de Residuos Domiciliarios

En esta sección, se presentan una serie estudios sobre situaciones del mundo real que fueron modeladas según los problemas de ruteo mencionados anteriormente y como fueron resueltas aplicando distintos métodos. Además, se expone el uso de nuevos sistemas tecnológicos que fueron implementados para para mejorar la gestión de desechos. Estos estudios no se limitan a la recolección de estos, sino que abarcan desde la generación hasta la eliminación de la basura. También involucran elementos que pueden servir para este trabajo, como el uso de algoritmos para la generación de rutas, selección de un conjunto de calles para aplicar una solución o recolección colaborativa.

2.2.1. Implementación basada en ARP

En un estudio aplicado sobre la optimización de rutas de recolección de residuos solidos realizada por Vecchi et al [68] se plantea una metodología para resolver instancias grandes para este problema modelado como un VRP. Su metodología involucra dos modelos matemáticos y un algoritmo exacto y se divide en tres fases secuenciales, es decir, que la salida de un paso es la entrada del paso siguiente. En la primera fase, se utiliza un modelo adaptado del problema de la p-mediana, conocido también como el problema de localización de instalaciones, para agrupar calles (arcos) que serán atendidas por cada vehículo de recolección, considerando la demanda de residuos y la capacidad de los camiones. La segunda fase aplica un modelo de Programación Lineal Entera Mixta (MILP), adaptado del problema de Ruteo de Arcos Capacitados (CARP), para minimizar la distancia total recorrida por cada camión en los grupos de arcos determinados anteriormente. La tercera fase emplea una versión adaptada del algoritmo de Hierholzer para establecer la secuencia de arcos a recorrer, generando así la ruta para cada vehículo. Este enfoque secuencial resultó ser eficiente al ser probado con datos reales en la región central de Campo Mourão, Paraná, Brasil. El objetivo era planificar la recolección y transporte de residuos para minimizar las distancias recorridas, así como los costos de mantenimiento y combustible de

los vehículos, lo que a su vez contribuye a la reducción de las emisiones de dióxido de carbono. Aunque la investigación se centró en una zona específica, la metodología puede aplicarse a otras áreas simplemente ajustando los datos de entrada.

El algoritmo de Hierholzer tiene un papel crucial en la tercera fase del método propuesto por Vecchi et al. para la optimización de rutas de vehículos. Originalmente diseñado para encontrar un circuito euleriano en un grafo, es decir, una ruta que visite cada arco exactamente una vez, este algoritmo se adapta en el trabajo de Vecchi et al. para abordar las particularidades del problema de recolección de residuos. En este contexto, el algoritmo se modifica para comenzar y terminar en puntos diferentes, adaptándose a las necesidades de rutas de recolección que no necesariamente son circuitos cerrados. Esta adaptación es esencial para convertir los grupos de arcos obtenidos en las fases anteriores en una secuencia lógica y eficiente de recolección, asegurando que cada calle se visite una sola vez y de la manera más eficiente posible. El uso de este algoritmo permite obtener una ruta que finalmente puede ser recorrida y utilizada por los vehículos de recolección.

2.2.2. Implementación basada en VRP

En un estudio realizado en 2021 [1], se propone el uso del VRP en un sistema que emplea dos submodelos. El primer submodelo se encarga de la gestión de una flota de vehículos recolectores, mientras que el segundo se ocupa de asignar recursos a los centros de reciclaje para maximizar la cantidad de bienes recuperados. Este enfoque innovador permite una mayor eficiencia en la recolección y el reciclaje de materiales, lo que contribuye al desarrollo sostenible. Por otro lado, en un estudio de 2020 [59], se propone un sistema de guía para vehículos de emergencia basado en la lógica difusa. Este enfoque utiliza información en tiempo real obtenida de sensores ubicados en las calles para generar rutas óptimas para vehículos de emergencia, como ambulancias o camiones de bomberos. Al proporcionar rutas más rápidas y eficientes, este sistema puede mejorar significativamente la capacidad de respuesta en situaciones de emergencia y, en última instancia, salvar vidas. Ambos proyectos están relacionados con el concepto de Smart City, que se refiere a la aplicación de tecnologías de la información y la comunicación (TIC) para mejorar la calidad de vida de los ciudadanos y garantizar un desarrollo sostenible. Estos proyectos y sistemas han sido implementados con éxito y mencionados en una publicación [10] que recopila ejemplos de soluciones innovadoras en el ámbito de las ciudades inteligentes.

Respecto a los modelos del primer trabajo mencionado [1], estos tienen la particularidad de tener en consideración parámetros inciertos, a diferencia de otros estudios que tienen un enfoque determinista. El segundo submodelo maximiza el valor de los materiales reciclados mediante una mejora en la manera en la que estos se separan, considerando una tasa de salida variable (estocástica) de las instalaciones de separación. Dado que la función objetivo del segundo submodelo no es determinista, sino probabilístico, este es resuelto mediante la utilización de programación con restricciones de probabilidad, esto es porque no se puede predecir con exactitud la cantidad de residuos separados en las instalaciones. Para identificar la mejor solución y evaluar el rendimiento del algoritmo se utilizan cuatro metaheurísticas; Simulated Annealing [44] y Algoritmo Genético [32] en sus formas clásicas, y dos algoritmos híbridos propuesto por los autores originales del trabajo men-

cionado: Algoritmo Genético-Simulated Annealing y Algoritmo Genético-Particle Swarm Optimization [1].

El modelo del segundo trabajo [59], fusiona los datos obtenidos por una red de sensores mediante lógica difusa actuando como puente entre la base de datos y el motor de ruteo OSRM (Open Source Routing Machine), este modelo actúa en dos pasos. El primero recoge los distintos parámetros que se miden con el fin de detectar congestiones de tráfico, siendo estas las emisiones de dióxido y monóxido de carbono, nivel de ruido y temperatura. Además se desarrolló una aplicación para smartphones Android que recoge información sobre los bloqueos o cortes de carretera, lugar y duración de congestión de trafico. La interpolación entre los valores leídos por los sensores y la información obtenida por los teléfonos inteligentes de los distintos usuarios en las carreteras, junto con una serie de reglas en el modelo de lógica difusa, dan lugar a un estimado de la congestión de trafico en un lugar determinado. Dicho valor sirve como entrada para la segunda fase del modelo, que utiliza este estimado para obtener una ruta que evite las calles congestionadas de la ciudad. El motor OSRM utiliza un algoritmo Dijkstra multinivel junto con información de Open Street Maps (OSM) para generar las rutas. Este algoritmo es una variación optimizada del Dijkstra original, donde el grafo original se divide en jerarquías o niveles, donde cada uno es reducido para eliminar nodos que no son necesarios para la búsqueda. Este acercamiento es especialmente útil cuando el grafo tiene una estructura jerárquica natural, como en el caso de las redes viales.

El sistema que se menciona en el párrafo anterior fue puesto a prueba en la India, aunque no se especifica en que ciudad específicamente. Los autores explican que el sistema funciona como se esperaba y fue capaz de guiar a un vehículo de emergencia a través de la ruta mas corta y con menos congestión. Como trabajo futuro proponen evaluar la veracidad de los datos obtenidos desde distintas fuentes para lograr un sistema mas sofisticado y confiable.

El VRP también ha sido utilizado como base para maximizar la cantidad de basura recogida en la ciudad de Bogotá, Colombia. En el trabajo publicado en 2016 [57] se crea un modelo de programación entera mixta (MIP) basado en VRP, que luego es resuelto en dos pasos. El primer paso es agrupar las calles, para luego resolver mediante MIP cada uno de los clusters o grupos. Se plantearon tres formas de agrupar las calles, dos de ellas son métodos heurísticos y la otra requiere de una selección previa de nodos que actúan como concentradores. Los métodos propuestos son: un algoritmo basado en centroide, un algoritmo de barrido y un modelo de localización. Se probaron los tres métodos de clustering en dos instancias de distinto tamaño, donde se concluyó que el método del barrido lograba mejores resultados. Aunque esto ultimo quedó sujeto a validación y pruebas mas exhaustivas.

2.2.3. Recolección de residuos domiciliarios en Argentina

En un trabajo publicado por un grupo de investigadores argentinos [12] se logró desarrollar un algoritmo para generar rutas para los camiones recolectores de basura en la ciudad de Morón, en Buenos Aires, Argentina. Ellos modelaron la red vial de la ciudad

como un grafo dirigido abierto, esto significa que todas las calles tienen un sentido y los camiones no parten ni terminan en un nodo fijo. Este caso se considera como un Rural Postman Problem, el cual se define como un problema en donde se tiene que recorrer al menos una vez un subconjunto de arcos obligatorios dentro de todo el grafo [16]. El rural Postman Problem se considera como una generalización del Chinese Postman Problem. En el trabajo de Braier et al. existe un conjunto de arcos obligatorios, que son las calles a recorrer y un conjunto de arcos opcionales que sirven para ceñirse a las leyes del tránsito de Argentina, como por ejemplo, la prohibición de virar a la izquierda en intersecciones señalizadas con semáforos o vueltas en U en avenidas. Además estos arcos "fantasma" se utilizan para corregir la ruta cuando esta involucra sub-tours, que son circuitos cerrados dentro de la ruta principal.

Este grupo de investigadores utilizó el solver SCIP para resolver el problema modelado con programación en enteros (Integer Programming). Se consideraron dos situaciones, cuando la ruta solución contiene sub-tours y cuando no, para luego comparar los resultados. Se encontró que las soluciones que, por medio de un paso extra en el algoritmo, eliminaban sub-tours de las soluciones, daban soluciones de mejor calidad en menos tiempo. El algoritmo desarrollado fue probado en la ciudad de Morón, que fue dividida en 5 zonas, para contrastarlo con las rutas que utilizaban los recolectores de basura anteriormente. Los resultados fueron favorables en el sentido de que tomaban menos tiempo y recorrían todas las calles.

El modelo planteado en este trabajo se compone de un grafo G=(N,A), que representa un sector de la red vial de la ciudad, donde un conjunto de arcos (A) son las calles y un conjunto de nodos (N) son las intersecciones entre ellas. También se define un subconjunto B para las calles bidireccionales.

La función objetivo del modelo busca minimizar los costos de recorrer todos los arcos que son obligatorios:

$$minimizar \sum w_{ij} \cdot x_{ij}, \forall (i,j) \in A$$

Parámetros:
$B = \{(i,j) \in (N,N)\}$ Conjunto de arcos bidireccionales
$I = \{i \in N\}$ Conjunto de nodos que pueden ser nodos de inicio
$T = \{i \in N\}$ Conjunto de nodos que pueden ser nodos de término
w_{ij}: Costo de recorrer en el arco (i,j).

Variables de decisión:
$x_{ij} \in Z_+$: Cantidad de veces que el arco (i,j) es recorrido
$s_i \in \{0,1\}$: Variable binaria que representa si el nodo i es el inicio de la ruta.
$t_i \in \{0,1\}$: Variable binaria que representa si el nodo i es el fin de la ruta.

Restricciones:

$$x_{ij} \geq 1, \quad \forall (i,j) \in A \setminus B \tag{1}$$

$$x_{ij} + x_{ji} \geq 1, \quad \forall (i,j) \in B \tag{2}$$

$$s_i + \sum_{(j,i)\in A} x_{ji} = \sum_{(i,j)\in A} x_{ij} + t_i, \quad \forall i \in I \tag{3}$$

$$\sum_{(j,i)\in A} x_{ji} = \sum_{(i,j)\in A} x_{ij} + t_i, \quad \forall i \in V \setminus I \tag{4}$$

$$\sum_{i\in I} s_i = 1 \tag{5}$$

$$\sum_{i\in N} t_i = 1 \tag{6}$$

$$x_{ij} \in \mathbb{Z}_+, \quad \forall (i,j) \in A \tag{7}$$

$$s_i \in \{0,1\}, \quad \forall i \in I \tag{8}$$

$$t_i \in \{0,1\}, \quad \forall i \in N \tag{9}$$

Las restricciones anteriores modelan el problema como un ARP donde el recorrido o tour debe ser abierto, es decir, que su nodo final no sea el mismo nodo de inicio. La primera restricción asegura que se recorran todos los arcos obligatorios unidireccionales, mientras que la segunda, lo hace con los bidireccionales, pero estos pueden recorrerse en cualquier sentido. La tercera y cuarta restricción son restricciones de flujo, estas aseguran que la cantidad de veces que se ingresa a un nodo sea constante a lo largo de todo el recorrido, esto incluyendo el nodo inicial y final. La quinta restricción asegura que solo exista un nodo inicial, lo mismo ocurre con el nodo final en la sexta restricción. Las ultimas tres restricciones definen el conjunto de valores que pueden tomar las variables de decisión, siendo los números enteros positivos para la cantidad de veces que se puede pasar por cada arco, y 0 o 1 para las variables de decisión que definen si un nodo se considera inicial o final.

2.2.4. Aplicación de la Tecnología Blockchain en la Gestión de Residuos

En un articulo presentado en 2020 [6], se propone un estado del arte sobre el uso de la tecnología blockchain aplicada al manejo de desperdicios. Este trabajo se basa en el concepto de la economía circular, la cual consiste en el reciclaje y recuperación de los productos al fin de su vida útil, con el fin de reducir la cantidad de residuos que terminan en los vertederos. Para lograr esto, la trazabilidad de los productos y materiales es fundamental en cada etapa de su ciclo de vida, desde la producción hasta su desecho, pasando por el consumo. Para lograr esto se busca registrarlos en un blockchain o cadena de bloques.

La tecnología blockchain permite registrar archivos en una base de datos distribuida de una manera transparente, confiable e inmutable, mantenida por los mismos usuarios que la utilizan. Esta tecnología se destaca como una opción prometedora y efectiva para

monitorear y controlar adecuadamente la gestión de residuos y el reciclaje. El uso de un blockchain puede proporcionar beneficios como la trazabilidad de los residuos, la prevención de fraudes, una manera efectiva de combatir los vertederos ilegales y fomentar el reciclaje a través de recompensas.

En la investigación anterior se revisaron 69 trabajos relacionados al tema, que fueron publicadas entre 2017 y 2021. En estos trabajos se proponen distintos sistemas que pueden ayudar a que la recolección de residuos sea mas colaborativa y democrática, involucrando todos los factores y procesos que participan en las fases del manejo de residuos. Se destacan los procesos de producción y consumo de bienes que luego se transforman en desechos, los cuales son recolectados, adquiridos y transportados, para luego ser clasificados, segregados y eliminados, con pequeño un porcentaje que es reciclado y reiniciando el ciclo. Otro aspecto destacable es que los desechos se asocian con datos, los cuales son fiscalizados, comprados y vendidos en la blockchain por quienes participan del sistema propuesto.

Esta tecnología, al ser experimental y poco conocida, presenta ciertas limitaciones que no permiten una gran adopción por el publico general. Del total de proyectos revisados, el 30 % falló, solo el 13 % logró desarrollar un prototipo, mientras que el solo el 9 % permanece en uso. A pesar de esto, la creciente cantidad de proyectos en los últimos dos años muestra que existe un mayor interés en el blockchain.

2.2.5. Tecnologías y Sistemas Computacionales Aplicados al Manejo de Residuos en la India

En otro articulo [5], se revisa el estado del arte de sistemas computacionales aplicados al manejo de residuos en la India. Se plantea que esta actividad en ese país es ineficiente, los camiones recolectores permanecen en funcionamiento durante la carga y descarga, recorren rutas fijas, los procesos se hacen de manera manual ni existe mayor información o datos sobre los desperdicios que se están manipulando, algo muy similar a lo que ocurre en nuestro país. El autor realiza una revisión exhaustiva de los métodos y técnicas existentes utilizados para la gestión de desechos, además propone un sistema que hace uso de tecnologías y técnicas avanzadas. Los sistemas que se revisan en el articulo mencionan el uso de tecnologías RFID y/o GPS [37, 29] para la identificación de contenedores, también menciona el uso sensores dentro de ellos [38] para generar información sobre el tipo y peso del contenido. El nuevo sistema planteado por al autor implica que esta información sea consumida por un servidor central que despacha los camiones recolectores en función del contenido y cantidad de contenedores, siguiendo rutas óptimas usando información de trafico. Uno de los trabajos [4] citados en la publicación anterior hace mención a la generación de rutas óptimas de recolección de basura utilizando un algoritmo genético, mientras que otro [39] habla de un algoritmo de enjambre de partículas (PSO) modificado. Las conclusiones de este estudio mencionan que la administración de los desechos solidos municipales es una actividad multidisciplinaria que involucra toda la cadena logística y múltiples actores. No solo es tarea del gobierno implementar nuevas tecnologías para optimizar esta actividad, sino también de cada persona, por lo que, la cultura del reciclaje y concienciación publica es importante. Cabe destacar que es deber de cada uno, como ciudadano, separar correctamente los desechos en biodegradables, inertes

y reciclables para una mejor utilización de los sistemas, tanto actuales como nuevos.

2.3. Métodos de Resolución de Problemas de Optimización

En materia de optimización, existen varias formas de resolver los problemas que se nos presentan, estos métodos pueden ser clasificados en dos categorías, métodos exactos y métodos aproximados. En esta subsección se explica en que consiste cada uno y se nombran algunos ejemplos.

2.3.1. Métodos de Resolución Exactos

Este tipo de métodos de resolución realizan una búsqueda exhaustiva del espacio de búsqueda de un problema, permitiendo encontrar la solución global a este, siendo esta su mayor ventaja y utilidad. Los métodos exactos suelen ser costosos en términos de tiempo, por lo que no se recomiendan para resolver problemas demasiado grandes o complejos como los NP-hard. Existen una variedad de algoritmos y técnicas que se consideran métodos exactos, algunos ejemplos son los algoritmos Branch and Cut, Branch and Price o Branch and Bound, que dividen el problema base en subproblemas que son resueltos y optimizados independientemente. Otras técnicas son la programación dinámica y programación lineal [42].

Actualmente existe software altamente especializado para resolver problemas de programación matemática. Uno de los mas conocidos y eficientes es Gurobi [40], un optimizador construido para aprovechar al máximo las capacidades del hardware en el cual está ejecutándose y soporta múltiples tipos de problemas, ya sean de programación lineal, cuadrática, entera o entera mixta, con y sin restricciones, entre otros. Este optimizador solo requiere que el usuario le entregue el modelo matemático del problema, esto es, conjuntos, restricciones, parámetros e instancias, para resolverlo internamente y entregar la solución. Actualmente Gurobi tiene soporte para distintos lenguajes de programación como Python, C, R, Java o MATLAB. También existen otros optimizadores como CPLEX [35] o COPT [62], que son desarrollados y mantenidos por otras compañías.

2.3.2. Métodos de Resolución Aproximados

Los métodos aproximados son especialmente útiles cuando se trata de problemas con un espacio de búsqueda demasiado grande para ser explorado por completo. Este enfoque utiliza metaheurísticas para obtener soluciones subóptimas en un tiempo razonable. Existe una gran variedad de métodos aproximados, y estos, se pueden clasificar en las siguientes categorías en base a la cantidad de soluciones que se manejan en cada paso de la búsqueda:

- Trayectoria: A partir de una solución inicial, estos algoritmos exploran las soluciones vecinas o adyacentes a esta, evaluando si estas son de mejor calidad o no según un criterio de evaluación. Cuando se detecta que una solución vecina es mejor, se utiliza esta para explorar su vecindario, generando así, una trayectoria por el espacio de soluciones, de ahí el nombre de esta clasificación. Algunos algoritmos de trayectoria son Simulated Annealing, Búsqueda Tabú o Hill Climbing.

- Población: Este tipo de algoritmos comienza con una población o conjunto de soluciones iniciales, las cuales, mediante distintos mecanismos van siendo modificadas y evaluadas para encontrar una solución aceptable. Muchas veces estos algoritmos están inspirados en la naturaleza, como el comportamiento de las abejas, peces u hormigas, o basados en fenómenos físicos como la gravedad o resonancia. Algunos ejemplos son los algoritmos genéticos, el Particle Swarm Optimization o los derivados del Ant Colony Optimization. Este ultimo es de gran interés para este trabajo, por lo que será explicado en mayor detalle posteriormente.

Mientras que pueden ser clasificados en las siguientes categorías con respecto a la forma en que se generan nuevas soluciones en la búsqueda:

- Perturbativos: Los algoritmos perturbativos se pueden considerar de trayectoria o de población, ya que, su principal característica es que introducen perturbaciones a las soluciones para explorar el espacio de búsqueda o bien, para escapar de óptimos locales. Algunos mecanismos perturbativos muy utilizados en algoritmos genéticos son los operadores de cruce y mutación, que intercambian elementos de un grupo de soluciones, para luego evaluar si estos cambios mejoran o no las soluciones involucradas.
- Constructivos: Los algoritmos constructivos, como su nombre lo indica, van construyendo las soluciones a medida que se exploran. Que elemento se agrega a la solución varía de un algoritmo a otro y como se evalúa este elemento. Ant Colony Optimization, algoritmo que será explicado en la sección 2.3.3, se considera dentro de esta categoría.

2.3.3. Ant Colony Optimization

Esta familia de algoritmos se basan en el comportamiento de las colonias de hormigas, en donde, cada hormiga va depositando feromonas en el camino que recorre con el fin de comunicar información sobre el camino o la fuente de comida a las demás hormigas. De esta manera se va creando un conocimiento colectivo compartido por toda la colonia que permite encontrar caminos favorables. Este sistema de comunicación se conoce como Estigmergia e inspiró al investigador Marco Dorigo para proponer esta metaheurística en su tesis doctoral en 1992 [17].

Los algoritmos Ant Colony Optimization (ACO) se consideran de tipo constructivos y poblacionales, esto quiere decir que no necesitan de soluciones predefinidas, sino que se van construyendo y se agrupan en poblaciones [26]. En estos algoritmos, las hormigas o agentes, van construyendo las soluciones y depositan feromonas en los caminos que deciden recorren entre nodos, influyendo en las decisiones de las demás hormigas. La manera en que una hormiga decide si dirigirse a un nodo u otro es probabilística y depende de la *información previa*, conocida y estática sobre las distancias o pesos entre nodos, y la *información dinámica*, que va cambiando según el aprendizaje de la colonia representado en las feromonas que depositan las hormigas y su tasa de evaporación; un mecanismo

que permite "olvidar" malos caminos o facilitar la exploración de otros al evitar una convergencia prematura a una solución poco favorable. Cabe destacar que, los caminos con mayor cantidad de feromonas representan los caminos incluidos en mejores soluciones.

La analogía de las colonias de hormigas buscando caminos óptimos para buscar comida hace que este tipo de algoritmos sea particularmente adecuado para resolver problemas de ruteo como el Travelling Salesman Problem, Vehicle Routing Problem, o en nuestro caso, el Arc Rounting Problem, aunque también puede extenderse a otros tipos de problemas. En la literatura, se pueden encontrar aplicaciones de los algoritmos ACO para problemas de asignación como Asignación Cuadrática [54], Coloreo de Grafos [22], problemas de planificación como Job Shop Scheduling [18] o problemas de subconjuntos como el Set Covering Problem [51], entre otros.

Como se mencionó anteriormente, los algoritmos ACO son una familia de algoritmos. Estos son utilizados principalmente para problemas de optimización combinatorial estáticos, es decir, que no cambian su estructura ni restricciones a lo largo del tiempo, todos tienen en común una estructura básica compuesta del siguiente pseudocódigo que fue adaptado del propuesto originalmente [26] por sus autores para dar mas detalles sobre los parámetros y su utilización:

Algorithm 1: Pseudo-código de Metaheurística ACO

```
1  procedure MetaheuristicaACO
2  Inicializar parámetros: α, β, τ0, ρ, #hormigas
3  Inicializar feromonas τ en τ0
4  Inicializar información heurística η (opcional)
5  while (no se complete condición de término) do
6      foreach hormiga en el conjunto de hormigas do
7          Inicializar hormiga en nodo aleatorio
8          while (solución no completa) do
9              Seleccionar siguiente nodo
10             Visitar nodo
11         end
12         Evaluar calidad de la solución
13         Aplicar búsqueda local (opcional)
14         if (la nueva solución es mejor que la actual) then
15             Actualizar mejor solución
16         end
17     end
18     Evaporar feromonas τ según parámetro ρ
19     Actualizar feromonas τ en todos los arcos
20 end
```

En primer lugar, se establecen los parámetros como cantidad de hormigas, tasa de evaporación de feromonas (ρ), condiciones de término, alfa, beta y además, la inicialización

de las feromonas iniciales (τ) según un valor inicial τ_0. Si se tiene información disponible sobre el problema a resolver, y esta puede ser utilizada para guiar la búsqueda, esta se utiliza junto a las feromonas. Comúnmente esta información es denominada información heurística (η).

Mientras no se hayan cumplido las condiciones de término, ya sea por cantidad de iteraciones, tiempo, calidad de solución u otro criterio, se ejecutan los pasos principales del algoritmo. Primero se inicializan las hormigas en un nodo aleatorio o bien, en algún nodo específico, luego mientras la solución no esté completa, según como se defina una solución completa, se va construyendo la solución seleccionando y visitando los nodos. Al terminar una solución se evalúa si calidad y opcionalmente se aplica una búsqueda local para mejorarla. Si la nueva solución es mejor que la actual, esta es actualizada. Este proceso ocurre para todas las hormigas. Luego de que estas han construido sus soluciones, se evaporan las feromonas según la tasa de evaporación (ρ) y se actualizan las feromonas (τ) de cada arco recorrido [26].

El mecanismo de selección del siguiente nodo ocurre en base a una regla de transición pseudoaleatoria. Esta regla asigna, a cada nodo vecino del nodo actual, una probabilidad de ser elegido basada en la siguiente fórmula:

$$p_{ij} = \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum \tau_{in}^{\alpha} \cdot \eta_{in}^{\beta}}$$

En donde p_{ij} es la probabilidad de ir del nodo i al nodo j. η_{ij} y τ_{ij} son la cantidad de feromonas y valor de la información heurística entre el par de nodos i y j, mientras que α y β son parámetros que controlan la importancia o peso de las feromonas e información heurística respectivamente. Comúnmente esta última se calcula como el inverso de la distancia entre i y j. El denominador en la ecuación representa la sumatoria de los caminos posibles a recorrer desde el nodo i, es decir, los arcos hacia los nodos vecinos. Esto nos da una probabilidad normalizada por cada arco candidato, el cual es elegido mediante la generación de un número aleatorio.

Por su parte, el depósito de feromonas es un componente crucial para desarrollar una memoria colectiva entre todos los agente u hormigas, que finalmente es utilizada para encontrar los caminos mas favorables. Este componente se puede separar en distintas fases que ocurren durante toda la construcción de una solución por parte de las hormigas. Las fases del depósito de feromonas son; inicialización, las feromonas de cada arco (i, j) son establecidas en un valor inicial fijo; actualización de feromonas, los arcos de las rutas con mejor calidad aumentan su cantidad de feromonas; evaporación de feromonas, se reducen las feromonas en todos los arcos, independiente si recibieron feromonas o no.

La actualización de las feromonas ocurre según la expresión $\tau_{ij} = \tau_{ij} + \sum_{k=1}^{m} \Delta t_{ij}^{k}$ donde τ_{ij} es la feromona actual del arco (i, j) que va a ser actualizada y Δt_{ij}^{k} es la inversa del largo total del tour de una hormiga k si el arco (i, j) pertenece a dicho tour.

Matemáticamente esta expresión se traduce como:

$$\Delta t_{ij}^{k} = \begin{cases} \frac{1}{C^k}, & \text{si el arco } (i,j) \text{ pertenece al tour } T^k \\ 0, & \text{cualquier otro caso} \end{cases}$$

En este paso, los arcos de las rutas que forman parte de las mejores soluciones reciben una mayor cantidad de feromonas.

Por otro lado, la evaporación de feromonas permite al algoritmo olvidar los caminos menos favorables. Esto se hace mediante la expresión $\tau_{ij} = (1 - \rho)\tau_{ij}$, donde ρ es el parámetro que controla la tasa de evaporación y τ_{ij} es el valor de la feromona para el arco (i, j).

La combinación de los mecanismo de actualización y evaporación permiten un equilibrio entre exploración y explotación en el algoritmo. Esto evita la convergencia prematura a óptimos locales y permite una búsqueda amplia en el espacio de soluciones. El depósito de feromonas en general, es un proceso clave en todas las variantes de algoritmos Ant Colony Optimization.

La figura 2.3 ejemplifica el proceso de depósito de feromonas. Volviendo a la analogía de las hormigas buscando comida, el punto A en la imagen puede representar el nido de las hormigas y el punto B una fuente de comida, mientras que las lineas de colores son los distintos caminos que han encontrado las hormigas para ir de A a B y viceversa. Se puede notar que existen caminos mas marcados que otros, que a su vez, son los caminos mas cortos y eficientes. Estos caminos, al ser los de mejor calidad, reciben una mayor cantidad de feromonas y por lo tanto, mas transitados. Por otra parte, los caminos menos transitados, van perdiendo relevancia.

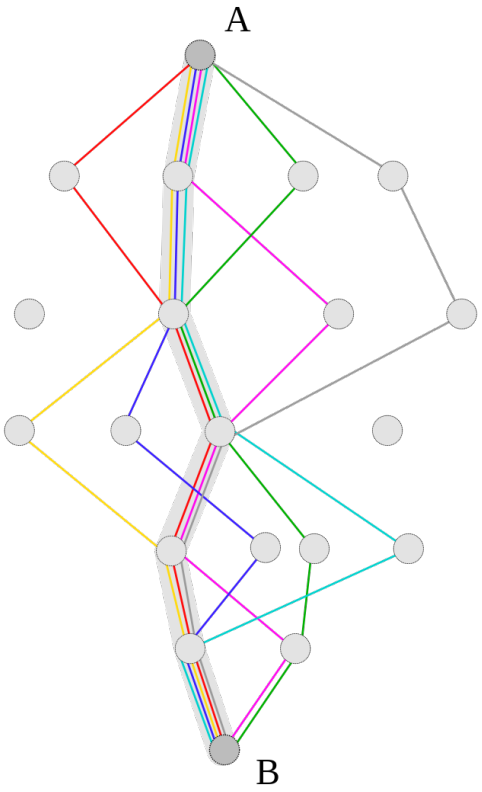


Figura 2.3: Depósito de feromonas. Fuente: Wikimedia Commons

Respecto a las variantes de los algoritmos ACO, lo que los diferencia entre si es la manera en que se implementa cada fase descrita anteriormente. Algunos algoritmos tienen distintas políticas de actualización de feromonas, otros tienen límites superiores e inferiores para estas, o también, agregan un rastro de feromonas adicionales según la mejor solución encontrada, ya sea, local o globalmente. También puede variar la manera en que las hormigas van construyendo las soluciones, estas pueden ser paralelas o secuenciales. Las distintas maneras en que actúan estos componentes dan origen a los diferentes versiones de algoritmos pertenecientes a la familia Ant Colony Optimization.

El primer algoritmo de este tipo fue el Ant System (AS) [17]. Originalmente existieron tres versiones de este algoritmo, sin embargo, se optó por la versión con mejor rendimiento. La diferencia entre las tres era la manera en que se actualizaban las feromonas, las dos con menor performance, Ant-Density y Ant-Quantity, lo hacían luego de cada movimiento de las hormigas. Por otra parte Ant-Cycle lo hacia luego de terminar el ciclo de construcción de caminos y en base a la calidad de estos. Esta última versión es la que se usó como base para los siguientes algoritmos.

Las variantes del AS representan una mejora al desempeño base de este. La primera variante fue el Elitist Ant System (EAS), que agregaba un conjunto adicional de feromonas que reforzaba los arcos según el mejor tour encontrado hasta el momento. Otra variante es el Rank-Based Ant System (AS_{rank}) que asigna un rango a las hormigas y según este rango

es la cantidad de feromonas que depositan en los arcos. Según los resultados obtenidos [14], quienes propusieron esta variante, el algoritmo se desempeña ligeramente mejor que el Elitist Ant System y sustancialmente mejor que el Ant System base.

Una mejora importante fue la variante MAX-MIN Ant System [65]. Esta variante agrega cuatro modificaciones al AS. En primer lugar, solo la mejor hormiga de la iteración o la mejor hasta el momento, es decir, la que haya encontrado la mejor solución, está habilitada para depositar feromonas. Se introducen límites inferiores y superiores para los valores de las feromonas en los arcos, esto es para contrarrestar un efecto secundario de la primera mejora que provoca que todas las soluciones converjan a una solución subóptima prematuramente. La tercera modificación consiste en inicializar las feromonas en el limite superior, junto con una tasa de evaporación pequeña, esto sirve para beneficiar la exploración del espacio de búsqueda, mas que la explotación de una posible solución. Finalmente, cuando no se detecta una mejora en las soluciones encontradas luego de una serie de iteraciones, los valores de las feromonas son reiniciados. Adicionalmente, se introduce un parámetro "a", que sirve para recalcular el valor de los límites inferiores y superiores según la fórmula $lim_i = lim_s/a$.

Existe otra variante que, si bien, está inspirada en el Ant System, agrega nuevos mecanismos que mejoran aun mas el desempeño de este y sus variantes mas directas. Este es el caso del Ant Colony System (ACS) [25], cuya principal característica reside en el mecanismo de construcción de rutas. En dicho mecanismo se considera un parámetro adicional q_0 que actúa como umbral de probabilidad entre un comportamiento u otro, siendo estos la posibilidad de elegir el siguiente nodo como la mejor opción posible en términos de calidad según la información disponible o elegir el nodo según la regla pseudoaleatoria clásica. Adicionalmente, las hormigas pueden remover una pequeña cantidad de feromonas de los arcos i y j luego de haberlo cruzado, esto evita la convergencia prematura a una solución y facilita la exploración de rutas que no han sido visitadas.

Otra particularidad de ACS es al momento de actualizar las feromonas globales, solo la mejor hormiga hasta el momento tiene permitido hacerlo, y lo hace después de cada iteración. Esto ultimo reduce el costo computacional de esta fase del algoritmo desde $O(n^2)$ a $O(n)$, donde n es el tamaño del espacio de búsqueda, haciendo que sea mas eficiente y tenga mejores resultados en instancias de Travelling Salesman Problem con mas de 100 nodos.

Cabe destacar que, para ACS, existe un valor recomendado para las feromonas iniciales. Este valor está dado por la expresión $1/nC^{nn}$, donde n es la cantidad de nodos y C^{nn} es el largo del *nearest neighbor tour*, que es una heurística que crea una ruta en base al nodo mas cercano no visitado. Esto en el contexto del Travelling Salesman Problem, que fue el problema base sobre el cual se construyó este algoritmo.

Lo que tienen en común todos los métodos de resolución, son los parámetros α, β, ρ y τ_0, además de los parámetros de control como cantidad de hormigas e iteraciones. En primer lugar, α y β son valores reales positivos que agregan o reducen el peso que tienen las feromonas e información heurística respectivamente. Es decir, si el valor de α

es mayor al de β, la información heurística tendrá mayor relevancia a la hora de calcular la probabilidad para seleccionar el siguiente arco a recorrer. Por otra parte, el parámetro ρ es la tasa de evaporación de las hormonas, como fue ya mencionado, que permite al algoritmo "olvidar" los caminos que son de mala calidad. Mientras que el parámetro tau está asociado al valor inicial de las feromonas y actúa como valor de referencia para la actualización de estas.

3. Definición del Problema

La tarea de generar rutas óptimas para uno o mas vehículos pertenece a la familia de problemas de optimización combinatoria. Estos consisten en hallar una combinación óptima dentro de un conjunto finito de elementos. Lo que se trata de optimizar puede variar según el objetivo del problema, como minimizar el peso de un conjunto de objetos o maximizar su valor, como en el caso del problema de la mochila. Otro ejemplo es la búsqueda de rutas, como encontrar la ruta más corta para visitar un conjunto de ciudades en el problema del vendedor viajero, o un conjunto de rutas que maximicen la cantidad de clientes atendidos, como en el VRP mencionado anteriormente. Cuando el espacio de búsqueda es pequeño, es factible revisar y evaluar todas las soluciones posibles para determinar la mejor, sin embargo, cuando el número de variables aumenta, se produce una explosión combinatoria, lo que significa que el problema crece de tal manera que es inviable analizar todo el universo de búsqueda en un plazo razonable.

En general, el problema de optimizar las rutas de los camiones recolectores de desechos se considera como un Vehicle Routing Problem (sección 2.1.1), es decir, una flota de camiones asignados a ciertos sectores de la ciudad tienen que seguir distintas rutas para recoger los desechos de las casas, minimizando los costos asociados a estas rutas. Sin embargo, este problema también puede ser modelado como un Arc Routing Problem (sección 2.1.2), de hecho, es la manera mas conveniente, ya que, los camiones deben pasar por todas las calles (aristas de un grafo) para recoger los desechos de todas las casas [36]. También se podría modelar como un Chinese Postman Problem (sección 2.1.3), aunque en nuestro caso en particular no sería lo mas apropiado, porque esto implica que los nodos de entrada y salida sean el mismo, y también, que las aristas solo se puedan recorrer una sola vez. Por motivos de claridad y entendimiento, nuestra situación se modelará como un caso particular del Arc Routing Problem cuyas particularidades serán descritas en esta sección.

3.1. Grafo de la Red Vial

Antes de comenzar a resolver el problema, es necesario establecer un conjunto de instancias para realizar pruebas. Nos referimos a un conjunto de datos que representan la red vial de una ciudad modelada como un grafo y un formato para representarlos.

La figura 3.1 presenta una instancia de ejemplo que nos podría servir para representar el problema. La figura muestra una grilla de 4x4 que contiene calles unidireccionales y bidireccionales, una avenida y las intersecciones entre ellas, elementos que se esperarían en una instancia real. Las calles son representadas por los arcos del grafo y la nodos representan las intersecciones de estas calles. Los arcos de los bordes exteriores de la grilla se recorren en sentido de las agujas del reloj, mientras que las filas 2 y 4 representan calles de doble sentido y la columna 3 representa una avenida. Este grafo también muestra las posibles intersecciones que se podrían encontrar en una red vial con estos elementos, es decir, intersecciones entre calles bidireccionales y una avenida, como en el nodo 7, entre una calle bidireccional y una unidireccional, como en el nodo 10 o entre calles unidireccio-

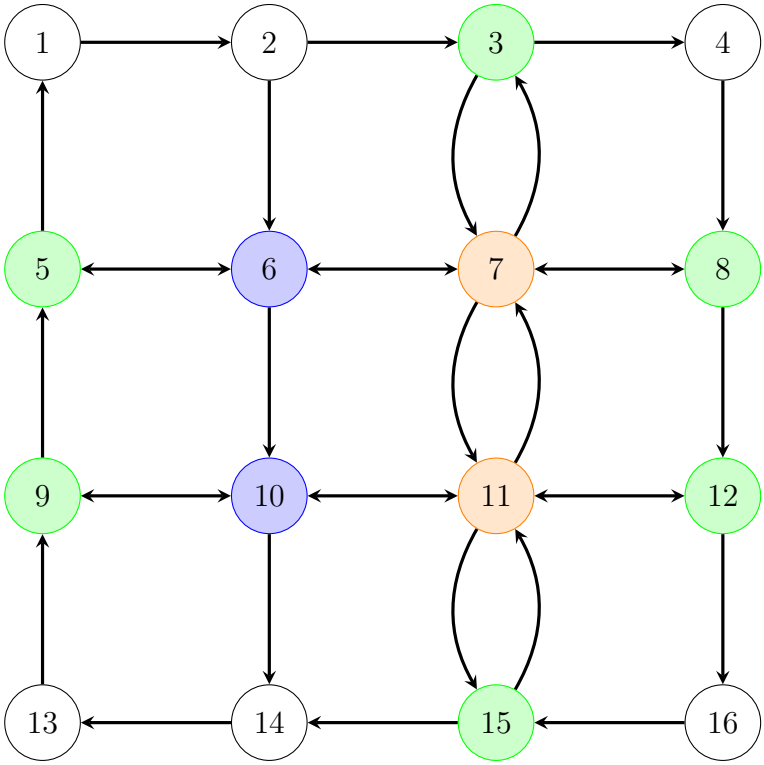


Figura 3.1: Ejemplo de Grafo de una red vial.

nales, nodos 12 o 15. Además se considera un subconjunto arbitrario de nodos de entrada compuesto por los nodos o intersecciones exteriores, los nodos de salida se encuentran dentro de este mismo subconjunto. La razón de por que se usa este tipo de instancia como ejemplo es porque contiene los tipos de calles e intersecciones que se podrían encontrar en una zona de una ciudad real.

El grafo presentado en la figura 3.1 nos sirve como base para poder representar una red vial. Sin embargo, para poder modelar el problema como un grafo es necesario realizar una transformación a los nodos destacados para prohibir ciertos virajes al recorrer la red, como vueltas en U o ingresar en una calle contra el tránsito. Esta transformación implica agregar nuevos nodos y arcos solo donde se pueda transitar. Por ejemplo, en la figura 3.2, se muestra la transformación de la intersección 10 de la figura 3.1, la cual es una intersección entre una calle unidireccional y una bidireccional. El nodo 10 es dividido en 6 subnodos que permiten la correcta circulación por la intersección evitando virajes en U. Los arcos generados representan las transiciones válidas dentro de la intersección y se denotan en lineas punteadas. Esta transformación se puede utilizar para representar otras restricciones comunes del tránsito como virajes prohibidos o señales de no entrar.

Siguiendo la misma lógica, es posible aplicar la transformación a todas las intersecciones de la red. La figura 3.3 muestra la transformación completa del grafo presentando en la figura 3.1. Se consideró para este ejemplo solo la restricción de viraje en U y el respeto al sentido de las calles en la red.

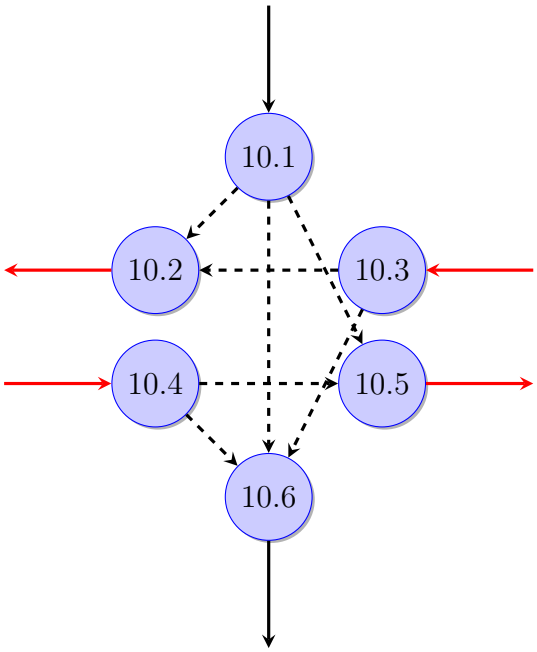


Figura 3.2: Transformación de nodos de intersecciones.

Notar que el tamaño de la red aumenta considerablemente en número de nodos y de arcos por lo que, si bien, este grafo nos permite modelar la red y sus restricciones de manera adecuada, incrementa su complejidad en términos de resolución. Esto no solo se debe al aumento de nodos y arcos, sino también a la creación de arcos en las intersecciones que no representan calles de la red y que por lo tanto no deben ser recorridos en su completitud.

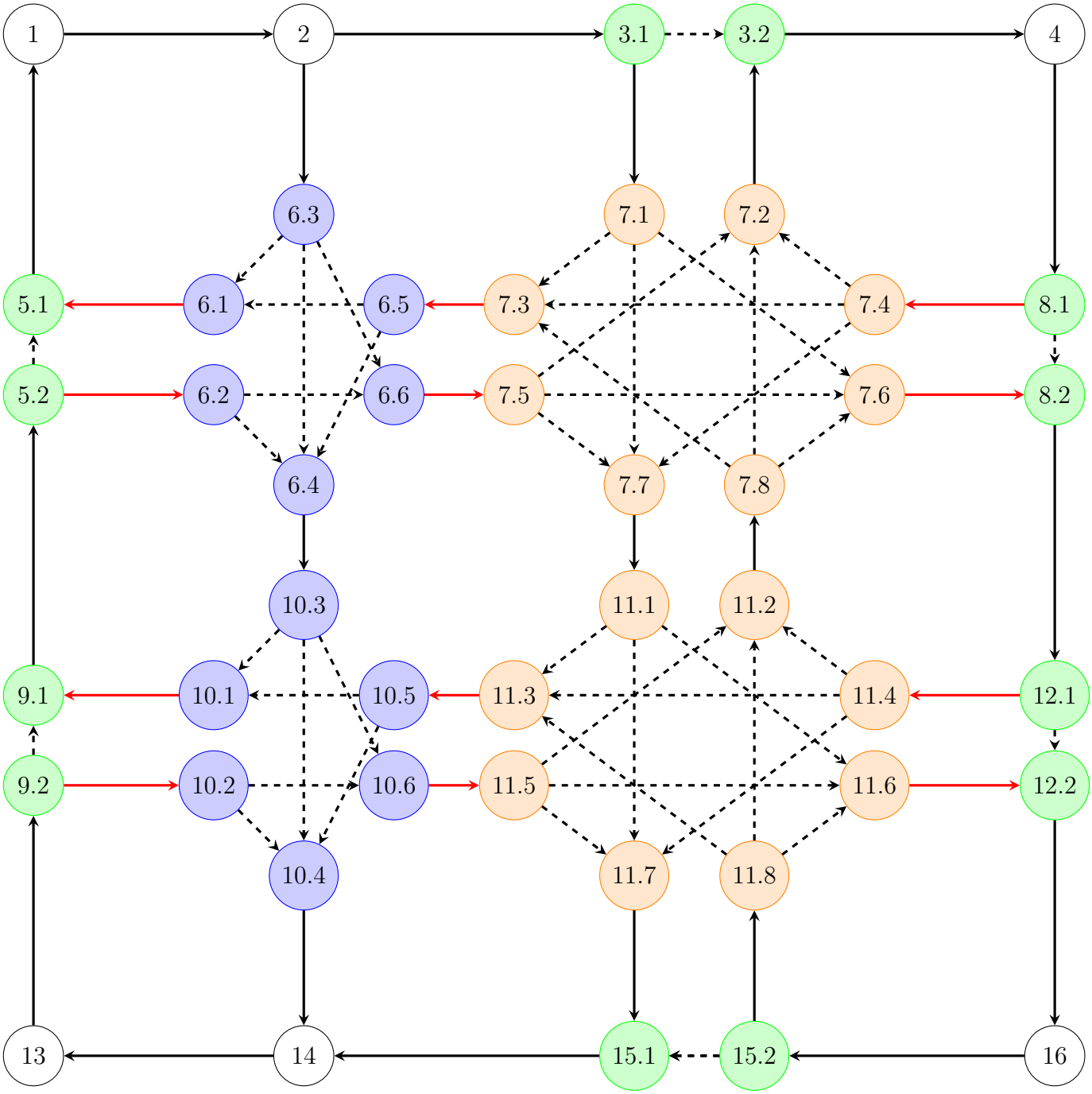


Figura 3.3: Grafo transformado.

3.2. Modelo Matemático

En base al modelo matemático planteado por Braier et al. [12] se plantea un modelo ajustado a la situación abordada en este trabajo. A continuación, se listan las principales características del problema a resolver:

- Existe un conjunto de calles o sector ha sido seleccionado previamente para un vehículo.
- Se asume que los desechos asociados a este sector caben sin problemas en el camión.
- El camión recoge toda la basura de una calle cuando pasa por esta.
- Consideramos un peso o costo asociado a cada calle, compuesto por el tiempo que toma recorrerla, más la cantidad de desechos en esa calle.
- Las calles se pueden recorrer mas de una vez.
- El costo de la basura solo se considera al pasar por una calle la primera vez, luego solo se considera el costo que toma recorrerla.
- Se consideran calles de uno o dos sentidos.
- Se consideran leyes y señales del tránsito.
- Existe un conjunto de nodos donde el camión puede entrar o salir de la zona.

Dado el grafo G = (N, A), que representa un sector de la red vial de una ciudad, donde N es el conjunto de nodos y A es el conjunto de arcos.

Parámetros:
$E = \{(i,j) \in A\}$: Conjunto de arcos dirigidos obligatorios.
$N = \{(i,j) \in A\}$: Conjunto de arcos dirigidos no obligatorios.
$B = \{((i,j),(k,m)) \in (A,A)\}$: Representa una calle bidireccional.
$I = \{i \in N\}$: Conjunto nodos que pueden iniciar una ruta.
$T = \{i \in N\}$: Conjunto de nodos que pueden terminar una ruta.
c_{ij}: Costo de recolectar el arco (i,j).
p_{ij}: Costo de recorrer en el arco (i,j).

Variables de decisión:
$x_{ij} \in Z_+$: Cantidad de veces que el arco (i,j) es recorrido
$s_i \in \{0,1\}$: Indica si el nodo i es el inicio de la ruta.
$t_i \in \{0,1\}$: Indica si el nodo i es el fin de la ruta.

Modelo

$$minimizar \sum_{(i,j)\in E} c_{ij} \cdot min(1, x_{ij}) + \sum_{(i,j)\in E} p_{ij} \cdot max(0, x_{ij} - 1)$$

Restricciones:

$$x_{ij} \geq 1, \quad \forall (i,j) \in E \setminus B \tag{1}$$

$$x_{ij} + x_{km} \geq 1, \quad \forall ((i,j),(k,m)) \in B \tag{2}$$

$$s_i + \sum_{(j,i)\in A} x_{ji} = \sum_{(i,j)\in A} x_{ij} + t_i, \quad \forall i \in I \tag{3}$$

$$\sum_{(j,i)\in A} x_{ji} = \sum_{(i,j)\in A} x_{ij} + t_i, \quad \forall i \in N \setminus I \tag{4}$$

$$\sum_{iinI} s_i = 1 \tag{5}$$

$$\sum_{iinT} t_i = 1 \tag{6}$$

$$s_i \in \{0,1\} \tag{7}$$

$$t_i \in \{0,1\} \tag{8}$$

$$x_{ij} \in \mathbb{Z}_+ \tag{9}$$

La función objetivo del problema busca minimizar el costo de recoger los desechos de todas las calles utilizando la ruta mas corta. Se tiene en cuenta que el costo por recolectar la basura de un arco, solo se considera una vez, mientras que, el costo de recorrer el arco aparece luego de haber pasado al menos una vez por este.

Luego, en orden de aparición, estas restricciones aseguran que los arcos dirigidos (i, j) que son obligatorios, se recorran al menos una vez 1. Los arcos bidireccionales, también deben recorrerse al menos una vez, pero en cualquier sentido 2. Luego, las restricciones de flujo 3 y 4 aseguran que el recorrido sea consistente en la cantidad de veces que se ingresa y sale de un nodo. Las restricciones 5 y 6 aseguran que solo exista un nodo de entrada y uno de salida dentro de los conjuntos que lo permiten. Las restricciones 7 y 8 definen el dominio de las variables que determinan si un nodo es de entrada o salida, siendo 0 o 1. Finalmente, la restricción 9 establece que la cantidad de veces que se pueda pasar por un nodo sea un numero entero positivo. Estas restricciones las mismas que fueron utilizadas en [12] y descritas en la sección del estado del arte.

3.3. Instancias

Ya teniendo la representación de las redes viales mediante grafos y la definición matemática del modelo, es necesario establecer un formato de instancias y crear estas mismas para poder desarrollar el algoritmo. Este paso es importante, ya que, es necesario que el conjunto de datos pueda ser interpretado por un computador. Para lograr esto, se toma como base el formato planteado originalmente por Corberán [19], el cual fue adaptado a nuestro contexto y se obtuvo la siguiente estructura. Cabe destacar que los datos son solo de ejemplo y no representan necesariamente el grafo mostrado previamente:

```
NOMBRE : TEST RESCOL
COMENTARIO : Ejemplo de formato
VERTICES : 46
ARISTAS_REQ : 33
ARISTAS_NOREQ : 45
RESTRICCIONES : 3
NODOS_INICIALES : 46
NODOS_TERMINO : 46
LISTA_ARISTAS_REQ :
=========================
bidireccionalidad nodo1 nodo2 costo_recorrer costo_recolectar
=========================
uni 0 1 8 9
uni 1 21 3 1
uni 22 3 4 3
...
uni 102 68 21 5
bi 103 95 34 8
bi 96 105 12 3
bi 111 104 23 4
LISTA_ARISTAS_NO_REQ :
=========================
bidireccionalidad nodo1 nodo2
=========================
uni 0 1
uni 1 21
uni 22 3
...
uni 102 68
bi 103 95
bi 96 105
bi 111 104
CORDENADAS_NODOS :
=========================
x y id_nodo
=========================
0 0 0
4.5 0 1
10 0 2
12 0 3
17 0 4
...
0 -17 5
4.5 -17 6
10 -17 7
RESTRICCIONES:
=========================
nodo1 nodo2 nodo3
=========================
3 8 4
5 9 3
6 7 2
NODOS_INICIALES:
=========================
nodo1
=========================
3
5
6
NODOS_TERMINO:
=========================
nodo1
=========================
8
20
23
```

Primero se provee información sobre la instancia, nombre, un comentario opcional, cantidad de vértices, cantidad de aristas requeridas, cantidad de aristas no requeridas, cantidad de restricciones y cantidad de nodos en los conjuntos de entrada y salida. Luego se listan las aristas donde es obligatorio pasar, donde cada linea indica primero si el arco es unidireccional o bidireccional, luego el par de nodos que conecta la arista (nodo1, nodo2) y los costos asociado a recorrer y recolectar esa arista. La lista de aristas no requeridas solamente necesita del sentido de la calle y el par de nodos que la compone. En la sección de coordenadas de los vértices, estos se listan a modo de cuadricula con coordenadas x e y, junto con la id del nodo al que pertenece. Luego, la sección de restricciones prohíbe recorrer desde nodo1 hasta el nodo3 pasando por el nodo2. Finalmente, se muestran los conjunto de nodos permitidos de entrada y salida del grafo.

En base a este formato, se diseñaron una serie de instancias, tanto para desarrollo, pruebas y validación. Las cuales son descritas a continuación y se agruparán según su naturaleza, sintéticas y casos reales.

3.3.1. Instancias Sintéticas

La primera instancia creada fue A1. Su diseño es simple, pero aporta la complejidad suficiente como para considerar distintos casos de prueba, como calles bidireccionales, unidireccionales y distintos puntos de entrada o salida. La figura 3.4 muestra la representación en grafo de esta instancia. Como se puede notar, consiste en una matriz de 5x5 con arcos bidireccionales horizontales y arcos unidireccionales verticales, donde se intercalan los sentidos según columnas. En esta instancia se definen como nodos iniciales los nodos 3, 5 y 6, mientras que el nodo de salida puede ser cualquiera:

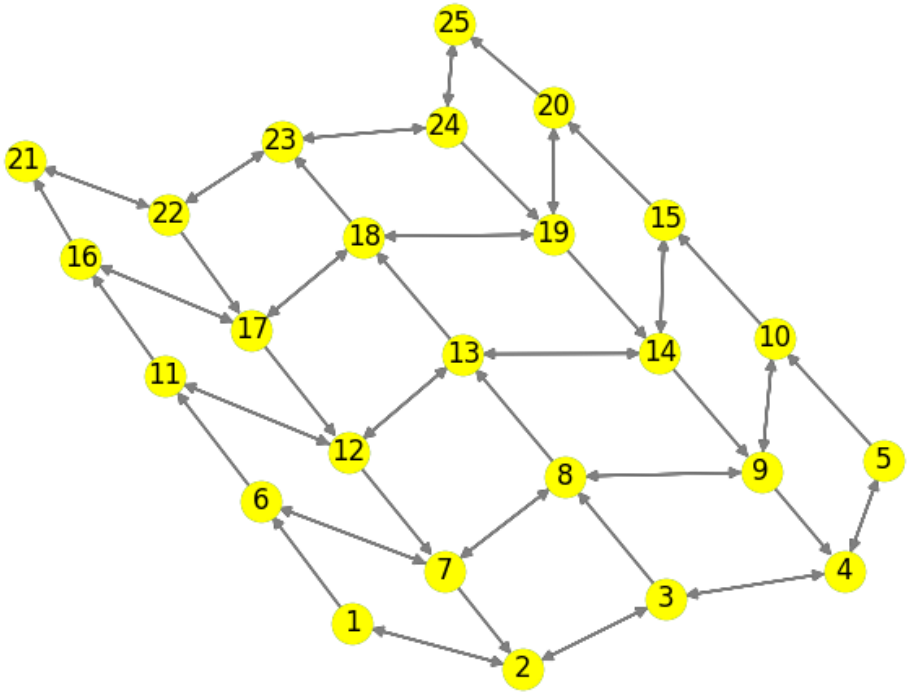


Figura 3.4: Visualización de instancia A1

La figura 3.5 muestra una instancia llamada AvenidaTest. Su diseño es similar a la instancia A1, pero con la diferencia de que consiste en una matriz de 4x4. Aunque es algo mas pequeña, su complejidad aumenta al tener mas combinaciones de arcos bidireccionales simulando avenidas. Como ya se mencionó, consiste en una matriz de 4x4, pero con arcos exteriores en sentido antihorario e internamente representa 3 avenidas y una calle de un solo sentido. Al igual que la instancia anterior, los nodos iniciales de esta también son el 3, 5 y 6, sin embargo, los nodos de salida son el 1 y el 16:

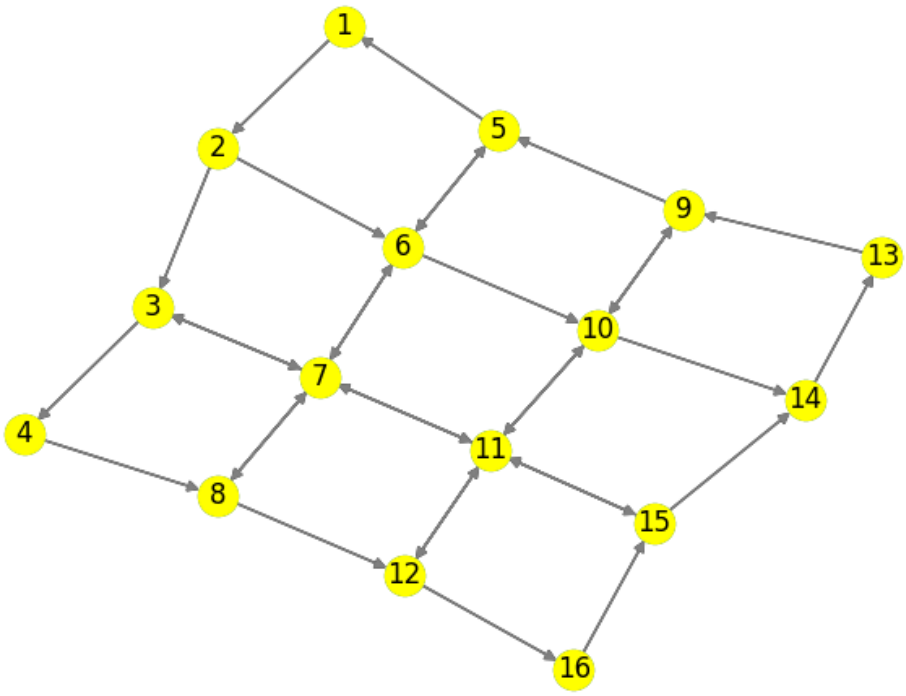


Figura 3.5: Visualización de instancia AvenidaTest

También se generaron dos conjuntos de instancias de manera automática siguiendo distribuciones normal y gamma. Estas fueron creadas en base a las instancias reales *CasoRealGrande* y *CasoRealMediano* que serán descritas mas adelante en la sección 3.3.2, la única diferencia entre las instancias base reales y las sintéticas son los valores de los pesos de las calles, mientras que la cantidad y sentido de las calles se mantiene. Para cada una de estas instancias reales base se generaron 40 instancias en dos grupos, uno para cada distribución. Cada instancia es similar, con la diferencia de que los pesos de los arcos varía aleatoriamente según las distribuciones mencionadas, tomando como rango los valores mínimos y máximos de su instancia real base. En las figuras 3.6 y 3.7 se muestran las frecuencias de los pesos de las calles de dos de estas instancias elegidas al azar:

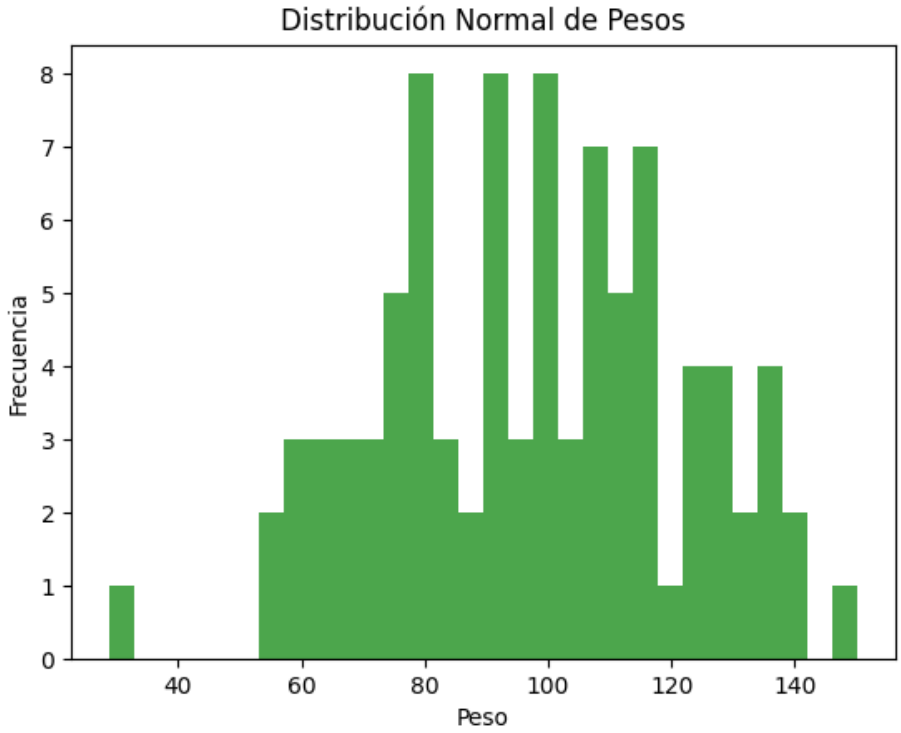


Figura 3.6: Frecuencia pesos de las calles de una instancia generada con una distribución normal.

La instancia generada con los pesos que se muestran en la imagen 3.6 fue creada en base a la instancia *CasoRealMediano*. Para generarla fue necesario usar el valor mínimo y máximo de los pesos de la instancia original para calcular los parámetros μ y σ, que representan la media y la desviación estándar de la distribución.

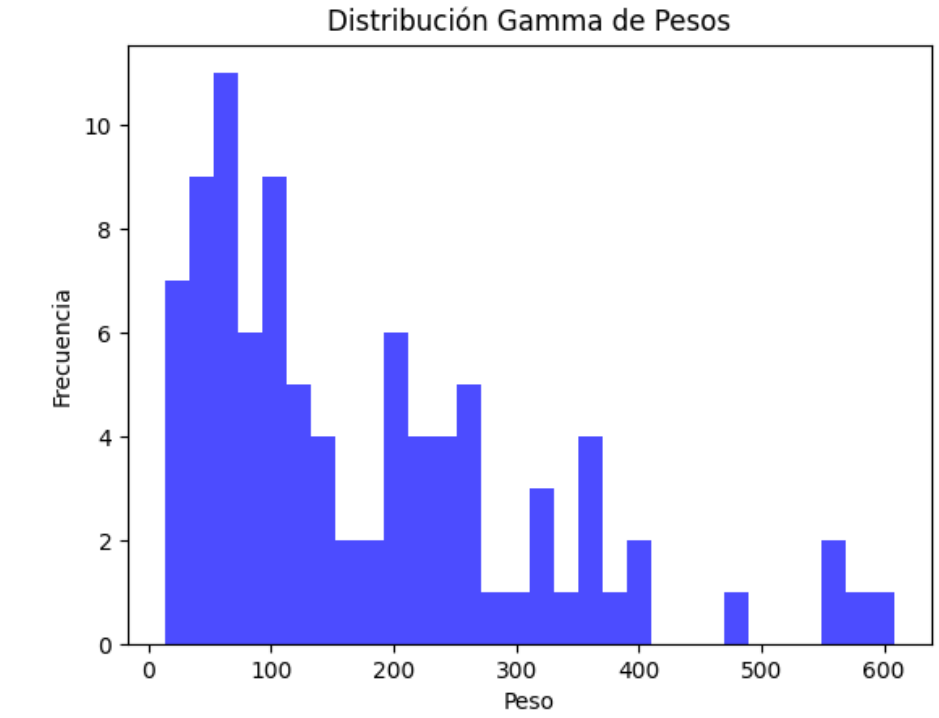


Figura 3.7: Frecuencia pesos de las calles de una instancia generada con una distribución gamma.

Para el caso de la imagen 3.7, la instancia fue generada en base a *CasoRealGrande*. Esta distribución requiere de dos parámetros, k y θ que representan la forma y la escala de la distribución respectivamente. Por su parte, los valores utilizados para estos parámetros fue 2, para k y la expresión $(pesoMaximo - pesoMinimo)/6$ para θ.

Las instancias sintéticas se denominan según su instancia base, tipo de distribución y un numero correlativo de 1 hasta 20. De esta manera, se generaron combinaciones entre las instancias CasoRealMediano, CasoRealGrande, distribución normal y gamma. Esto resultó en los siguientes grupos de instancias, cada uno con 20 variantes numeradas del 1 al 20:

- CasoRealMediano_variante_normal_1 .. 20
- CasoRealMediano_variante_gamma_1 .. 20
- CasoRealGrande_variante_normal_1 .. 20
- CasoRealGrande_variante_gamma_1 .. 20

3.3.2. Casos Reales

La siguientes tres instancias corresponden a sectores de la comuna de Renca, Región Metropolitana. Los datos que conforman estas instancias fueron medidos y provistos por

otro equipo de trabajo que es parte del proyecto en cual este trabajo se llevó a cabo. Fue necesario un preprocesado de estos datos para su adaptación al formato descrito anteriormente en la subsección 3.3.

La primera instancia se trata de *CasoRealPequeño*, ilustrada en la figura 3.8. Este se trata del caso menos complejo de resolver y tiene la particularidad de que tiene puntos de inicio y termino fijos, siendo estos los nodos 32709 y 32677 respectivamente. La configuración de esta instancia impide que el nodo inicial sea otro, dado que, el nodo 32709 no es accesible de otra manera que no sea como punto de inicio. Por otra parte, el nodo terminal podría se cualquiera, por lo cual, el nodo 32677 se eligió arbitrariamente.

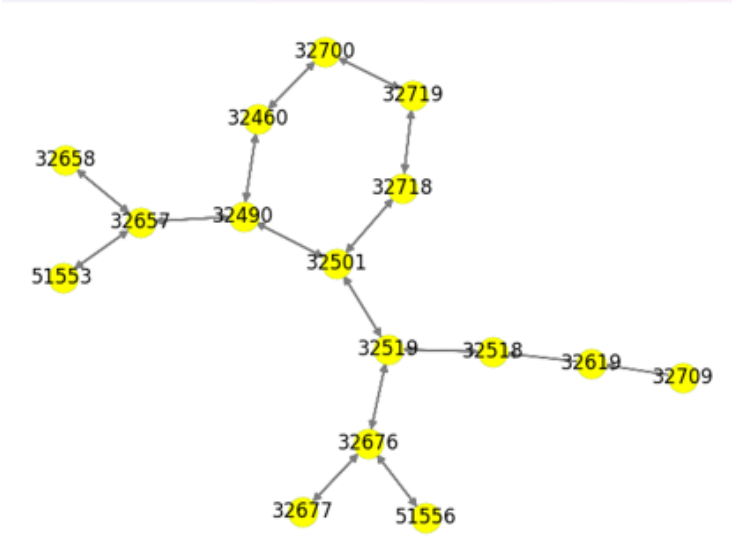


Figura 3.8: Visualización de instancia CasoRealPequeño

La segunda instancia real es *CasoRealMediano*, ilustrada en la figura 3.9, este es un sector significativamente mas grande que el anterior y fue el punto de inflexión para la resolución mediante métodos exactos y aproximados. Esta instancia tiene como nodo inicial el 31837 y terminal el 31965. Para este caso, esta instancia nos fue proporcionada por otro equipo de trabajo, quienes definieron el nodo inicial, no así el nodo de término, que fue seleccionado aleatoriamente.

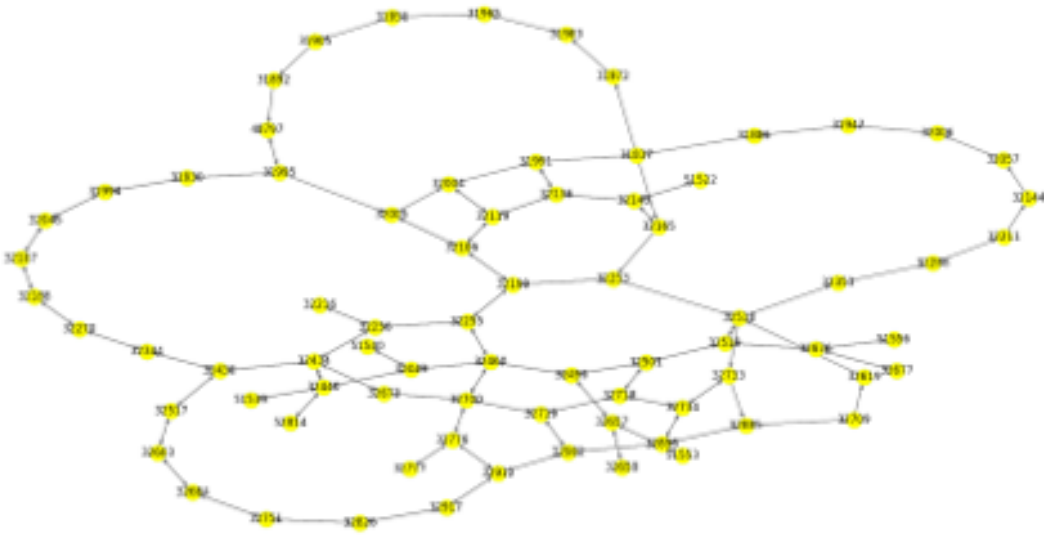


Figura 3.9: Visualización de instancia CasoRealMediano

La ultima instancia real fue *CasoRealGrande*. Para esta se omitirá la visualización dado su tamaño y poca claridad al ser representada gráficamente. Sin embargo es de similares características que *CasoRealMediano*. Esta instancia tiene como nodo inicial el 31837 y terminal el 31492. Al igual que el caso anterior, el nodo inicial estaba preestablecido, mientras que el nodo terminal fue escogido arbitrariamente. Para esta instancia se requirió reparar el nodo 47103, el cual no tenía salida y fue necesario convertir su arco de entrada en bidireccional.

4. Propuestas de Solución

En este trabajo se considera resolver este problema utilizando métodos exactos y metaheurísticas. En esta sección se presenta la descripción de las diferentes propuestas de solución evaluadas en este trabajo de tesis.

Preliminarmente, se plantea utilizar el solver Gurobi como método exacto. El objetivo de estos experimentos es determinar si es factible utilizar Gurobi en la resolución de instancias grandes. Independiente de este resultado, se plantea utilizar metaheurísticas para resolver el problema, específicamente algoritmos derivados del Ant Colony Optimization (ACO), como el Ant System, MAX-MIN Ant System o ACS. A continuación, se entregan los detalles de cada propuesta de solución.

4.1. Métodos Exactos

El modelo propuesto en la sección 3.2 es implementado para resolver una instancia del problema en Gurobi. Sin embargo, la información que nos entrega el solver es insuficiente para generar una ruta válida, dado que solo muestra una asignación de valores a las distintas variables del modelo que representan los arcos. Esta lista de valores no indica el orden secuencial que deben seguir los arcos para crear una ruta, sino solo la cantidad de veces que es recorrido tal arco. Para solucionar esto, se requiere de técnicas adicionales las cuales permitan generar una ruta a partir de los resultados obtenidos con el modelo. Un algoritmo Backtracking fue implementado como complemento para resolver las soluciones encontradas por el solver Gurobi.

Backtracking también es una técnica algorítmica exacta, que consiste en buscar soluciones de manera exhaustiva y de naturaleza recursiva construyendo paso a paso una ruta. Cada solución encontrada se compara con las restricciones o criterios de parada para determinar si la solución actual es válida o no. Si no es válida, el algoritmo da marcha atrás para buscar por otra rama del árbol de búsqueda. Este método, si bien no es eficiente, es apropiado para buscar la ruta óptima según la lista de asignaciones que entrega la resolución del modelo matemático propuesto, ya que, se van descartando pasadas por los arcos a medida que se van recorriendo, si el algoritmo no encuentra un arco por visitar, se devuelve hasta encontrar un punto donde pueda seguir explorando. El algoritmo 2 muestra el pseudocódigo del algoritmo Backtracking implementado en este trabajo.

Algorithm 2: pseudocódigo para el Algoritmo de Backtracking

```
1  if camino = none then
2  |  return nodoInicial;
3  end
4  if solucionEncontrada then
5  |  if noQuedanArcosPorRecorrer() then
6  |  |  return solucion;
7  |  else
8  |  |  return none;
9  |  end
10 end
11 if noQuedanArcosPorRecorrer() then
12 |  return none;
13 end
14 foreach nodo in arcosNodoActual do
15 |  nodoSiguiente ← nodo;
16 |  AgregarNodoAlCamino() ;            // Agrega el nodo al camino actual
17 |  ActualizarMapaAdyacencia() ;       // Actualiza el mapa de adyacencia
18 |  resultado ← Backtrack() ;          // Llama recursivamente a Backtrack
19 |  if resultado ≠ none then
20 |  |  return resultado ;       // Si se encontró una solución, la retorna
21 |  end
22 |  DeshacerCamino() ;     // Revierte el último nodo agregado al camino
23 end
24 return none ;                  // Retorna none si no se encuentra solución
```

Se destaca que este algoritmo tiene dos modos de búsqueda. El primero considera una restricción extra al buscar rutas, impidiendo devolverse a un nodo A pasando por B, es decir, que realice una vuelta en U. Cuando se agotan las posibilidades de búsqueda, el algoritmo vuelve a iniciarse, pero esta vez sin esta restricción extra, para realizar una búsqueda mas relajada. Esto se hace para simular restricciones de tránsito.

Las soluciones que son encontradas mediante Backtracking no son únicas. Al ejecutar el algoritmo múltiples veces sobre una misma solución de Gurobi es posible encontrar distintas rutas que satisfacen las condiciones impuestas. Dado que este algoritmo explora todo el universo de posibles combinaciones. El algoritmo Backtracking planteado es costoso en término de tiempo y puede considerarse un problema por si mismo el mejorar su eficiencia, pero no es el foco de este trabajo. Esto se puede considerar como trabajo futuro.

4.2. Metaheurísticas: Ant Colony Optimization

Como ya fue mencionado anteriormente, las metaheurísticas son técnicas generales que permiten resolver problemas de optimización. En la literatura [26], se ha reportado que las metaheurísticas del tipo poblacional como Ant Colony Optimization han sido eficaces para resolver problemas de ruteo como TSP. Dado que nuestro problema también es de

esta naturaleza, este tipo de técnica nos podría ser de utilidad. Para ello, se propone utilizar las metaheurísticas Ant System, MAX-MIN Ant System y Ant Colony System.

Si bien cada algoritmo implementado actúa de manera distinta en como se inicializan, evaporan y actualizan las feromonas, todos comparten la estructura base. El algoritmo 3 describe de manera general la estructura de los algoritmos ACO diseñados para resolver el problema planteado en este proyecto. Primero, se inicializan las feromonas, estructuras y datos preliminares asociados la instancia. Los algoritmos definen un número de épocas (*epocasMax*) que corresponden al numero de repeticiones que es ejecutado el algoritmo en su totalidad, donde además se reinicializa toda la información aprendida durante la ejecución, excepto el mejor resultado obtenido hasta el término de la ejecución anterior. Si bien, esto le da al algoritmo mas oportunidades de encontrar mejores soluciones, lo hace a costa de un mayor tiempo de ejecución.

Cada época consiste en un número de repeticiones (*iteracionesMax*), donde en cada una de ellas, las hormigas construyen una ruta y se depositan las feromonas, además de evaluar las soluciones y preparar las estructuras para la siguiente iteración.

A continuación se muestra un pseudocódigo para un caso base:

Algorithm 3: pseudocódigo ACO Base

```
1  LeerInstancia() ;                          // Carga los datos de la instancia
2  InicializarHormigas() ;                    // Configura las hormigas iniciales
3  InicializarFeromonas() ;  // Establece los niveles iniciales de feromonas
4  for epoca ← 0 to epocasMax do
5      for iteracion ← 0 to iteracionesMax do
6          foreach hormiga in Hormigas do
7              while not SolucionCompleta() do
8                  ElegirSiguienteNodo() ;          // Decide el siguiente nodo a
                    visitar
9                  VisitarNodo() ;                  // Visita el nodo seleccionado
10                 EvaporarFeromonasLocales() ;  // Opcional: Evaporación local
                    de feromonas
11             end
12             BuscarSalida() ; // Opcional: Buscar una salida en la solución
                actual
13         end
14         EvaporarFeromonas() ;            // Evaporación global de feromonas
15         ActualizarFeromonas() ;      // Refuerzo de feromonas basado en la
            calidad de la solución
16         GuardarMejorSolucion() ; // Almacena la mejor solución encontrada
17         Limpiar() ;           // Prepara las estructuras para la siguiente
            iteración
18     end
19     Resetear() ;             // Reinicia el estado para la siguiente época
20 end
21 MostrarResultados() ;                       // Presenta los resultados finales
```

Cabe destacar que, los algoritmos se implementaron según sus definiciones clásicas sin mayores modificaciones salvo la manera en que se calcula el valor de las feromonas globales cuando se actualizan y el uso de feromonas locales, además de introducir nuevos mecanismos que serán descritos posteriormente al final de esta subsección . Las modificaciones realizadas se hicieron con el propósito de adaptar el algoritmo original a nuestro contexto específico. Para el calculo de las feromonas globales, se determinó utilizar la cantidad de pasadas y largo del tour para actualizar las feromonas.

También es necesario mencionar que una solución se compone de dos rutas, una de recorrido y una de salida. La primera comienza en el momento en que la hormiga comienza su trayecto en un nodo de inicio y termina cuando todos los arcos han sido recorridos al menos una vez, pudiendo terminar en cualquier nodo del grafo. La segunda ruta comienza en el último nodo visitado donde ya han sido recorridos todos los arcos y termina cuando la hormiga visita un nodo de salida. La solución se compone de la concatenación de ambas rutas.

A continuación se detallan los algoritmos en términos de los componentes implementados en cada caso:

4.2.1. Feromonas: tipos e inicialización

Se tienen dos tipos de feromonas, las de recorrido y las de salida, cuya diferencia está en su utilidad y parámetros asociados, siendo estos β y β_{sa} respectivamente. El primer tipo de feromonas es el mismo que se describe en la sección 2.3.3, que corresponde a la definición y uso clásicos de estas feromonas. Las feromonas de salida comparten la misma definición, pero se usan para generar y compartir información de las rutas mas favorables para salir del grafo una vez se termina de construir la ruta que recorre todos los arcos al menos una vez y se asocia a los arcos recorridos en esta ruta de salida. El parámetro α es el mismo para ambos conjuntos de feromonas.

Al momento de inicializar el grafo, dependiendo del algoritmo que se está utilizando, es el valor inicial de los conjuntos de feromonas. Para el caso de Ant System, el valor inicial está dado por un parámetro τ_0. En el caso de MAX-MIN Ant System, el valor inicial es el mismo que se establece para el valor del límite superior de las feromonas, siendo este el parámetro *umbral_superior*. En el caso de Ant Colony System, también se usa τ_0, sin embargo, esto es una simplificación del valor recomendado por su autor original.

4.2.2. Construcción de rutas

La construcción de rutas es un proceso que es compartido y utilizado por todos los algoritmos de nuestra propuesta de solución. El mecanismo que construye las rutas funciona en base a la regla de transición descrita anteriormente, que asigna a cada nodo vecino del nodo actual una probabilidad de ser elegido como el siguiente a ser visitado. Las probabilidades se normalizan y acumulan, para ser escogidas mediante la generación de un número aleatorio entre 0 y 1. Una vez se ha seleccionado el siguiente nodo, este es visitado actualizando la posición actual de la hormiga y agregando al camino el arco que conecta el nodo anterior con el actual.

En el caso de ACS existe un mecanismo alternativo. Se genera un numero aleatorio q que, si es menor que el umbral q_0, se selecciona el arco con mas feromonas de entre los arcos vecinos. En el caso contrario, se usa la regla de transición tradicional.

Durante el proceso de construcción de rutas se utiliza un subconjunto de feromonas que son locales para cada hormiga. Estas feromonas se utilizan para evitar que una hormiga vuelva a visitar arcos que ya ha recorrido, por lo que, cada vez que se visita un arco, se disminuye su cantidad de feromonas según una tasa de evaporación dada por el parámetro ρ_{sec}. Dado que estas feromonas pueden ser actualizadas múltiples veces en un mismo recorrido, es posible que su valor caiga en un estado de underflow. Para evitar esto se establece un límite inferior que impide esta situación. Esta característica, propia de MAX-MIN Ant System, fue implementada como un mecanismo base para todos los algoritmos para evitar que los valores de las feromonas locales se acerquen demasiado a 0.

Al ser un conjunto local, las feromonas de una hormiga no interfieren con el conjunto

de otra, ni tampoco con el conjunto de feromonas globales. Al termino de cada iteración, las feromonas locales son reiniciadas según los valores actualizados de las feromonas globales, que serán explicadas a continuación.

Una particularidad importante es que cuando las hormigas actualizan sus feromonas locales, es posible que estas caigan en un estado de underflow, debido a que son recorridas múltiples veces.

4.2.3. Deposito global de feromonas

El depósito global ocurre luego de que todas las hormigas han creado sus rutas. Si bien, la manera en que ocurre este depósito es distinta en cada algoritmo, todos tienen el mismo propósito. Los arcos que son recorridos menos veces en rutas de menor longitud son los que reciben mas cantidad de feromonas, esto refuerza las rutas mas cortas y eficientes. Esta actualización es aplicada tanto para las feromonas de recorrido como las de salida.

Este proceso tiene sus particularidades según el algoritmo. En el caso de Ant System, todas las hormigas actualizan las feromonas de los arcos que recorrieron. Mientras que para MAX-MIN Ant System, solo la hormiga con la mejor ruta puede actualizar sus feromonas y el valor de estas deben estar dentro de los límites establecidos por los parámetros *umbral_inferior* y *umbral_superior*. Por otra parte, en ACS no existe un límite explícito y solo la hormiga con la mejor ruta puede actualizar, al igual que en MAX-MIN.

4.2.4. Evaporación de feromonas

La evaporación de las feromonas ocurre de igual manera tanto para Ant System como para MAX-MIN. Para cada arco del grafo se disminuye su cantidad de feromonas según los parámetros ρ y ρ_{sa} que son las tasas de evaporación para los conjuntos de feromonas de recorrido y salida respectivamente. La evaporación en el algoritmo ACS no ocurre como un componente o proceso separado, sino que está integrado en la expresión que actualiza las feromonas.

4.2.5. Mecanismos nuevos

Durante el proceso de desarrollo de este proyecto se fueron introduciendo nuevos mecanismos que ayudaron a que el algoritmo fuese mas eficiente, tanto en términos de calidad, como de tiempo. Las propuestas de mecanismos nuevos se pueden dividir entre componentes y comportamientos. Al primer grupo pertenece la matriz secundaria de feromonas, que es utilizada para salir del grafo y la matriz de feromonas locales para cada hormiga, cada uno con sus respectivos parámetros asociados.

Otro componente con el que se hicieron pruebas fue el “Oscilador”. Este componente permite variar en tiempo de ejecución el valor de los parámetros del algoritmo. Por ejemplo, puede modificar el valor del parámetro α dentro de un rango e incrementos específicos. Tiene dos modos de funcionamiento, uno mueve el valor del parámetro describiendo un movimiento sinusoidal, mientras que el otro es aleatorio, pero siempre manteniéndose dentro de los límites especificados. De esta manera el algoritmo puede moverse entre distintos

estados de exploración y explotación de manera dinámica. Los parámetros asociados al Oscilador son *min*, *max*, *inc*, donde los dos primeros establecen el rango en el cual debe moverse el valor del parámetro y el tercero es en cuanto debe variar el valor. Este mecanismo solo está disponible para los parámetros base α, β, ρ y ρ_{sec}.

Dentro del conjunto de comportamientos se encuentran los mecanismos "limitador" y "beta0". El primero limita la cantidad de veces que se puede pasar por un arco según el parámetro *valor_limitador*, esto permite recorrer libremente un arco una cierta cantidad de veces, que luego de ser agotadas, se evita pasar por este arco para darle prioridad a los arcos vecinos que aún no han sido recorridos, independientemente de su costo o largo. Cuando todos los arcos candidatos a recorrerse han sido agotados, se ignora esta regla para el nodo actual.

Para el segundo mecanismo, "beta0", se ignora la información heurística durante la generación de rutas. La razón de esto es que, el objetivo del algoritmo es encontrar el camino que recorra todos los arcos al menos una vez en la menor cantidad de veces. Al considerar la información heurística, las hormigas podían decidir pasar por un arco ya visitado si este tiene un peso menor que otro arco adyacente no visitado, siendo este el que debería tener mayor prioridad. Por otra parte, para la matriz de salida si es importante considerar el parámetro β porque en este caso las hormigas deben llegar a la salida siguiendo el camino mas rápido y barato.

Todos los mecanismo descritos en esta subsección son paramétricos, es decir, tienen un flag asociado que permite o no, su funcionamiento.

4.2.6. Parámetros

A continuación, se listan a modo de resumen, los parámetros que están involucrados en los algoritmos previamente descritos.

Parámetros en común para todos los algoritmos:

- α: Controla la importancia de las feromonas.
- β: Controla la importancia de la información heurística en la ruta principal.
- β_{sa}: Controla la importancia de la información heurística en la ruta de salida.
- #Hormigas: Cantidad de hormigas.
- ρ: Tasa de evaporación de las feromonas globales.
- ρ_{sec}: Tasa de evaporación de las feromonas locales.
- ρ_{sa}: Tasa de evaporación de las feromonas de salida.
- τ_0: Valor inicial de los conjuntos de feromonas.
- umbral_inferior: Límite inferior de las feromonas.

Parámetros exclusivos para MAX-MIN Ant System:

- umbral_superior: Límite superior de las feromonas.

- umbral_sin_mejora_limite: Cantidad de iteraciones que pueden transcurrir sin que haya una mejora antes de reinicializar las feromonas.
- a: Valor arbitrario que controla el límite inferior en base al límite superior.

Parámetros exclusivos para ACS:

- q_0: Umbral para elegir el siguiente nodo en base a la mejor opción o en base a la regla de transición pseudoaleatoria.

Parámetros de mecanismos adicionales:

- beta0: permite o no usar este mecanismo
- oscilador: permite o no usar este mecanismo
- limitador: permite o no usar este mecanismo
- valor_limitador: valor numérico de la cantidad de veces que se permite pasar por un arco
- inc_alfa : incremento del oscilador para el parámetro α
- min_alfa : limite inferior del parámetro α
- max_alfa : limite superior del parámetro α
- inc_beta : incremento del oscilador para el parámetro β
- min_beta : limite inferior del parámetro β
- max_beta : limite superior del parámetro β
- inc_rho : incremento del oscilador para el parámetro ρ
- min_rho : limite inferior del parámetro ρ
- max_rho : limite superior del parámetro ρ
- inc_rho_secundario : incremento del oscilador para el parámetro ρ_{sec}
- min_rho_secundario : limite inferior del parámetro ρ_{sec}
- max_rho_secundario : limite superior del parámetro ρ_{sec}

4.2.7. Estructuras de Datos

La estructura de datos principal de este algoritmo, denominada Graph, consiste en la representación de un grafo. A su vez, Graph está compuesto por subestructuras para representar los nodos y aristas. Esta estructura es poblada con los datos obtenidos desde la instancia al momento de la lectura de esta. Graph además guarda metadatos sobre la instancia que está representando como el nombre de la instancia, cantidad de nodos y aristas.

Por su parte, la subestructura que representa los nodos almacena la id, coordenadas, lista de arcos entrantes y salientes del nodo en cuestión, además de crear un subnodo interno para cada arco entrante o saliente, esto se hace para simular el sentido del tránsito y permitir o denegar ciertas transiciones como vueltas en u. De la misma manera, la subestructura de arcos almacena la id, costos de recolección y recorrido del arco, las veces

que ha sido recorrido, si es obligatorio, si es bidireccional y los nodos que conecta. Ambas estructuras se almacenan a modo de mapas dentro de Graph, con el fin de hacer mas óptima la búsqueda de cada elemento al momento de ejecutar el algoritmo. Estos mapas consisten en un par clave, valor, donde la clave corresponde a la id del arco o nodo y el valor corresponde a un puntero al objeto en si. Las hormigas o agentes que ingresan a un nodo, siempre lo hacen mediante un arco de entrada y solo pueden salir por un arco de salida. De esta manera es imposible que una hormiga pueda transitar en contrasentido, dar vueltas en u o ingresar por una calle no permitida, siempre y cuando esta restricción esté explicita en la instancia.

Las demás estructuras del algoritmo son objetos de una clase base, clases derivadas de esta y clases auxiliares. La clase ACO provee de los métodos y propiedades base a las clases derivadas por cada algoritmo implementado. Cada clase que hereda de ACO implementa, cuando corresponde, de manera propia sus componentes mediante la sobrecarga de los métodos base provistos por su clase padre. Las clases auxiliares proveen servicios para la lectura de instancias, manejo de formato, generación de números pseudo-aleatorios y configuración de parámetros. Por otra parte, la clase Ant Colony System tiene un método de funcionamiento ligeramente distinto al ACO base, por lo que fue necesario escribir el algoritmo por separado.

Por su parte, las hormigas y feromonas, que están contenidas en la clase base ACO. Se componen de una id, nodo actual de la hormiga, un mapa de arcos visitados y cantidad de veces que fueron visitados, el camino secuencial que sigue la hormiga, el costo y largo del camino que se está recorriendo, además de una copia local de las feromonas. Estas ultimas, definidas como un mapa arco-feromona.

Otra estructura clave es un mapa de arcos visitados, esta estructura es utilizada como medio de comprobación para determinar si una ruta de recorrido está completa o no. Se inicializa como una copia de todos los arcos del grafo y se van removiendo a medida que se recorren. Si esta lista está vacía, la ruta actual se considera como completada. La implementación de esta estructura significó una gran mejora en cuanto a la comprobación de salida respecto a una lógica anterior menos eficiente.

4.3. Finetuning o Sintonización Fina

El motivo de realizar un ajuste fino de los parámetros para los grupos de instancias es encontrar cual es una configuración que entregue soluciones de calidad para un conjunto de instancias o similares.

Dada la naturaleza estocástica o aleatoria del algoritmo, los resultados que se obtienen varían de ejecución en ejecución. Esto es especialmente notable cuando se resuelven instancias de gran tamaño. Por este motivo, es favorable realizar un finetuning de instancias, para detectar como la combinación de valores de los parámetros afecta el rendimiento, tanto para bien, como para mal. Al conocer como se comporta el algoritmo bajo cierto conjunto de condiciones (instancias y parámetros), es posible asumir que este se comportará de manera similar cuando estas condiciones varían levemente, como por ejemplo, si

varían los pesos de las calles. Cabe destacar que un conjunto de parámetros podría no servir para otra instancia radicalmente distinta.

El software utilizado para esto fue irace [53], un configurador automático de algoritmos que realiza una serie de experimentos para determinar estadísticamente cual es la mejor combinación de parámetros. Esto lo logra haciendo competir distintas configuraciones del algoritmo, evaluando y comparando los resultados, de esta manera, irace descarta los valores mas desfavorables y puede enfocarse en un espacio de búsqueda donde, es mas probable encontrar mejores resultados. El proceso se itera hasta alcanzar un criterio de término, ya sea en cantidad de experimentos o tiempo, y nos entrega un ranking de cuales fueron las mejores ejecuciones junto con sus parámetros y otras métricas.

5. Experimentos

En esta sección se describen los experimentos realizados para evaluar la eficacia de las técnicas propuestas para resolver el problema planteado. Los resultados obtenidos son analizados para comparar los métodos y proponer mejoras a estos.

5.1. Setup Experimental

Para comenzar con los experimentos se implementaron ambas propuestas de solución, el método exacto con Gurobi y el método aproximado con los algoritmos ACO. Para el caso de Gurobi, se utilizó GurobiPy, una librería de Python que permite utilizar el solver. Por otra parte, para programar los algoritmos ACO se utilizó el lenguaje C++. Dichos programas se pueden encontrar en los siguientes repositorios públicos: `https://github.com/newUser161/MetodoExactoGurobiPy` y `https://github.com/newuser161/rescol`

Para llevar a cabo los experimentos de esta investigación se utilizó el siguiente hardware.

- **Procesador (CPU)**:
  - Modelo: Intel Core i9-14900KF
  - Velocidad del Reloj: 3.2 GHz
  - Núcleos/Cantidad de Hilos: 8 núcleos / 32 hilos
- **Memoria RAM**:
  - Capacidad: 32 GB
  - Tipo: DDR5
  - Frecuencia: 7200 MHz
- **Almacenamiento**:
  - Tipo y Capacidad: SSD de 1 TB
- **Sistema Operativo**:
  - Versión: Windows 11 Home

Para los experimentos realizados se definieron valores para los parámetros con el fin de mantener una consistencia entre ejecuciones. Estos parámetros fueron inicialmente definidos de manera arbitraria para luego ser sintonizados sistemáticamente. Estos parámetros, utilizados solamente para la resolución mediante metaheurísticas se definen en las tablas 5.1, 5.2 y 5.3 para los recursos algorítmicos.

Parámetros ACO Base	Valor inicial
α	1.0
β	2.0
β_{sa}	2.0
Cantidad de Hormigas	10
ρ	0.5
ρ_{sec}	0.8
ρ_{sa}	0.8
τ_0	1.0
umbral_inferior	0.001

Tabla 5.1: Parámetros base

Parámetros MAX-MIN Ant System	Valor inicial
umbral_superior	161161.161
Umbral_sin_mejora_limite	10
a	13

Tabla 5.2: Parámetros MAX-MIN Ant System

Recursos Algorítmicos	Valor
Cantidad de iteraciones	1000
Épocas	3

Tabla 5.3: Recursos Algorítmicos

5.1.1. Consideraciones en las Pruebas

Se estableció un límite de 1000 iteraciones, que posteriormente, en la fase de fine tuning, se cambiará a un límite calculado según número de evaluaciones dividido en cantidad de hormigas. Definimos como evaluaciones la comprobación de una solución generada por una hormiga, mientras que una iteración corresponde a la generación de soluciones por parte de todas las hormigas. Esto con el fin de mantener la cantidad de iteraciones constante al utilizar distintas cantidades de hormigas para comparar distintos métodos.

La implementación de Gurobi no puede encontrar la ruta óptima mediante Backtracking a partir de la instancia mediana. Esto sirve como punto de inflexión entre calidad de la solución y tiempo de búsqueda. Por una parte, Gurobi si puede modelar y encontrar correctamente una solución, sin embargo, el algoritmo Backtracking no es capaz de generar la ruta a partir de los datos entregados por el solver en un tiempo razonable. Se realizó un experimento donde se dejó en ejecución el algoritmo durante mas de 12 horas y no fue posible encontrar la ruta. Por esta razón se decide excluir el método exacto Gurobi en los resultados, a excepción del largo total de la ruta óptima, la cual si es posible obtener.

Los métodos de resolución evaluados fueron Ant System, MAX-MIN Ant System con distintos mecanismos y Gurobi junto con Backtracking. En esta entrega no se analizan resultados utilizando Ant Colony System, dado que su implementación aun no es lo suficientemente estable como para someterlo a pruebas exhaustivas, aunque si fue implementado. Para los experimentos con metaheurísticas, se realizan distintas combinaciones entre métodos y mecanismos. Por motivos de legibilidad, se establece la siguiente nomenclatura para los nombres de las combinaciones utilizadas; *GBt* para Gurobi y Backtracking,*AS* para Ant System, *MMAS* para MAX-MIN Ant System; y para cada una de ellas se puede utilizar *MS* para matriz de salida, *b0* para beta0, *osc* para oscilador, *lim* para limitador. No se probaron todas las combinaciones posibles para los métodos y mecanismo, al menos en las primeras pruebas, sino que se fue experimentando con las combinaciones que daban mejores resultados. Sin embargo, durante el finetuning, si se comprobaron todas las posibles combinaciones, ya que, estas pruebas fueron automatizadas.

El componente oscilador no fue incluido en estas pruebas a pesar de haber reportado buenos resultados en pruebas aisladas. Se decidió no incluirlo dado que aumenta la complejidad y cantidad de las pruebas, además de agregar aleatoriedad a parámetros que deseablemente debiesen ser valores fijos o que no varíen demasiado.

Los valores escogidos para los rangos de los parámetros fueron determinados según la experiencia personal adquirida durante el desarrollo del algoritmo. Algunos valores muy altos, por ejemplo sobre 5 y 0.5, para β y ρ respectivamente, impedían un correcto funcionamiento del algoritmo en general. Por lo tanto ciertos rangos de valores fueron excluidos de las pruebas. Otros fueron limitados a rangos amplios, pero dentro de lo razonable, como el número de hormigas o umbrales. En general, los rangos establecidos para las pruebas nos aseguran que el algoritmo se desempeñe bien.

Los resultados que se mostrarán en la siguiente subsección 5.2 corresponden a las

instancias A1, CasoRealPequeño, CasoRealMediano y CasoRealGrande para las pruebas unitarias por instancia y comparación de métodos. Por motivos de legibilidad, se omitirán las tablas completas con los resultados, estas se podrán encontrar al final de este informe en el Anexo A. Mientras que los resultados mostrados serán mediante boxplots que muestran el largo y costo de los tours, además del promedio del tiempo de ejecución de los experimentos. Posteriormente se mostrarán gráficos de convergencia que muestran como van evolucionando las soluciones a medida que se ejecuta el algoritmo. Cabe destacar que cada prueba corresponde a 10 ejecuciones completas del algoritmo. No se realizaron pruebas sobre la instancia AvenidaTest dada su similitud con la instancia A1, por lo tanto, esta instancia se destinó netamente a pruebas internas de desarrollo.

5.1.2. Metodología del Finetuning

Los parámetros que fueron incluidos para su optimización fueron los que se muestran en la tabla 5.4:

Parámetros a optimizar	Rango de valores	Tipo	Restricción
algoritmo	(0,1)	Categórico	N/A
hormigas	(2, 30)	Entero	N/A
α	(0.01, 5)	Real	N/A
β	(0.01, 3)	Real	N/A
β_{sa}	(0.01, 0.5)	Real	N/A
ρ	(0.05, 0.3)	Real	N/A
ρ_{sec}	(0.05, 0.3)	Real	N/A
ρ_{sa}	(0.05, 0.3)	Real	N/A
τ_0	(1, 10)	Real	N/A
umbral_inferior	(0.001)	Categórico	N/A
valor_limitador	(4,3,2,1)	Categórico	N/A
umbral_superior	(500,10000)	Entero	Solo para MAX-MIN Ant System
umbral_sin_mejora_limite	(10,30)	Entero	Solo para MAX-MIN
a	(10, 50)	Entero	Solo para MAX-MIN Ant System

Tabla 5.4: Parámetros irace

Mientras que los recursos algorítmicos o presupuesto definidos para irace y los algoritmos en general son los que se muestran en la tabla 5.5:

Recursos Algorítmicos	Valor	Correspondencia
Cantidad de evaluaciones	5000	ACO
Cantidad de experimentos	3000	irace
Épocas	1	ACO

Tabla 5.5: Recursos Algorítmicos irace y ACO

Las instancias que fueron utilizadas para ser configuradas fueron aquellas generadas sintéticamente según las distribuciones normal y gamma. De dividieron en dos grupos, para entrenamiento y validación. Los grupos de instancias quedaron de esta manera:

- **Entrenamiento**:
  - CasoRealMediano_variante_normal_1 .. 10
  - CasoRealMediano_variante_gamma_1 .. 10
  - CasoRealGrande_variante_normal_1 .. 10
  - CasoRealGrande_variante_gamma_1 .. 10
- **Validación**:
  - CasoRealMediano_variante_normal_11 .. 20
  - CasoRealMediano_variante_gamma_11 .. 20
  - CasoRealGrande_variante_normal_11 .. 20
  - CasoRealGrande_variante_gamma_11 .. 20

5.1.3. Proceso y Ejecución del Finetuning

Para estas pruebas se decidió utilizar solo una época. Dado que se está evaluando solo una ejecución completa del algoritmo no es necesario realizar múltiples épocas. Además de que esto podría influir en los resultados si por motivos de aleatoriedad se alcanzan mejores soluciones en épocas posteriores.

Se realizó una serie de pruebas preliminares a la prueba definitiva con las instancias generadas según las distribuciones normal y gamma. Esta serie de pruebas se realizaron de manera individual por instancia y variando los mecanismos que usa el algoritmo manualmente. Las pruebas preliminares revelaron una serie de problemas de rendimiento con el algoritmo. Estos problemas se mostraban solo con ciertas configuraciones de parámetros, en donde algunas ejecuciones tardaban horas en completarse, mientras que otras ejecuciones paralelas lo hacían en minutos. Para solucionar esto, fue necesario replicar en el ambiente de prueba las configuraciones problemáticas y comprobar paso a paso la ejecución del algoritmo para determinar el porqué del mal desempeño. Este proceso de depuración permitió identificar un rango de valores especialmente perjudiciales para el algoritmo. Específicamente, un valor β sobre 5 le daba preferencia a la información heurística

por sobre la información provista por las feromonas y en el caso de que un arco con un costo muy alto fuese la única conexión entre dos secciones de la instancia, este era muy poco probable que fuese elegido, dando origen a un bucle dentro de una de las secciones. Adicionalmente, si hay un valor de ρ sobre 0.6, el algoritmo olvida demasiado rápido la información obtenida por las hormigas, impidiendo la convergencia a una solución en base al aprendizaje.

Un caso similar fue el del parámetro β_{sa}. Para las pruebas sobre la instancia CasoRealPequeño se debió bajar el valor límite de este parámetro de 5 a 1, ya que, esta instancia tiene algunos arcos que actúan como puente entre dos clusters de nodos y el peso de dicho arco es notablemente mayor que el resto de las aristas, por lo que un valor de β_{sa} muy alto impide recorrer este arco, provocando un bucle en el sector donde esté la hormiga. En estas pruebas los demás parámetros se mantuvieron sin cambios.

5.2. Resultados y Discusión

A continuación, se revisa por instancia los respectivos resultados.

5.2.1. Instancia A1

En primer lugar, se utilizó el solver Gurobi para resolver esta instancia. Luego de haberla procesado, el reporte de Gurobi indicó que se encontraron 9 soluciones en 0.05 segundos, en donde el valor mas bajo, 255, representa la mejor solución global en términos de costo. El nodo de entrada de esta ruta fue el 6 y el de salida fue el 24. A continuación se muestra el resultado obtenido como solución al modelo matemático por Gurobi. Se listan los valores de las variables de decisión para cada arco recorrido, esto indica la cantidad de veces que el arco $x_{(i,j)}$ fue recorrido:

$x_{(1,6)} = 1$	$x_{(11,16)} = 2$	$x_{(1,2)} = 0$	$x_{(7,8)} = 1$	$x_{(14,13)} = 0$	$x_{(19,20)} = 0$
$x_{(7,2)} = 1$	$x_{(17,12)} = 3$	$x_{(2,1)} = 1$	$x_{(8,7)} = 0$	$x_{(13,14)} = 1$	$x_{(20,19)} = 1$
$x_{(3,8)} = 1$	$x_{(13,18)} = 1$	$x_{(2,3)} = 1$	$x_{(8,9)} = 1$	$x_{(14,15)} = 1$	$x_{(21,22)} = 1$
$x_{(9,4)} = 2$	$x_{(19,14)} = 1$	$x_{(3,2)} = 1$	$x_{(9,8)} = 0$	$x_{(15,14)} = 0$	$x_{(22,21)} = 0$
$x_{(5,10)} = 1$	$x_{(15,20)} = 2$	$x_{(3,4)} = 0$	$x_{(9,10)} = 1$	$x_{(16,17)} = 1$	$x_{(22,23)} = 1$
$x_{(6,11)} = 1$	$x_{(16,21)} = 1$	$x_{(4,3)} = 1$	$x_{(10,9)} = 1$	$x_{(17,16)} = 0$	$x_{(23,22)} = 1$
$x_{(12,7)} = 1$	$x_{(22,17)} = 1$	$x_{(4,5)} = 1$	$x_{(11,12)} = 0$	$x_{(17,18)} = 0$	$x_{(23,24)} = 1$
$x_{(8,13)} = 1$	$x_{(18,23)} = 1$	$x_{(5,4)} = 0$	$x_{(12,11)} = 1$	$x_{(18,17)} = 1$	$x_{(24,23)} = 0$
$x_{(14,9)} = 1$	$x_{(24,19)} = 1$	$x_{(6,7)} = 1$	$x_{(12,13)} = 1$	$x_{(18,19)} = 0$	$x_{(24,25)} = 0$
$x_{(10,15)} = 1$	$x_{(20,25)} = 1$	$x_{(7,6)} = 0$	$x_{(13,12)} = 0$	$x_{(19,18)} = 1$	$x_{(25,24)} = 1$

La ruta generada mediante Backtracking en base a la lista de arcos anterior es la que se observa en la figura 5.1.

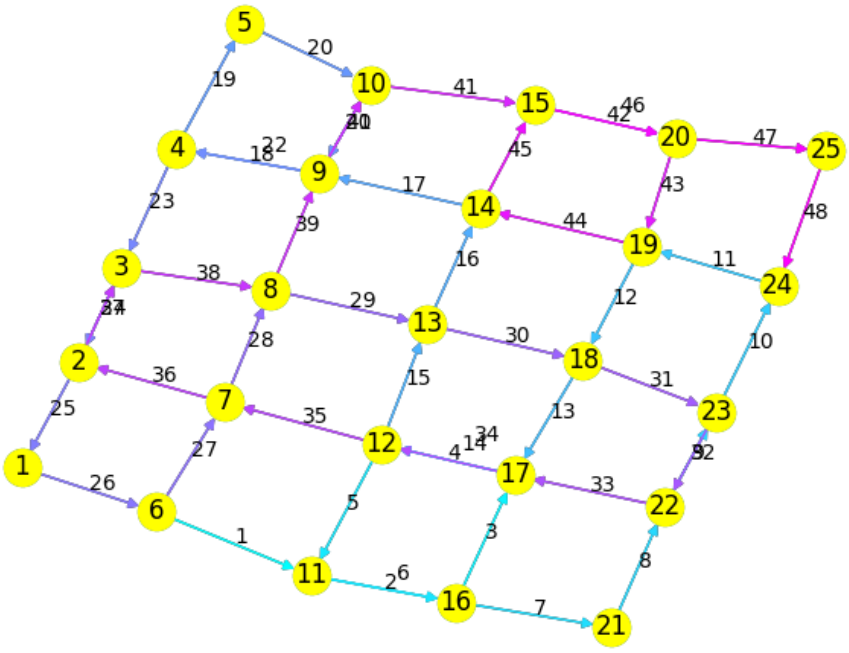


Figura 5.1: Ruta generada por Backtracking - A1.

Los números sobre los arcos indican el orden en que fueron recorridos. Como se puede observar, la ruta comienza en el nodo que fue marcado como inicial (6) y termina en el terminal (24).

Los resultados para esta misma instancia resuelta mediante las metaheurísticas ACO se muestran en las figuras 5.2, 5.3 y 5.4:

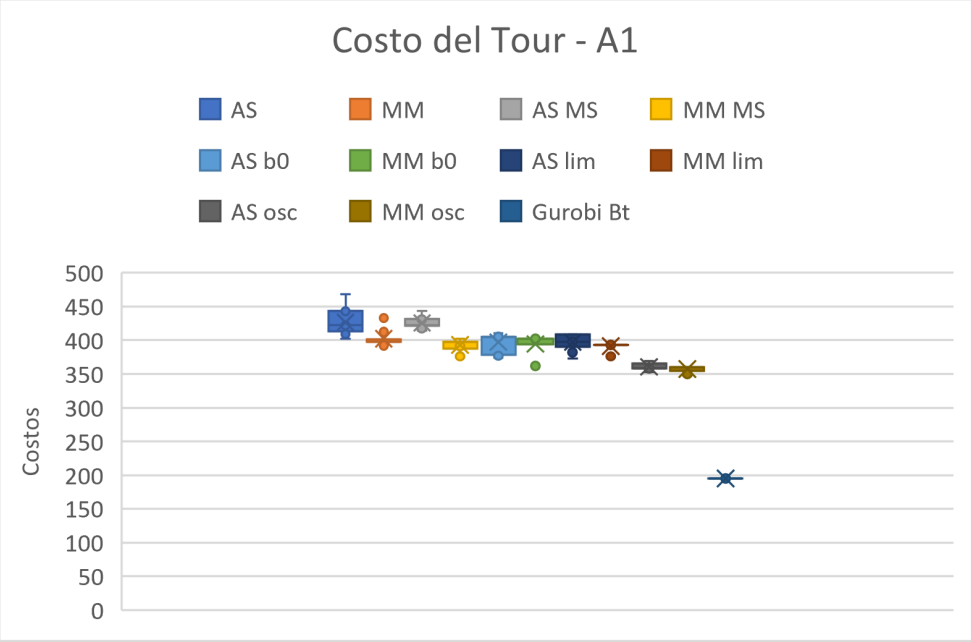


Figura 5.2: Comparación de costos - A1.

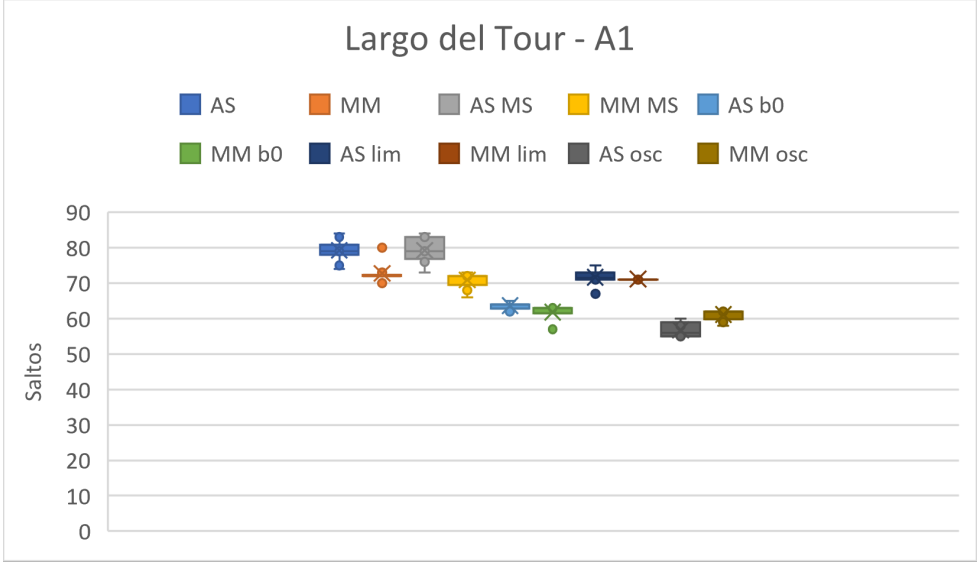


Figura 5.3: Comparación del largo total del tour - A1.

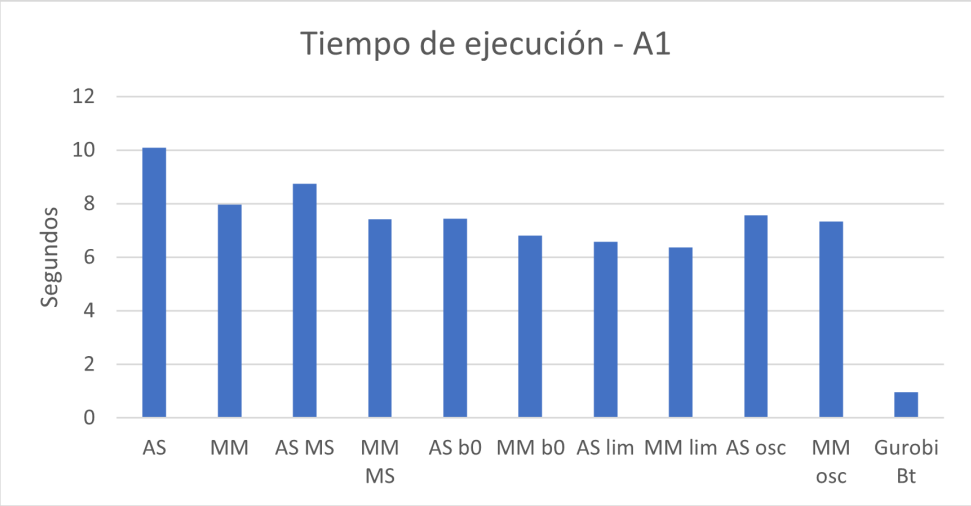


Figura 5.4: Comparación de tiempo de ejecución - A1.

Como es natural de esperar, los mejores resultados tanto en tiempo como en calidad fueron los obtenidos con Gurobi y Backtracking, dado que son métodos completos. Por el lado de las metaheurísticas, las mejores opciones fueron ambos algoritmos con el mecanismo oscilador, seguido por MAX-MIN Ant System con beta0.

5.2.2. Instancia CasoRealPequeño

Los resultados reportados por Gurobi indicaron que fueron encontradas 2 soluciones en 0.01 segundos, en donde el valor mas bajo fue 956 en términos de costo. El nodo de entrada de esta ruta fue el 32709 y el de salida fue el 32677. A continuación se muestra el resultado obtenido como solución al modelo matemático por Gurobi. Se listan los valores de las variables de decisión para cada arco recorrido, esto indica la cantidad de veces que el arco $x_{(i,j)}$ fue recorrido:

$x_{(32719,32700)} = 0$	$x_{(32700,32719)} = 1$	$x_{(32676,32677)} = 1$	$x_{(32718,32501)} = 1$	$x_{(32490,32657)} = 1$	$x_{(32490,32460)} = 1$
$x_{(32700,32460)} = 0$	$x_{(32719,32718)} = 1$	$x_{(51553,32657)} = 1$	$x_{(32619,32518)} = 1$	$x_{(32518,32519)} = 1$	$x_{(32519,32501)} = 1$
$x_{(32718,32719)} = 0$	$x_{(32657,32490)} = 1$	$x_{(32519,32518)} = 0$	$x_{(32519,32676)} = 1$	$x_{(32677,32676)} = 0$	$x_{(32501,32718)} = 0$
$x_{(32676,32519)} = 0$	$x_{(32501,32490)} = 1$	$x_{(32460,32490)} = 0$	$x_{(32676,51556)} = 1$	$x_{(32501,32519)} = 1$	$x_{(32460,32700)} = 1$
$x_{(32657,32658)} = 1$	$x_{(32490,32501)} = 0$	$x_{(32657,51553)} = 1$	$x_{(32658,32657)} = 1$	$x_{(51556,32676)} = 1$	$x_{(32709,32619)} = 1$

La ruta generada mediante Backtracking en base a la lista de arcos anterior es la que se observa en la figura 5.5.

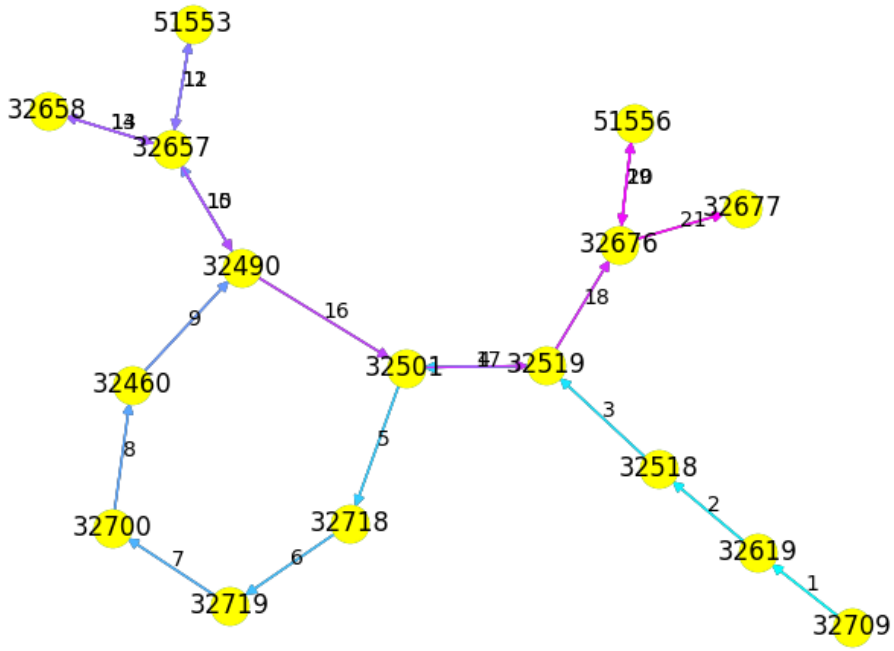


Figura 5.5: Ruta generada por Backtracking - CasoRealPequeño.

Los resultados para esta misma instancia resuelta mediante las metaheurísticas ACO se muestran en las figuras 5.6, 5.7 y 5.8:



Figura 5.6: Comparación de costos - CasoRealPequeño.

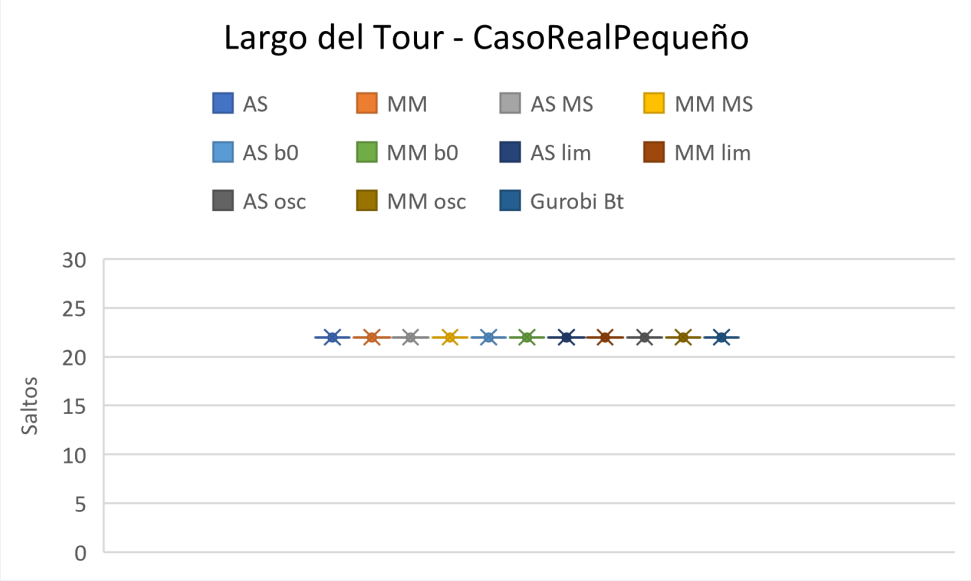


Figura 5.7: Comparación del largo total del tour - CasoRealPequeño.

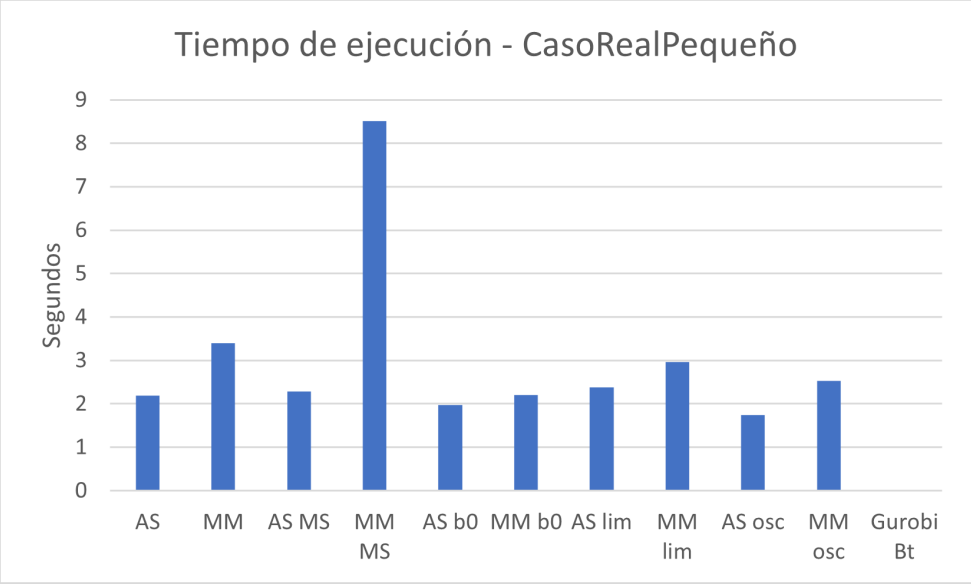


Figura 5.8: Comparación de tiempo de ejecución - CasoRealPequeño.

En este caso, todas las versiones de la metaheurística alcanzaron el óptimo global. Esto se debe a la baja complejidad de la instancia. Las únicas diferencias ocurrieron en torno al tiempo que le tomó a cada algoritmo completar sus ejecuciones, en donde la mejor fue Ant System con oscilador, seguido por Ant System con beta0. En el caso de Gurobi, encontrar la solución óptima fue prácticamente inmediata y no se alcanza a apreciar en el gráfico.

5.2.3. Instancia CasoRealMediano

A partir de este punto Gurobi con Backtracking no puede encontrar la ruta óptima, aunque si reporta una posible solución global. Esta última es agregada al gráfico de largo por motivos de comparación.

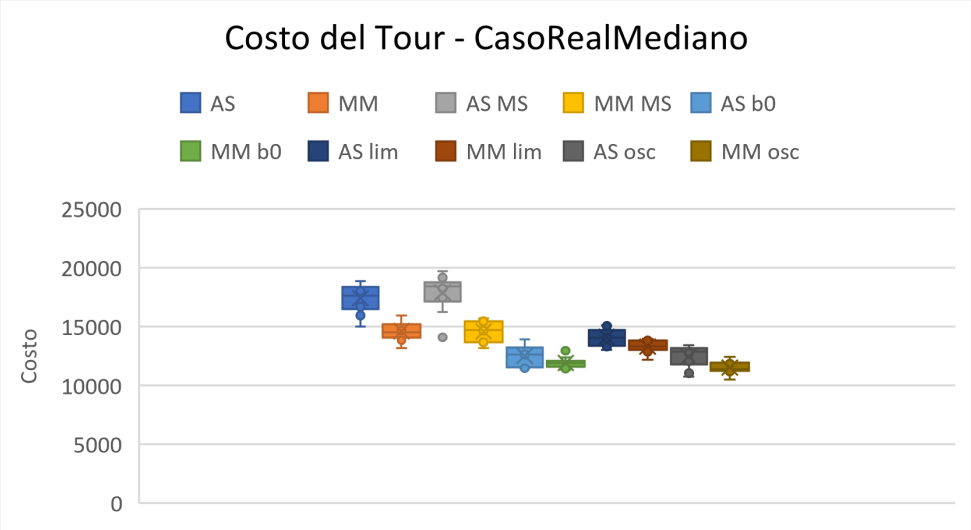


Figura 5.9: Comparación de costos - CasoRealMediano.

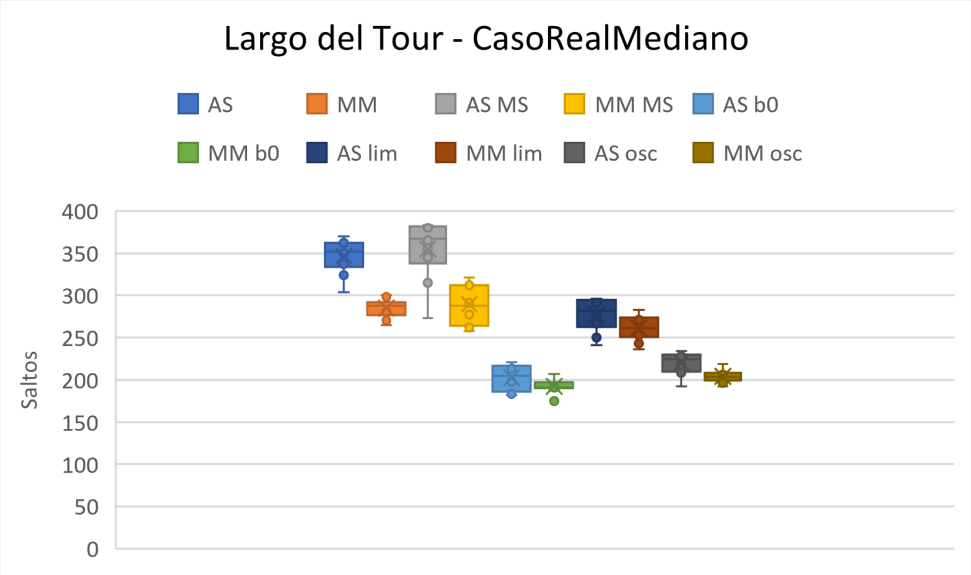


Figura 5.10: Comparación del largo total del tour - CasoRealMediano.

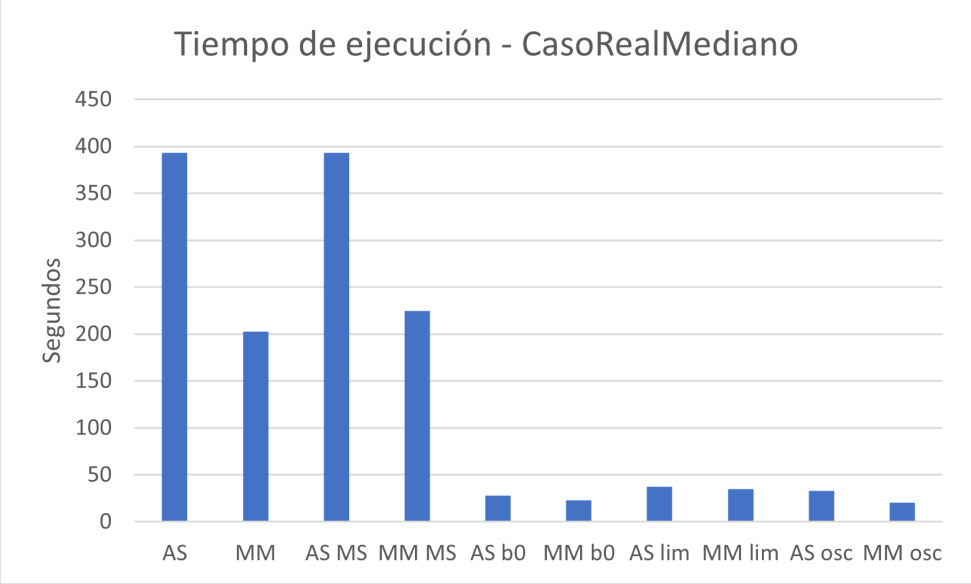


Figura 5.11: Comparación de tiempo de ejecución - CasoRealMediano.

Si bien, el software que maneja los algoritmos puede crear una visualización de las rutas encontradas, se decidió no mostrarlas debido a la baja resolución de la imagen generada y la superposición de las etiquetas de nodos y orden de pasos, que impide ver claramente el recorrido.

Para esta instancia los mejores resultados en términos de tiempo y calidad fueron obtenidos tanto por Ant System como MAX-MIN Ant System, ambos con beta0 y oscilador. Los segundos mejores resultados se obtienen con los mismos algoritmos, pero utilizando el limitador. Se destaca una notable mejora en los tiempos de ejecución cuando se utilizan los mecanismos de beta0, limitador, oscilador y matriz de salida, este último en menor medida.

5.2.4. Instancia CasoRealGrande

Para esta instancia no fue posible obtener resultados en un tiempo razonable para los algoritmos base sin mecanismos adicionales. Por esto, solo se mostrará una comparación entre Ant System y MAX-MIN Ant System, ambos con los mecanismos previamente utilizados. Al igual que la instancia anterior, solo fue posible obtener el largo óptimo de la ruta utilizando Gurobi, por lo que fue incluido en el gráfico 5.13. Tampoco es posible visualizar claramente el recorrido de la solución.



Figura 5.12: Comparación de costos - CasoRealGrande.



Figura 5.13: Comparación del largo total del tour - CasoRealGrande.

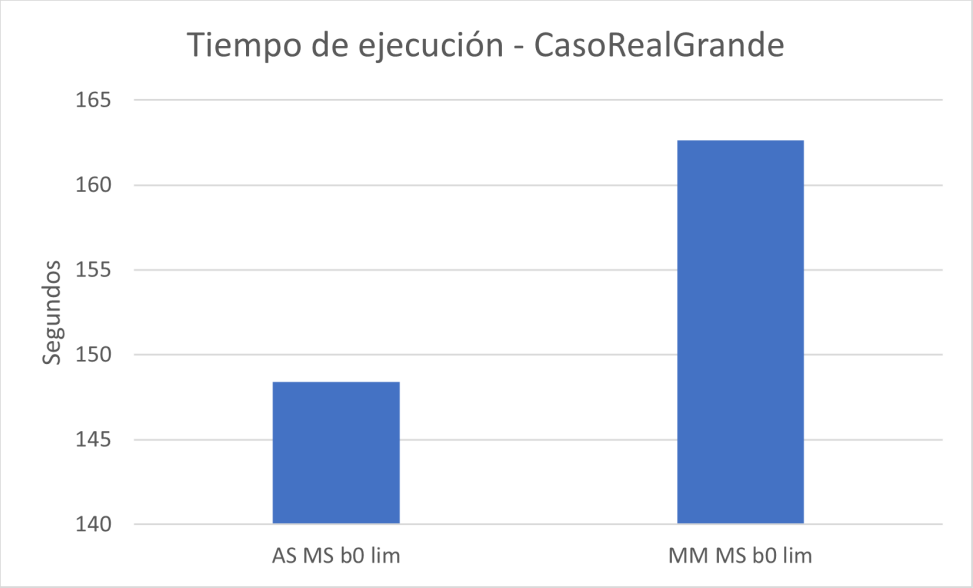


Figura 5.14: Comparación de tiempo de ejecución - CasoRealGrande.

5.2.5. Comparación Entre Resultados de ACO

En base a los gráficos mostrados anteriormente es posible notar que existe una diferencia notable de rendimiento en las instancias mas complejas como A1, CasoRealMediano. La mayor diferencia corresponde al uso mecanismos adicionales en ambas versiones del algoritmo, Ant System y MAX-MIN Ant System, dichos mecanismos mejoran tanto la calidad como el tiempo de ejecución en distintas medidas según la instancia. Por otra parte, las diferencias entre Ant System y MAX-MIN Ant System son mas notorias cuando se compara el tiempo que tarda cada uno. El rendimiento de uno u otro método puede variar según la instancia, siendo para algunas mas favorable Ant System y para otras MAX-MIN Ant System.

Mediante gráficos de convergencia, podemos notar como la ejecución del algoritmo va encontrando mejores soluciones a medida que avanza, tanto en evaluaciones, como en épocas, estas últimas marcadas por colores. A continuación en las figuras **??**, **??**, **??** y **??** podemos ver el comportamiento del algoritmo en todas las ejecuciones combinadas en cada método para la instancia CasoRealMediano. Los gráficos de convergencia de las demás instancias se podrán encontrar en el Anexo B.

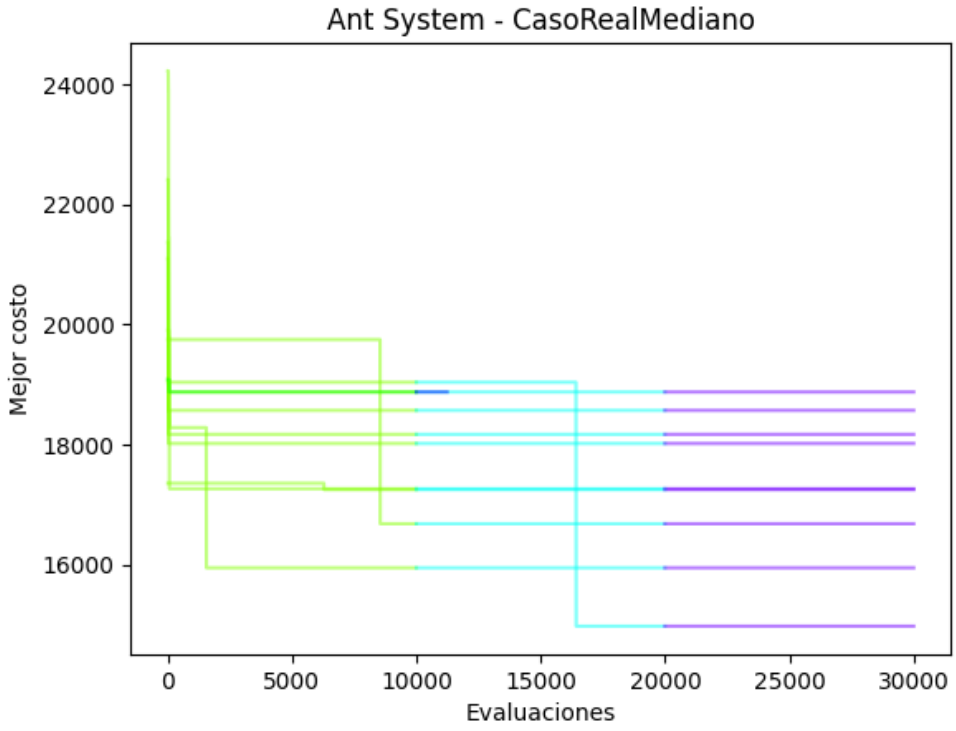


Figura 5.15: Gráfico de convergencia - Ant System - CasoRealMediano.

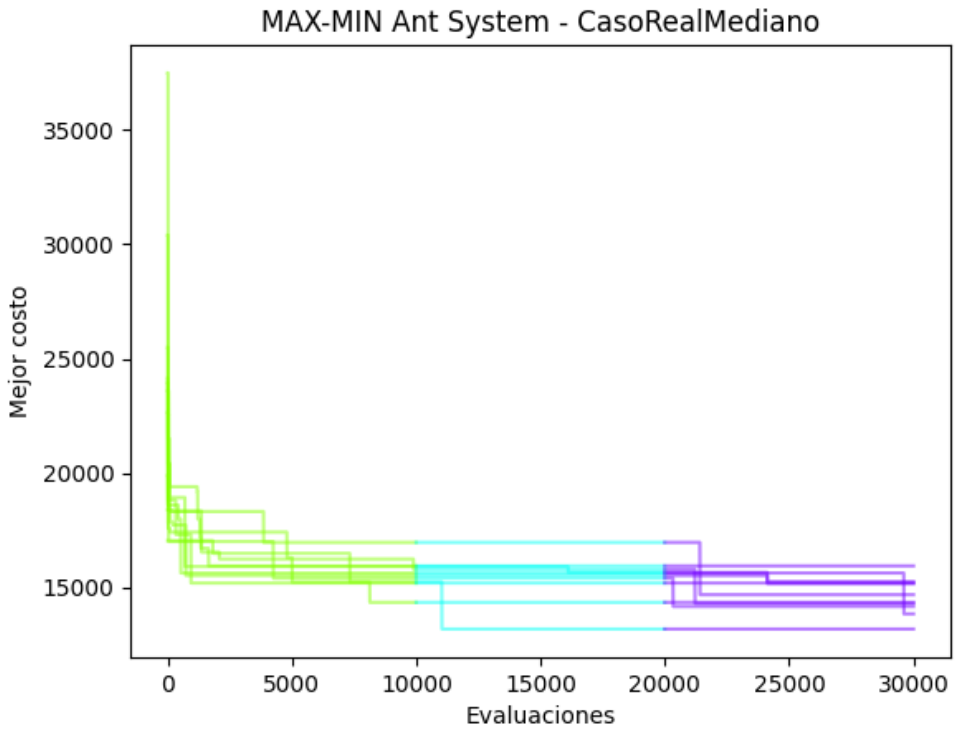


Figura 5.16: Gráfico de convergencia - MAX-MIN Ant System - CasoRealMediano.

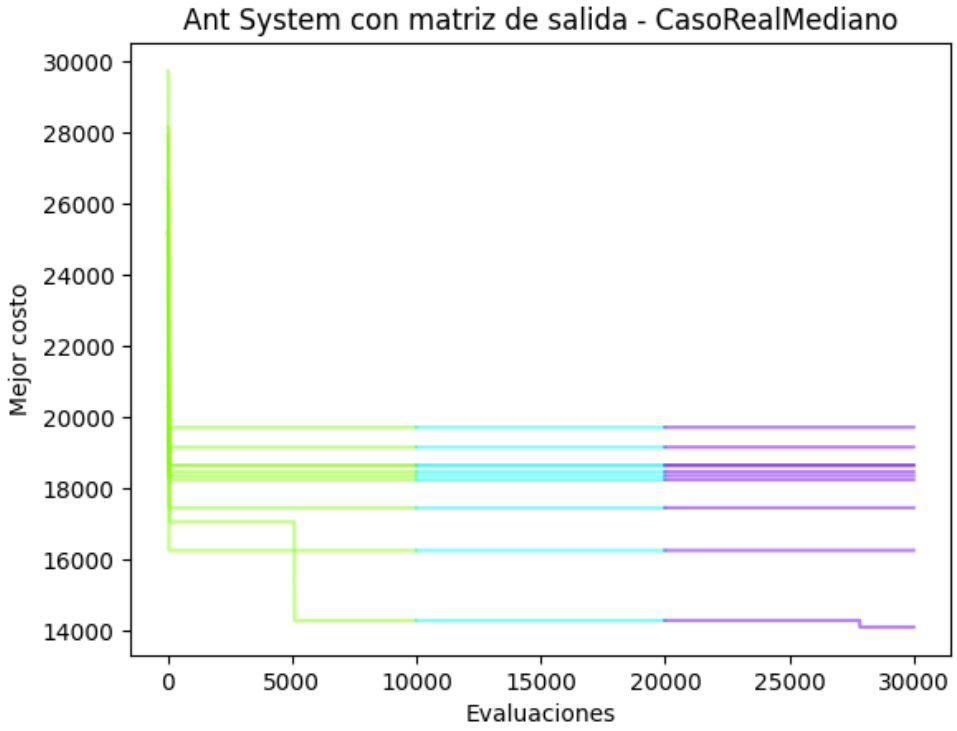


Figura 5.17: Gráfico de convergencia - Ant System con matriz de salida - CasoRealMediano.

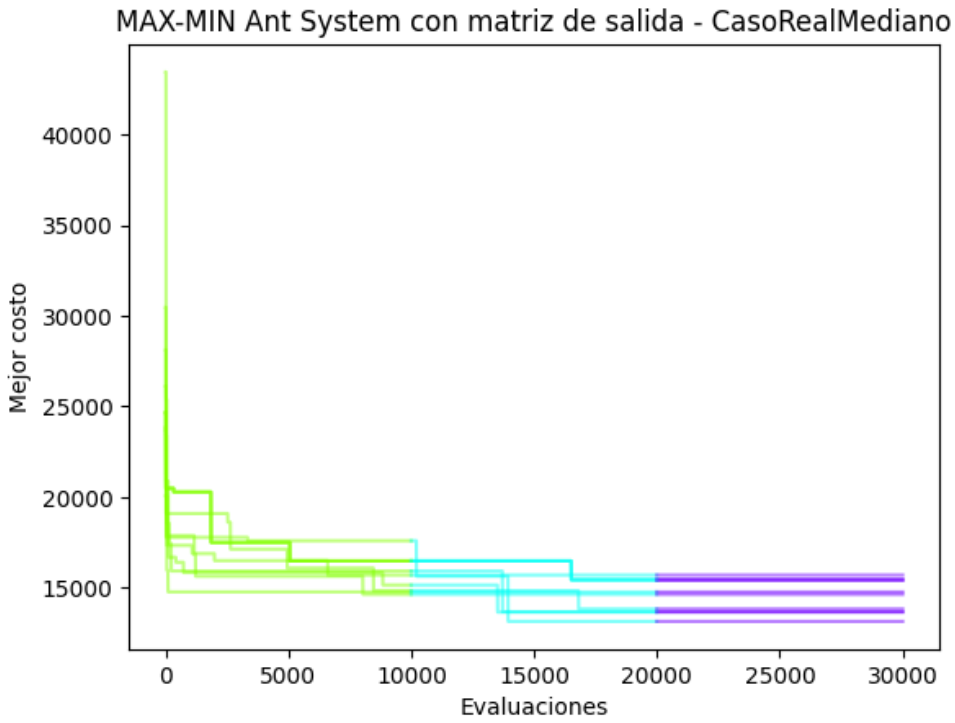


Figura 5.18: Gráfico de convergencia - MAX-MIN Ant System con matriz de salida - CasoRealMediano.

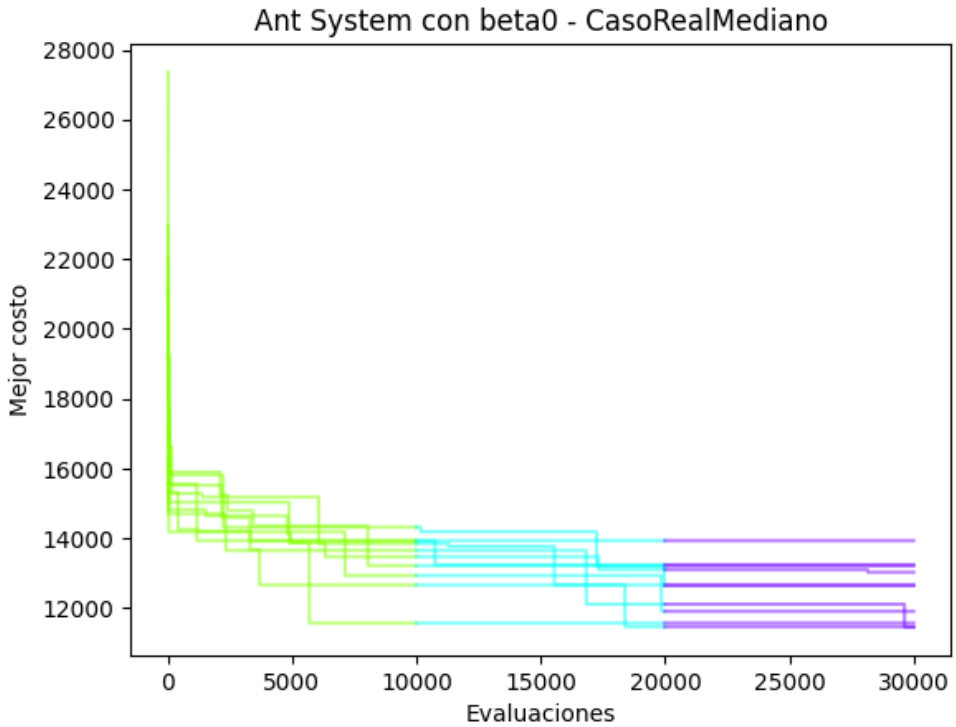


Figura 5.19: Gráfico de convergencia - Ant System con beta0 - CasoRealMediano.

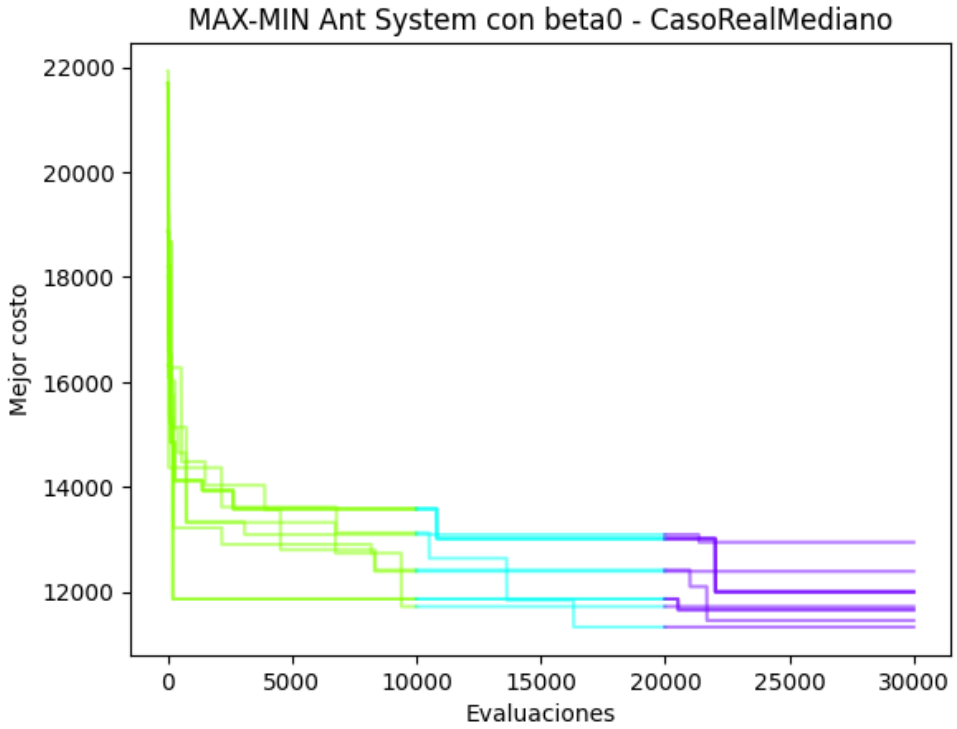


Figura 5.20: Gráfico de convergencia - MAX-MIN Ant System con beta0 - CasoRealMediano.

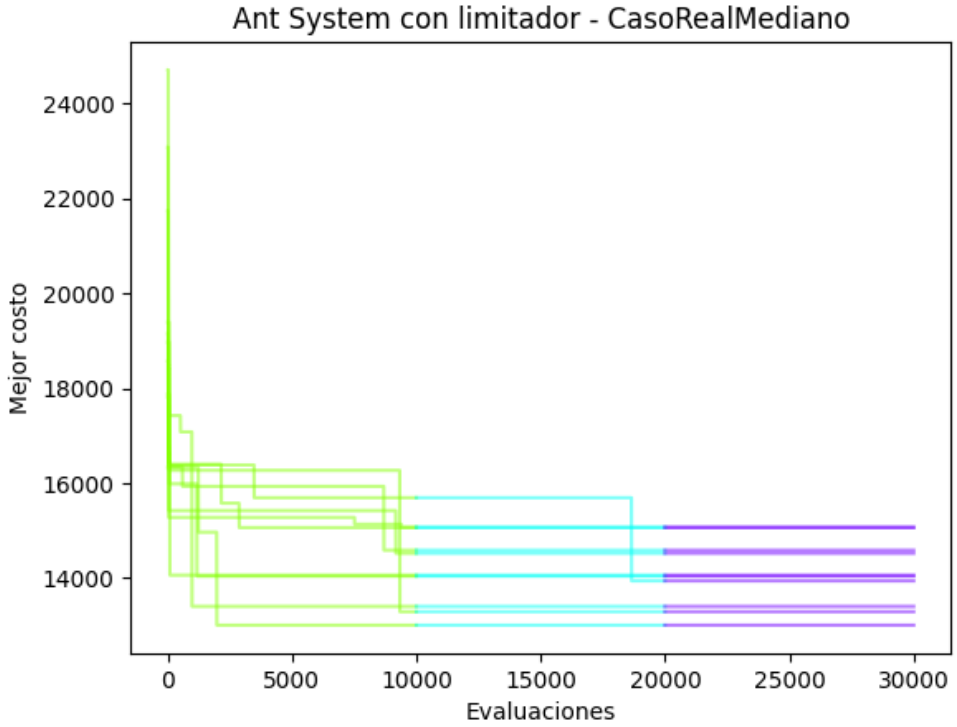


Figura 5.21: Gráfico de convergencia - Ant System con limitador - CasoRealMediano.

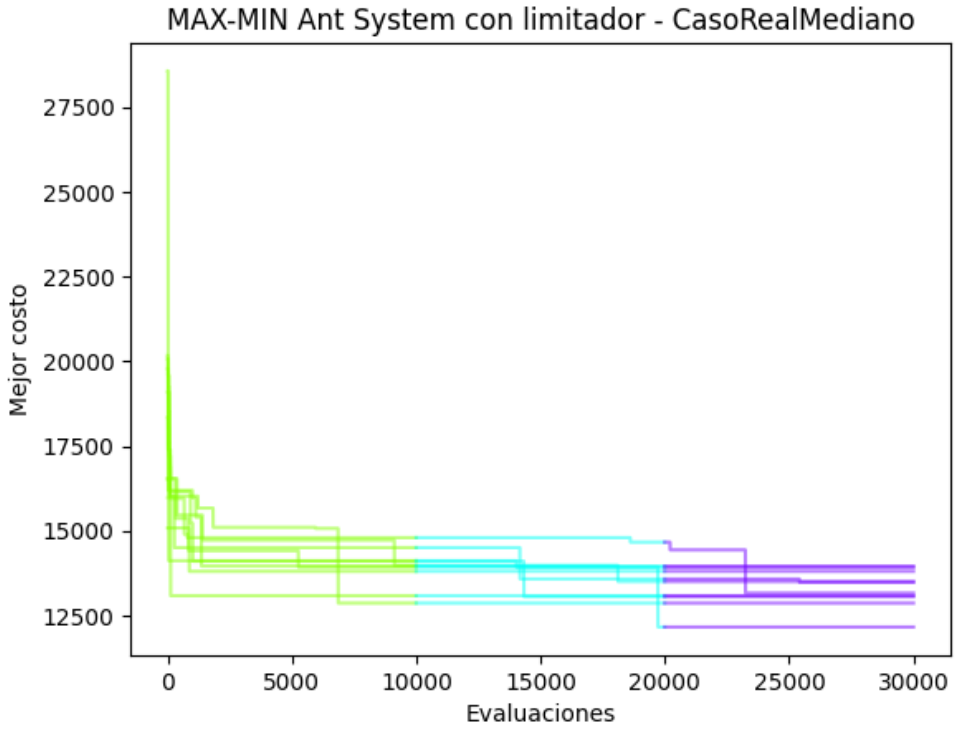


Figura 5.22: Gráfico de convergencia - MAX-MIN Ant System con limitador - CasoRealMediano.

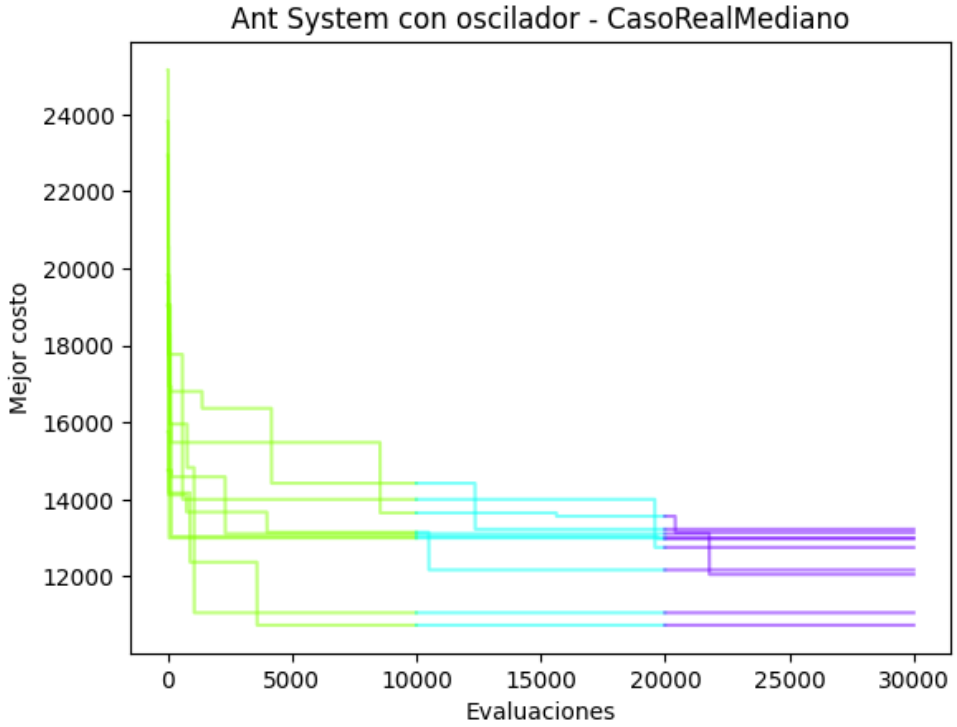


Figura 5.23: Gráfico de convergencia - Ant System con oscilador - CasoRealMediano.

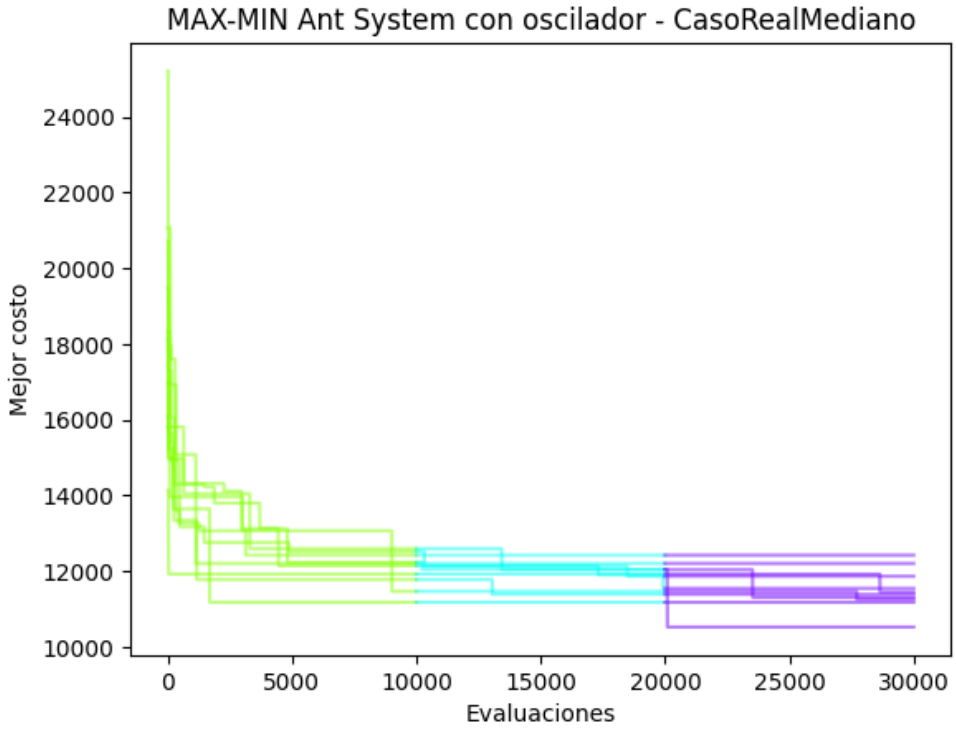


Figura 5.24: Gráfico de convergencia - MAX-MIN Ant System con oscilador - CasoRealMediano.

En estos gráficos podemos notar como en la primera época se encuentran buenas soluciones de manera muy agresiva, para luego comenzar a encontrar soluciones ligeramente mejores en las evaluaciones posteriores. En estos gráficos también se puede observar como la utilización del concepto de épocas puede aportar a la exploración del conjunto de posibles soluciones. Se destaca también que, el método Ant System tiene una preferencia por la exploración, mientras que MAX-MIN Ant System lo tiene por la exploración, este efecto se puede intensificar o disminuir con un mecanismo u otro. Por ejemplo, Ant System con matriz de salida es mas exploratorio que Ant System con beta0. Esto se nota por la dispersión de los gráficos.

5.2.6. Resultados del Finetuning

En esta sección se mostrarán los resultados de la sintonización fina según lo reportado por irace. Los resultados consisten en una serie de tablas y gráficos que aportan información estadística de los experimentos realizados. La información que será mostrada no solo incluye a las mejores configuraciones, también llamadas Elite, sino también para todas las pruebas realizadas por el configurador.

En cuanto a las mejores configuraciones en términos de parámetros todas comparten los mismos componentes. Los mecanismos utilizados por estas configuraciones fueron: limitador de pasadas con valor 1; ignorar la información heurística en recorrido o beta0; matriz de salida; y método de resolución Ant System. Solo difieren en los valores de los parámetros α, β_{sa}, ρ_{sec} y ρ_{sa}. La tabla 5.6 muestra estos valores.

.ID.	algoritmo	hormigas	α	β_{sa}	ρ	ρ_{sec}	ρ_{sa}	τ_0
340	Ant System	14	4.5966	0.1181	0.2847	0.2961	0.2519	6.1541
289	Ant System	13	4.6545	0.1002	0.2911	0.2835	0.2088	3.7126
262	Ant System	9	4.7635	0.0175	0.273	0.1594	0.1397	5.9303
346	Ant System	16	4.0754	0.049	0.2926	0.2561	0.1265	3.3429
219	Ant System	17	4.7066	0.0515	0.2907	0.2132	0.1281	4.2803
294	Ant System	13	4.7617	0.0254	0.2716	0.2163	0.1468	7.9705

Tabla 5.6: Valores de las 6 mejores configuraciones

De estos resultados se desprende que, para instancias grandes o medianas, es preferible usar el algoritmo base Ant System, con una cantidad intermedia de hormigas. Es destacable que se haya preferido valores altos para α y valores bajos para β_{sa}, ρ_{sec} y ρ_{sa}. Esto tiene sentido, ya que, permite encontrar una salida rápidamente sin considerar dema-

siado el costo de atravesar un arco. Los valores bajos de ρ, ρ_{sec} y ρ_{sa} aportan un equilibrio entre evaporación de feromonas para soluciones desfavorables y convergencia a soluciones favorables. Por otra parte el valor de τ_0 no es relevante, ya que solo se trata del valor inicial de las feromonas.

A continuación, las figuras 5.25 y 5.26 muestran un gráfico de violín para las 3 mejores configuraciones del conjunto de validación en términos de *Ratio of Performance to Deviation* (RPD) y costo respectivamente.

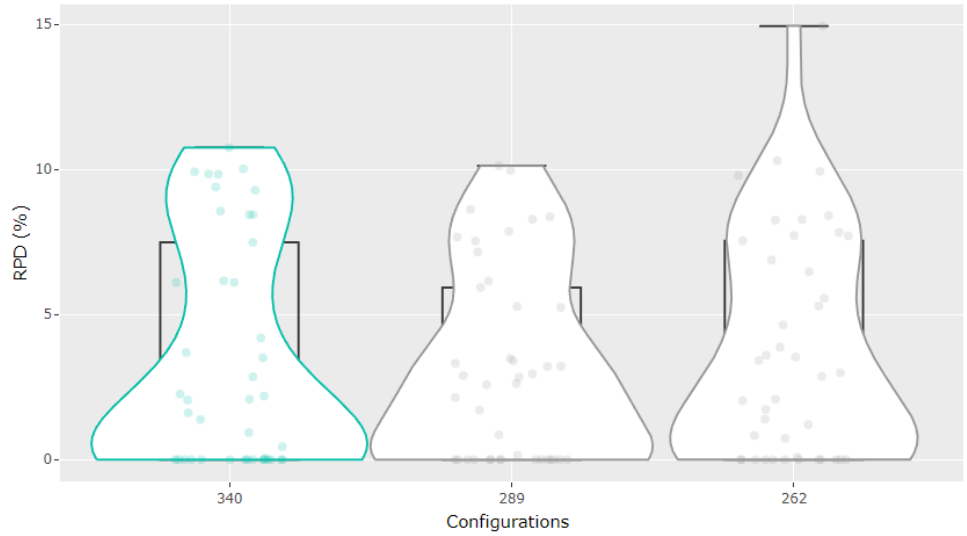


Figura 5.25: Gráfico de violín - Mejores configuraciones Validación.

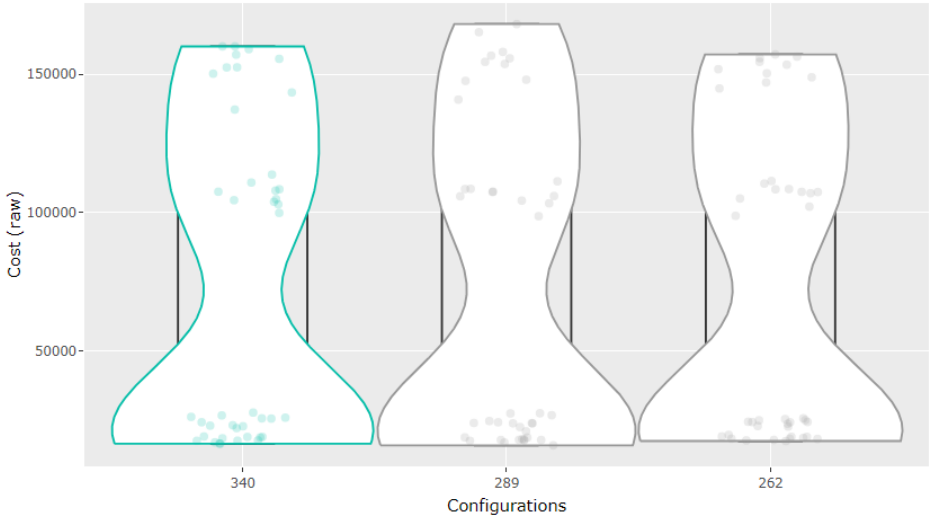


Figura 5.26: Gráfico de violín - Mejores configuraciones Costo Validación.

La tabla 5.7 muestra algunas estadísticas de las configuraciones mostradas en los

gráficos anteriores.

ID	n_instancias	promedio	d. estándar	mediana	min	max
340	42	72914.21904761904	57228.08851625239	27245.55	16506.5	160012
289	42	73217.67857142857	58019.01390477274	27487.55	15939.9	168058
262	42	72803.0738095238	56893.59179580198	25551.65	17488	157111

Tabla 5.7: Top 3 configuraciones de validación

Estas 3 configuraciones son similares en términos de calidad de la solución, aunque irace considera que la mejor fue la 340.

Para el conjunto de entrenamiento o búsqueda de los mejores parámetros se muestran los mejores 6 resultados, los datos son los siguientes. En las figuras 5.27 y 5.28 se muestran sus respectivos gráficos de violín.

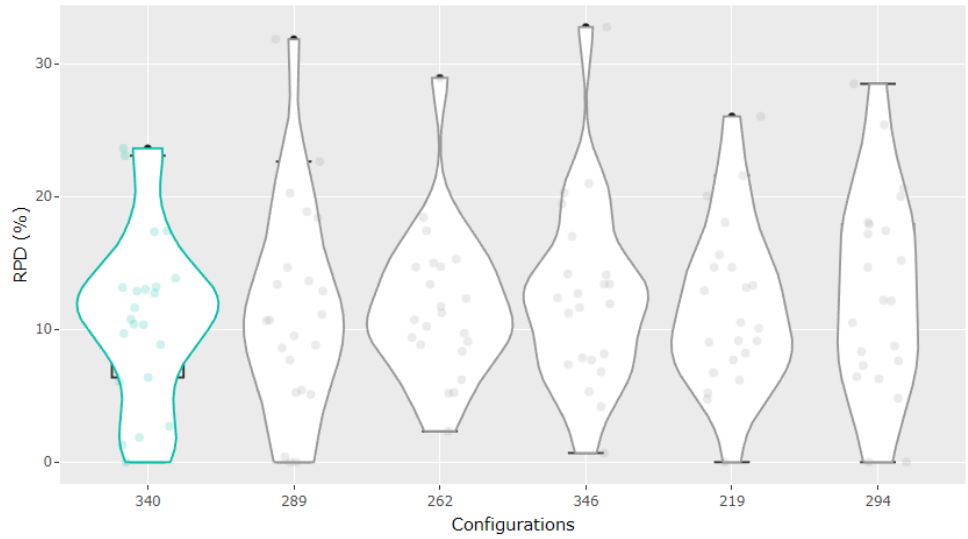


Figura 5.27: Gráfico de violín - Mejores configuraciones - Entrenamiento.

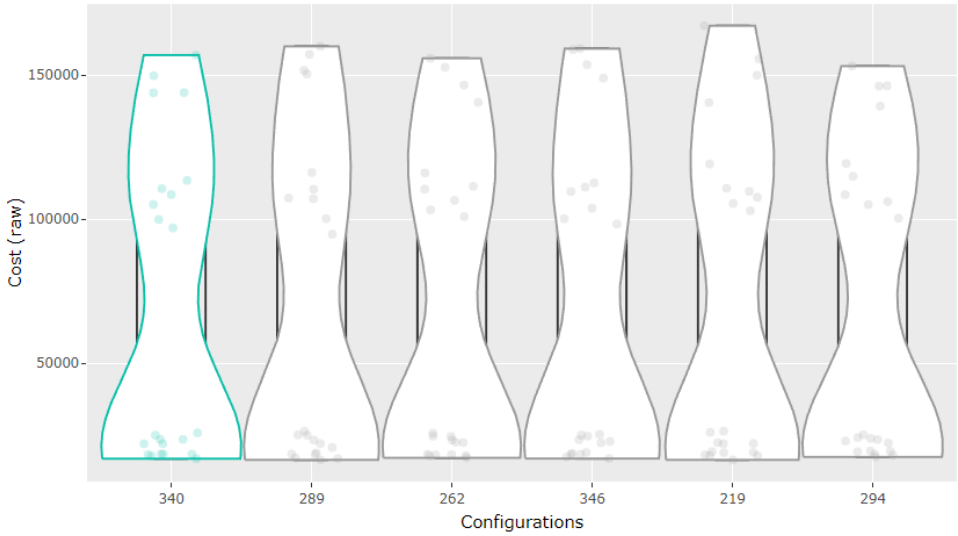


Figura 5.28: Gráfico de violín - Mejores configuraciones Costo - Entrenamiento.

Mientras que en la tabla 5.8 se muestran sus datos estadísticos

ID	n_instancias	promedio	d. estándar	mediana	min	max
340	22	67284.28181818181	54159.34353198757	25481.2	16895	156958
289	22	68405.7	56078.56712988441	25806.4	16513.9	159994
262	22	68010.4909090909	54683.93975881427	25219.45	17223.8	155852
346	22	68628.73636363636	55968.44597511254	25236.8	16970	159211
219	22	68936.5	56434.43807973318	26288.25	16506.3	167150
294	22	67919.55	54003.61591787606	24745.7	17532.7	153126

Tabla 5.8: Top 6 configuraciones de entrenamiento

En el Anexo C se puede encontrar una visualización de coordenadas paralelas, tanto para las soluciones Elite, como para todas las configuraciones que fueron testeadas.

6. Metodología de Trabajo

Este trabajo se realiza dentro del marco de un proyecto FONDEF. Esto significa colaborar con distintos equipos de trabajo donde cada uno contribuye con sus respectivas tareas, siendo algunas de estas la recopilación de datos de los camiones recolectores y calles por donde pasan, *clustering* o agrupamiento de calles, desarrollo de interfaces gráficas o creación de un algoritmo que solucione las instancias provistas, que es el tema que aborda esta tesis. En el transcurso del proyecto, se participa como expositor en un coloquio sobre Sistemas de Recolección Colaborativa de Residuos Sólidos Domiciliarios, instancia que permitió la presentación del algoritmo desarrollado. Asimismo, se asiste a un workshop, evento que brindó la oportunidad de exponer el trabajo a un público especializado, contando con la presencia de uno de los coautores del algoritmo MAX-MIN, Thomas Stützle.

Para lograr los objetivos planteados en este trabajo, es necesario realizar una investigación sobre el Estado del Arte de problemas de ruteo para conocer cual es el conocimiento actual sobre ésta área. También es necesario, para contextualizar nuestro trabajo, conocer las aplicaciones del mundo real de las distintas maneras en que se ha tratado de resolver el problema del manejo de desperdicios sólidos municipales. Luego de haber recopilado suficiente información, se procede a definir el problema en base a los conocimientos recabados y elaborar un modelo matemático que permita representar el problema. Esto nos permitió tener una base teórica sólida sobre la cual trabajar y generar instancias que sirvieron de prueba mientras se desarrollaba el algoritmo.

La fase de investigación involucra la búsqueda y lectura de publicaciones académicas realizadas por otros investigadores, ya sea para universidades, revistas científicas o medios especializados. Esto asegura que la información es fidedigna y ha sido revisada. El contenido que se encuentra en este documento se encuentra acompañado de su referencia original, con el fin de evitar plagio.

Posteriormente se diseña un algoritmo que use una o mas técnicas para resolver el problema formulado. Luego viene la fase de experimentación, para buscar cual es la mejor alternativa de solución. Esta fase es crucial en el proyecto, ya que, permite conocer empíricamente como se comportan los distintos algoritmos y configuraciones de estos en distintos escenarios. Paralelamente se trabaja en una fase de ajuste fino mediante el software irace, donde se prueban distintos valores para los parámetros que modifican el comportamiento del algoritmo y por consiguiente, su rendimiento. Esto con el fin de encontrar cual es la mejor configuración, es decir, que entregue una solución de calidad en el menor tiempo posible.

En general, este trabajo se realiza de manera iterativa. Los procesos de generación de instancias, experimentación, rediseño y mejoramiento del algoritmo ocurrieron múltiples veces. Por otra parte, los procesos de análisis de resultados y reporte ocurrieron de manera paralela al proceso de desarrollo, los resultados fueron obtenidos y reportados en distintas fases del desarrollo del algoritmo. Dichos procesos se extendieron hasta la fecha de entrega

de este documento. Cabe destacar que todos los procesos involucrados en este proyecto se retroalimentan entre sí.

La figura 6.1 resume el proceso de trabajo a modo de carta Gantt. Es posible que este formato no sea el mas apropiado para reflejar un trabajo cíclico, de todas maneras se muestran los procesos como iteraciones cuyas dependencias requieren del proceso anterior. Esto con el fin de representar las iteraciones realizadas.

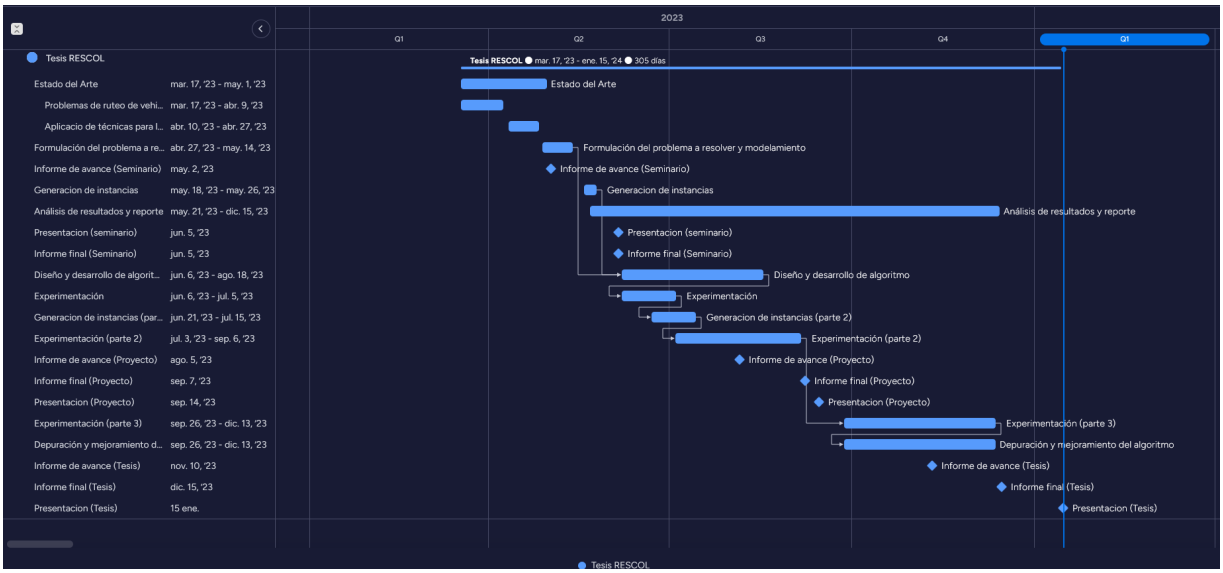


Figura 6.1: Carta Gantt Tesis RESCOL.

En el anexo D se puede ver la figura a escala completa.

Conclusión

En esta tercera iteración del trabajo de Tesis se lograron los objetivos específicos relacionados al planteamiento, evaluación, desarrollo y mejora de alternativas de solución, al igual que la definición de casos de prueba e instancias a ser resueltas. Esto se suma a los objetivos logrados previamente en la iteración anterior, relacionados a la definición del problema y planteamiento del Estado del Arte de este.

En base al trabajo realizado previamente, donde se estableció un modelo del problema, formato de instancias y contexto, fue posible el desarrollo de una metaheurística Ant Colony Optimization base permite ser ampliada a otras metaheurísticas derivadas de la familia ACO. Con el propósito de explorar distintas maneras de resolver el problema y evaluar el desempeño de cada uno. A esta versión base le fueron agregados dos métodos de resolución, Ant System y MAX-MIN, que son algunos de los algoritmos mas importantes de los derivados de ACO. Si bien, se pudo implementar Any Colony System, no se realizaron pruebas con este método debido a que no se logró estabilizar su funcionamiento en pruebas exhaustivas.

Se logró una mejora del rendimiento base del algoritmo gracias a la implementación de componentes adicionales. Estos componentes fueron diseñados para hacer mas eficientes e inteligentes a las hormigas en cuanto a las decisiones que toman para crear las rutas. Se destacan los mecanismos para limitar la cantidad de pasadas por cada arco y darle prioridad a los que no han sido visitados; la matriz de feromonas de salida, que permite a las hormigas aprender sobre las rutas mas cortas para salir del grafo; el componente oscilador, que permite modificar los parámetros del algoritmo en tiempo de ejecución; y el comportamiento beta0, que ignora la información heurística del grafo. Todos estos componentes en conjunto resultaron en una gran mejora en tiempo y calidad de solución.

Mediante la generación de instancias sintéticas derivadas de instancias reales y el uso de irace fue posible encontrar un conjunto óptimo de parámetros para este conjunto de instancias. De esta manera se puede extrapolar el comportamiento del algoritmo para situaciones similares a las resueltas, esto en término de los mecanismos y valores de parámetros elegidos al momento de generar un tour para la instancia seleccionada. Adicionalmente, gracias a las pruebas de irace, fue posible detectar y corregir en su mayoría, potenciales problemas de rendimiento del algoritmo. También fue posible determinar empíricamente rangos de valores favorables y perjudiciales para los parámetros clave como α, β y ρ.

A partir de los resultados de las cinco instancias iniciales, dos de prueba y tres del mundo real fue posible establecer una base sobre la cual trabajar. También permitió obtener métricas que resultaron vitales para comparar el rendimiento del algoritmo y calidad de las soluciones generadas. Basándose en el costo y largo de las rutas, además del tiempo que toma generarlas, el algoritmo fue pasando por distintas fases de depuración y mejora.

Como trabajo futuro se plantea seguir trabajando en la metaheurística para lograr mejores resultados. Esto mediante la implementación de mecanismos avanzados, búsqueda local u otros métodos de resolución. Si bien existe una extensa literatura sobre problemas de ruteo y maneras de solucionarlos, siempre existe la posibilidad de experimentar con nuevas formas de resolver estos problemas.

Bibliografía

[1] Navid Akbarpour et al. «An innovative waste management system in a smart city under stochastic optimization using vehicle routing problem». En: *Soft Computing* 25 (2021), págs. 6707-6727.

[2] Mazhar Ansari Ardeh. «Transfer Optimisation in Genetic Programming for Solving Uncertain Capacitated Arc Routing Problem». Tesis doct. Victoria University of Wellington/Te Herenga Waka, 2022.

[3] Claudia Archetti y Maria Grazia Speranza. «Vehicle routing problems with split deliveries». En: *International transactions in operational research* 19.1-2 (2012), págs. 3-22.

[4] Ramiz Assaf y Yahya Saleh. «Vehicle-routing optimization for municipal solid waste collection using genetic algorithm: The case of southern Nablus city». En: *Civil and environmental engineering reports* 26.3 (2017), págs. 43-57.

[5] Rubul Kumar Bania. «International Journal of Scientific Research and Reviews». En: ().

[6] Gavina Baralla et al. «Waste management: A comprehensive state of the art about the rise of blockchain technology». En: *Computers in Industry* 145 (2023), pág. 103812.

[7] Edward J Beltrami y Lawrence D Bodin. «Networks and vehicle routing for municipal waste collection». En: *Networks* 4.1 (1974), págs. 65-94.

[8] Bernd Bilitewski, Georg Härdtle y Klaus Marek. *Waste management*. Springer Science & Business Media, 1996.

[9] Lawrence D Bodin y Samuel J Kursh. «A computer-assisted system for the routing and scheduling of street sweepers». En: *Operations Research* 26.4 (1978), págs. 525-537.

[10] Eleni Boumpa et al. «A Review of the Vehicle Routing Problem and the Current Routing Services in Smart Cities». En: *Analytics* 2.1 (2022), págs. 1-16.

[11] Kris Braekers, Katrien Ramaekers e Inneke Van Nieuwenhuyse. «The vehicle routing problem: State of the art classification and review». En: *Computers & industrial engineering* 99 (2016), págs. 300-313.

[12] Gustavo Braier et al. «An integer programming approach to a real-world recyclable waste collection problem in Argentina». En: *Waste Management & Research* 35.5 (2017), págs. 525-533.

[13] José Brandão y Richard Eglese. «A deterministic tabu search algorithm for the capacitated arc routing problem». En: *Computers & Operations Research* 35.4 (2008), págs. 1112-1126.

[14] Bernd Bullnheimer, Richard F Hartl y Christine Strauss. «An improved ant system algorithm for the vehicle routing problem». En: *Annals of operations research* 89 (1999), págs. 319-328.

[15] Ann Melissa Campbell y Jill Hardin Wilson. «Forty years of periodic vehicle routing». En: *Networks* 63.1 (2014), págs. 2-15.

[16] N Christofides et al. «An algorithm for the rural postman problem». En: *Report IC. OR* 81 (1981), pág. 81.

[17] Alberto Colorni, Marco Dorigo, Vittorio Maniezzo et al. «An Investigation of some Properties of an."Ant Algorithm".» En: *Ppsn*. Vol. 92. 1992. 1992.

[18] Alberto Colorni et al. «Ant system for job-shop scheduling». En: *JORBEL-Belgian Journal of Operations Research, Statistics, and Computer Science* 34.1 (1994), págs. 39-53.

[19] A. Corberán et al. *Arc Routing Problems: Data Instances*. 2022. URL: https://www.uv.es/corberan/instancias.htm (visitado 30-08-2023).

[20] Ángel Corberán, Isaac Plana y Jose Maria Sanchis. «Chapter 4: The Chinese Postman Problem on Directed, Mixed, and Windy Graphs». En: oct. de 2013, págs. 65-83. ISBN: 978-1-61197-366-2. DOI: 10.1137/1.9781611973679.ch4.

[21] Ángel Corberán et al. «Arc routing problems: A review of the past, present, and future». En: *Networks* 77.1 (2021), págs. 88-115.

[22] Daniele Costa y Alain Hertz. «Ants can colour graphs». En: *Journal of the operational research society* 48.3 (1997), págs. 295-305.

[23] G. Dantzig, R. Fulkerson y S. Johnson. «Solution of a Large-Scale Traveling-Salesman Problem». En: *Journal of the Operations Research Society of America* 2.4 (1954), págs. 393-410. DOI: 10.1287/opre.2.4.393. eprint: https://doi.org/10.1287/opre.2.4.393. URL: https://doi.org/10.1287/opre.2.4.393.

[24] George B Dantzig y John H Ramser. «The truck dispatching problem». En: *Management science* 6.1 (1959), págs. 80-91.

[25] Marco Dorigo y Luca Maria Gambardella. «Ant colony system: a cooperative learning approach to the traveling salesman problem». En: *IEEE Transactions on evolutionary computation* 1.1 (1997), págs. 53-66.

[26] Marco Dorigo y Thomas Stützle. *Ant Colony Optimization*. The MIT Press, jun. de 2004. ISBN: 9780262256032. DOI: 10.7551/mitpress/1290.001.0001. URL: https://doi.org/10.7551/mitpress/1290.001.0001.

[27] Moshe Dror, Helman Stern y Pierre Trudeau. «Postman tour on a graph with precedence relation on arcs». En: *Networks* 17.3 (1987), págs. 283-294.

[28] Burak Eksioglu, Arif Volkan Vural y Arnold Reisman. «The vehicle routing problem: A taxonomic review». En: *Computers & Industrial Engineering* 57.4 (2009), págs. 1472-1483.

[29] Maurizio Faccio, Alessandro Persona y Giorgia Zanin. «Waste collection multi objective model with real time traceability data». En: *Waste management* 31.12 (2011), págs. 2391-2405.

[30] Matthew J Franchetti. *Solid waste analysis and minimization: a systems approach*. McGraw-Hill Education, 2009.

[31] Asvin Goel y Volker Gruhn. «A General Vehicle Routing Problem». En: *European Journal of Operational Research* 191.3 (2008), págs. 650-660. ISSN: 0377-2217. DOI: https://doi.org/10.1016/j.ejor.2006.12.065. URL: https://www.sciencedirect.com/science/article/pii/S0377221707001233.

[32] DE Goldberg. «Genetic Algorithms in Search, Optimization and Machine Learning,. Addition-Westly». En: *Reading MA* (1989).

[33] Bruce L Golden y Richard T Wong. «Capacitated arc routing problems». En: *Networks* 11.3 (1981), págs. 305-315.

[34] Mei Gu Guan. «Graphic programming using odd or even points Chinese Mathematics». En: *V ol* 1.2 (1962), págs. 73-2.

[35] LLC Gurobi Optimization. *Gurobi Optimizer*. N/A. URL: `https://www.gurobi.com/solutions/gurobi-optimizer/` (visitado 29-05-2023).

[36] Hui Han y Eva Ponce Cueto. «Waste collection vehicle routing problem: literature review». En: *PROMET-Traffic&Transportation* 27.4 (2015), págs. 345-358.

[37] MA Hannan et al. «Radio Frequency Identification (RFID) and communication technologies for solid waste bin and truck monitoring system». En: *Waste management* 31.12 (2011), págs. 2406-2413.

[38] MA Hannan et al. «A review on technologies and their usage in solid waste monitoring and management systems: Issues and challenges». En: *Waste Management* 43 (2015), págs. 509-523.

[39] MA Hannan et al. «Capacitated vehicle-routing problem model for scheduled solid waste collection and route optimization using PSO algorithm». En: *Waste management* 71 (2018), págs. 31-41.

[40] IBM. *IBM CPLEX Optimizer*. N/A. URL: `https://www.ibm.com/mx-es/analytics/cplex-optimizer` (visitado 05-06-2023).

[41] División de Información y Economía Ambiental. *Estado del Medio Ambiente*. Inf. téc. Ministerio del Medio Ambiente de Chile, 2022.

[42] L. Jourdan, M. Basseur y E.-G. Talbi. «Hybridizing exact methods and metaheuristics: A taxonomy». En: *European Journal of Operational Research* 199.3 (2009), págs. 620-629. ISSN: 0377-2217. DOI: `https://doi.org/10.1016/j.ejor.2007.07.035`. URL: `https://www.sciencedirect.com/science/article/pii/S0377221708003597`.

[43] Muhammed Emre Keskin, Mustafa Yılmaz y Chefi Triki. «Solving the hierarchical windy postman problem with variable service costs using a math-heuristic algorithm». En: *Soft Computing* (2023), págs. 1-17.

[44] Scott Kirkpatrick, C Daniel Gelatt Jr y Mario P Vecchi. «Optimization by simulated annealing». En: *science* 220.4598 (1983), págs. 671-680.

[45] M Kiuchi et al. «An exact algorithm for the capacitated arc routing problem using parallel branch and bound method». En: *Spring national conference of the operational research society of Japan*. INFORMS Catonsville, MD, USA. 1995, págs. 28-29.

[46] Suresh Nanda Kumar y Ramasamy Panneerselvam. «A survey on the vehicle routing problem and its variants». En: (2012).

[47] Philippe Lacomme, Christian Prins y Wahiba Ramdane-Chérif. «A genetic algorithm for the capacitated arc routing problem and its extensions». En: *Applications of Evolutionary Computing: EvoWorkshops 2001: EvoCOP, EvoFlight, EvoIASP, EvoLearn, and EvoSTIM Como, Italy, April 18–20, 2001 Proceedings*. Springer. 2001, págs. 473-483.

[48] Philippe Lacomme, Christian Prins y Marc Sevaux. «A genetic algorithm for a bi-objective capacitated arc routing problem». En: *Computers & Operations Research* 33.12 (2006), págs. 3473-3493.

[49] Gilbert Laporte. «The vehicle routing problem: An overview of exact and approximate algorithms». En: *European journal of operational research* 59.3 (1992), págs. 345-358.

[50] Gilbert Laporte. «Fifty years of vehicle routing». En: *Transportation science* 43.4 (2009), págs. 408-416.

[51] Guillermo Leguizamon y Zbigniew Michalewicz. «A new version of ant system for subset problems». En: *Proceedings of the 1999 Congress on Evolutionary Computation-CEC99 (Cat. No. 99TH8406)*. Vol. 2. IEEE. 1999, págs. 1459-1464.

[52] Humberto Longo, Marcus Poggi De Aragao y Eduardo Uchoa. «Solving capacitated arc routing problems using a transformation to the CVRP». En: *Computers & Operations Research* 33.6 (2006), págs. 1823-1837.

[53] Manuel López-Ibáñez et al. «The irace package: Iterated racing for automatic algorithm configuration». English. En: *Operations Research Perspectives* 3 (2016). Publisher Copyright: © 2016, págs. 43-58. ISSN: 2214-7160. DOI: 10.1016/j.orp.2016.09.002.

[54] Vittorio Maniezzo y Alberto Colorni. «The ant system applied to the quadratic assignment problem». En: *IEEE Transactions on knowledge and data engineering* 11.5 (1999), págs. 769-778.

[55] Yi Mei, Ke Tang y Xin Yao. «Capacitated arc routing problem in uncertain environments». En: *IEEE Congress on Evolutionary Computation*. IEEE. 2010, págs. 1-8.

[56] Jairo R Montoya-Torres et al. «A literature review on the vehicle routing problem with multiple depots». En: *Computers & Industrial Engineering* 79 (2015), págs. 115-129.

[57] Johana Andrea Patiño Chirva, Yesica Xiomara Daza Cruz y Eduyn Ramiro Lopez-Santana. «A Hybrid Mixed-Integer Optimization and Clustering Approach to Selective Collection Services Problem of Domestic SolidWaste». En: *Ingeniería* 21.2 (2016), págs. 235-257.

[58] Municipalidad de Providencia. *Recolección de basura*. 2022. URL: https://providencia.cl/provi/site/artic/20220117/pags/20220117180607.html (visitado 17-01-2022).

[59] Rashmi Ranjan Rout et al. «Fuzzy logic-based emergency vehicle routing: An IoT system development for smart city applications». En: *Computers & Electrical Engineering* 88 (2020), pág. 106839.

[60] Luis Santos, João Coutinho-Rodrigues y John R Current. «An improved ant colony optimization based algorithm for the capacitated arc routing problem». En: *Transportation Research Part B: Methodological* 44.2 (2010), págs. 246-266.

[61] Yasufumi Saruwatari, Ryuichi Hirabayashi y Naonori Nishida. «Node duplication lower bounds for the capacitated arc routing problem». En: *Journal of the Operations Research Society of Japan* 35.2 (1992), págs. 119-133.

[62] Shanshu. *COPT Cardinal Optimizer*. N/A. URL: `https://www.shanshu.ai/copt` (visitado 05-06-2023).

[63] Amy Shoemaker y Sagar Vare. «Edmonds' blossom algorithm». En: *CME* (2016).

[64] Statista. *Global waste generation - statistics & facts*. 2023. URL: `https://www.statista.com/topics/4983/waste-generation-worldwide/#topicOverview` (visitado 23-03-2023).

[65] Thomas Stützle y Holger H Hoos. «MAX–MIN ant system». En: *Future generation computer systems* 16.8 (2000), págs. 889-914.

[66] Kaarthik Sundar, Saravanan Venkatachalam y Sivakumar Rathinam. «Formulations and algorithms for the multiple depot, fuel-constrained, multiple vehicle routing problem». En: *2016 American Control Conference (ACC)*. IEEE. 2016, págs. 6489-6494.

[67] Harold Thimbleby. «The directed chinese postman problem». En: *Software: Practice and Experience* 33.11 (2003), págs. 1081-1096.

[68] Thelma PB Vecchi et al. «A sequential approach for the optimization of truck routes for solid waste collection». En: *Process Safety and Environmental Protection* 102 (2016), págs. 238-250.

[69] Scott Allen Welz. *Optimal solutions for the capacitated arc routing problem using integer programming*. University of Cincinnati, 1994.

Anexo A: Tablas de métricas

Tabla 6.1: Resultados Gurobi + Backtracking - A1

Costo ruta	Longitud ruta	Nodo inicial	Nodo terminal	Tiempo resolución (s)	Tiempo modelo	Tiempo Backtracking
195	49	6	24	0,0300	0,0235	0,0065
195	49	6	24	0,1535	0,0200	0,1335
195	49	6	24	0,0309	0,0274	0,0035
195	49	6	24	3,8957	0,0185	3,8772
195	49	6	24	0,3828	0,0207	0,3621
195	49	6	24	0,0845	0,0238	0,0606
195	49	6	24	3,4299	0,0381	3,3918
195	49	6	24	0,3130	0,0197	0,2932
195	49	6	24	0,0292	0,0197	0,0095
195	49	6	24	1,3062	0,0170	1,2892
Promedio ± Desviación estándar						
195 ± 0	49 ± 0	-	-	$0,9656 \pm 1,4758$	$0,0228 \pm 0,0061$	$0,9427 \pm 1,4739$

Tabla 6.2: Resultados Ant System - A1

Costo ruta	Longitud ruta	Nodo inicial	Nodo terminal	Tiempo resolución (s)
402	74	3	21	9,285105
414	80	3	21	9,280354
409	75	6	8	17,135371
468	83	6	8	9,414249
422	79	6	24	9,33504
422	79	3	21	9,319538
445	80	6	11	9,239018
417	79	3	12	9,344922
443	84	3	20	9,251343
422	79	3	23	9,284512
Promedio ± Desviación estándar				
$426,4 \pm 19,8617$	$79,2 \pm 3,0477$	-	-	$10,08 \pm 2,4763$

Tabla 6.3: Resultados MAX-MIN Ant System - A1

Costo ruta	Longitud ruta	Nodo inicial	Nodo terminal	Tiempo resolución (s)
398	72	3	11	7,996122
412	73	3	25	7,94053
398	72	6	14	7,968163
398	72	3	8	8,023846
398	72	3	14	8,012125
398	72	3	19	7,942992
392	70	3	21	7,958283
398	72	3	21	7,922468
398	72	3	9	7,946106
433	80	3	23	7,951164
Promedio $\pm$ Desviación estándar				
$402,3 \pm 11,8888$	$72,7 \pm 2{,}6687$	-	-	$7,9661 \pm 0{,}033$

Tabla 6.4: Resultados Gurobi + Backtracking - CasoRealPequeño

Costo ruta	Longitud ruta	Nodo inicial	Nodo terminal	Tiempo resolución (s)	Tiempo modelo	Tiempo Backtracking
956	22	32709	32677	0,0157	0,0122	0,0035
956	22	32709	32677	0,0180	0,0150	0,0030
956	22	32709	32677	0,0164	0,0139	0,0025
956	22	32709	32677	0,0331	0,0291	0,0040
956	22	32709	32677	0,0155	0,0125	0,0030
956	22	32709	32677	0,0172	0,0142	0,0030
956	22	32709	32677	0,0172	0,0152	0,0020
956	22	32709	32677	0,0165	0,0135	0,0030
956	22	32709	32677	0,0161	0,0131	0,0030
956	22	32709	32677	0,0190	0,0150	0,0040
Promedio ± Desviación estándar						
956 ± 0	22 ± 0	-	-	$0,0185 \pm 0,0053$	$0,0154 \pm 0,0049$	$0,0031 \pm 0,0006$

Tabla 6.5: Resultados Ant System - CasoRealPequeño

Costo ruta	Longitud ruta	Nodo inicial	Nodo terminal	Tiempo resolución (s)
956,73	21	32709	32677	2,19621
956,73	21	32709	32677	2,195915
956,73	21	32709	32677	2,203605
956,73	21	32709	32677	2,185924
956,73	21	32709	32677	2,176795
956,73	21	32709	32677	2,174464
956,73	21	32709	32677	2,170247
956,73	21	32709	32677	2,16699
956,73	21	32709	32677	2,229806
956,73	21	32709	32677	2,206433
Promedio $\pm$ Desviación estándar				
$956,73 \pm 0$	21 ± 0	-	-	$2,1906 \pm 0,0196$

Tabla 6.6: Resultados MAX-MIN Ant System - CasoRealPequeño

956,73	21	32709	32677	3,37801
956,73	21	32709	32677	3,429353
956,73	21	32709	32677	3,39893
956,73	21	32709	32677	3,414488
956,73	21	32709	32677	3,350569
956,73	21	32709	32677	3,360752
956,73	21	32709	32677	3,43115
956,73	21	32709	32677	3,392004
956,73	21	32709	32677	3,393956
956,73	21	32709	32677	3,360175
Promedio $\pm$ Desviación estándar				
$1837,2 \pm 58,4347$	$42 \pm 1,9436$	-	-	$3,3909 \pm 0,02$

Tabla 6.7: Resultados Ant System - CasoRealMediano

Costo ruta	Longitud ruta	Nodo inicial	Nodo terminal	Tiempo resolución (s)
14977,4	304	31837	31965	416,629772
16682,4	362	31837	31965	412,140916
17264,2	337	31837	31965	417,315531
18569,6	354	31837	31965	417,996751
18875,1	370	31837	31965	414,74548
17250,4	344	31837	31965	417,929479
18170,9	350	31837	31965	410,528251
18018	364	31837	31965	413,987511
15948,7	324	31837	31965	410,491773
18319,8	357	31837	31965	201,233081
Promedio ± Desviación estándar				
$17407,65 \pm 1242,6037$	$346,6 \pm 20,2386$	-	-	$393,2998 \pm 67,54$

Tabla 6.8: Resultados MAX-MIN Ant System - CasoRealMediano

Costo ruta	Longitud ruta	Nodo inicial	Nodo terminal	Tiempo resolución (s)
16328	366	31837	31965	186,0153
15128	299	31837	31965	208,6141
17058	375	31837	31965	210,6925
14520	319	31837	31965	209,2988
16168	368	31837	31965	204,7647
16862	416	31837	31965	208,5779
17100	350	31837	31965	208,7970
16185	322	31837	31965	208,4538
15983	336	31837	31965	207,9620
16385	376	31837	31965	207,0822
Promedio ± Desviación estándar				
$14600,09 \pm 805,0651$	$284,8 \pm 11,2229$	-	-	$202,3784 \pm 3,25$

El resto de resultados serán publicados en el siguiente repositorio `https://github.com/newuser161/rescol`

Anexo B: Gráficos de convergencia

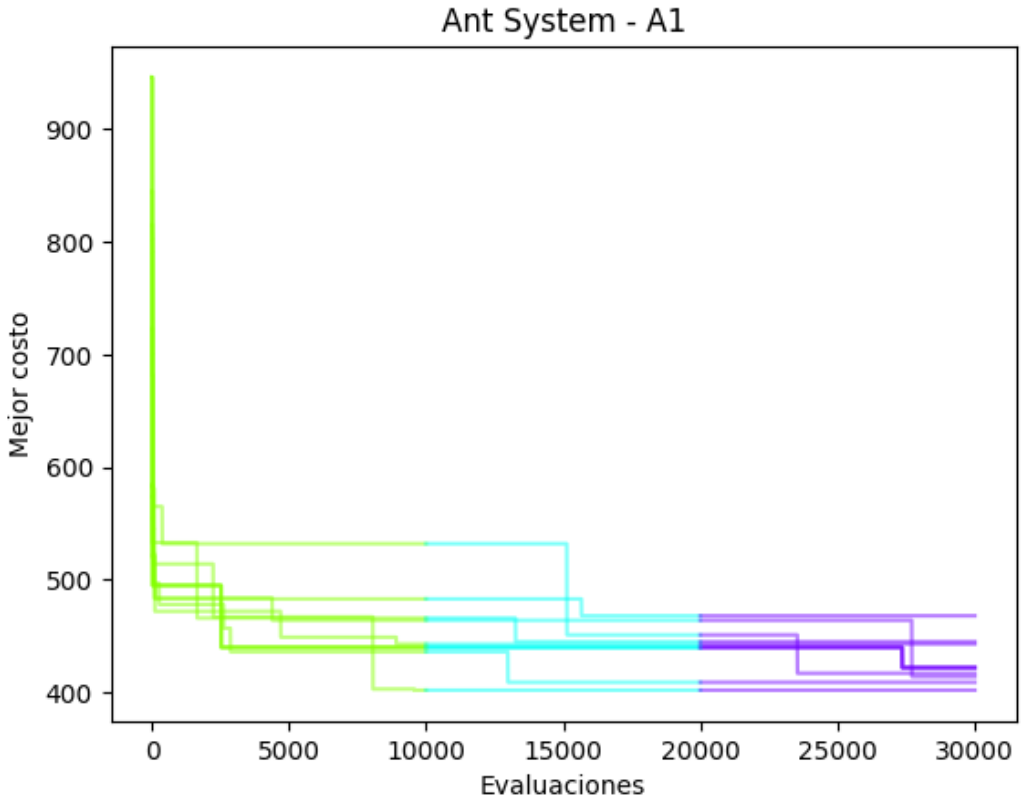


Figura 6.2: Gráfico de convergencia - A1 - Ant System

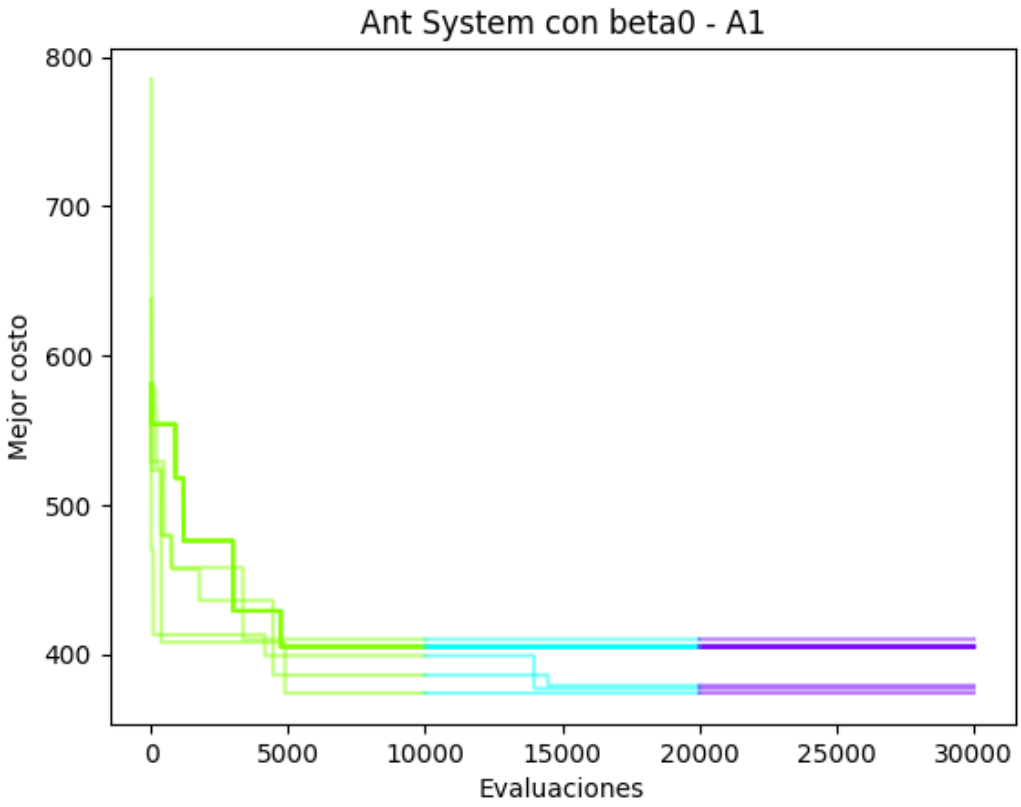


Figura 6.3: Gráfico de convergencia - A1 - Ant System con beta0

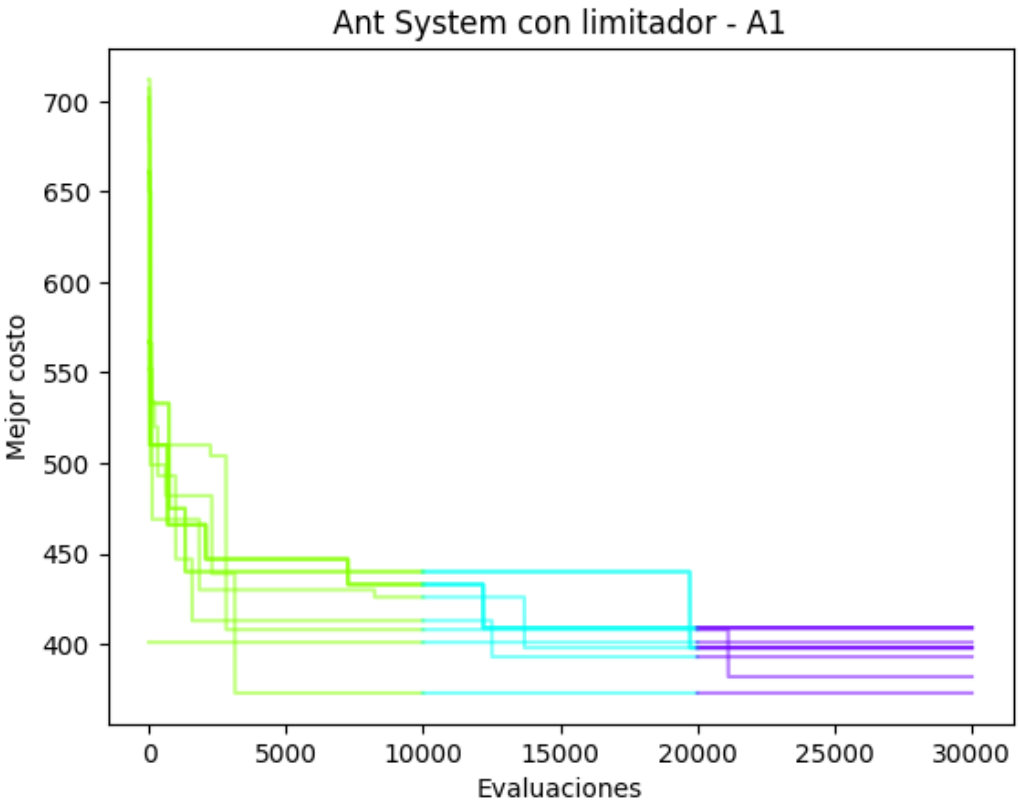


Figura 6.4: Gráfico de convergencia - A1 - Ant System con limitador

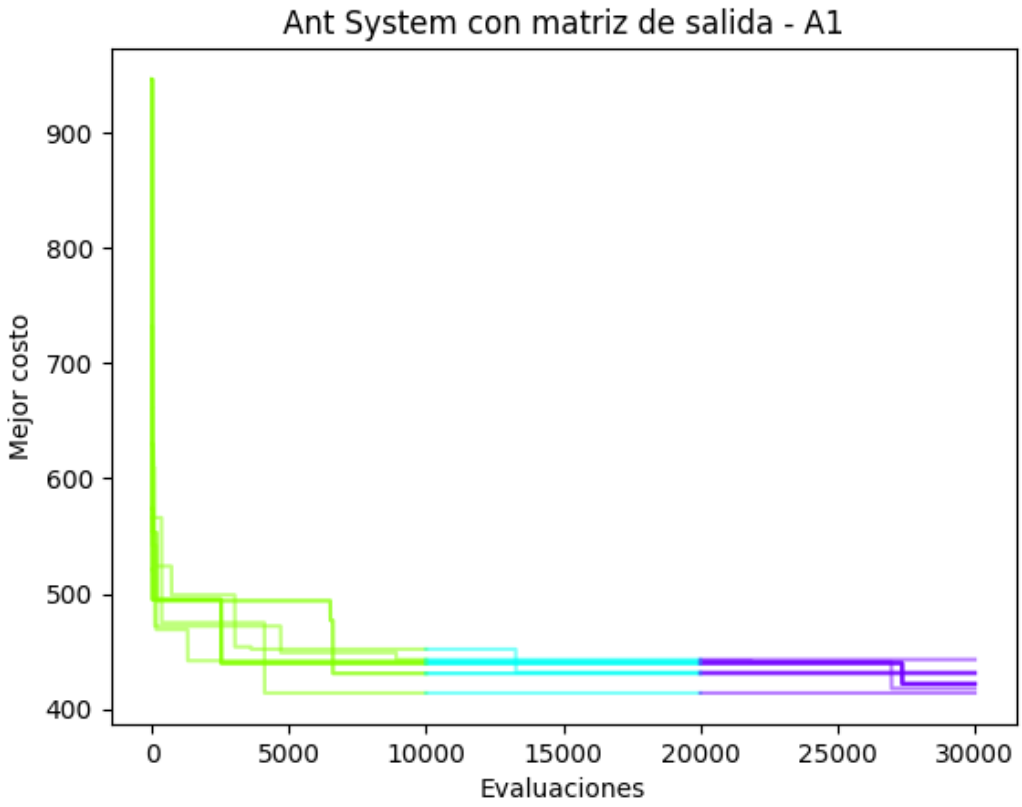


Figura 6.5: Gráfico de convergencia - A1 - Ant System con matriz de salida

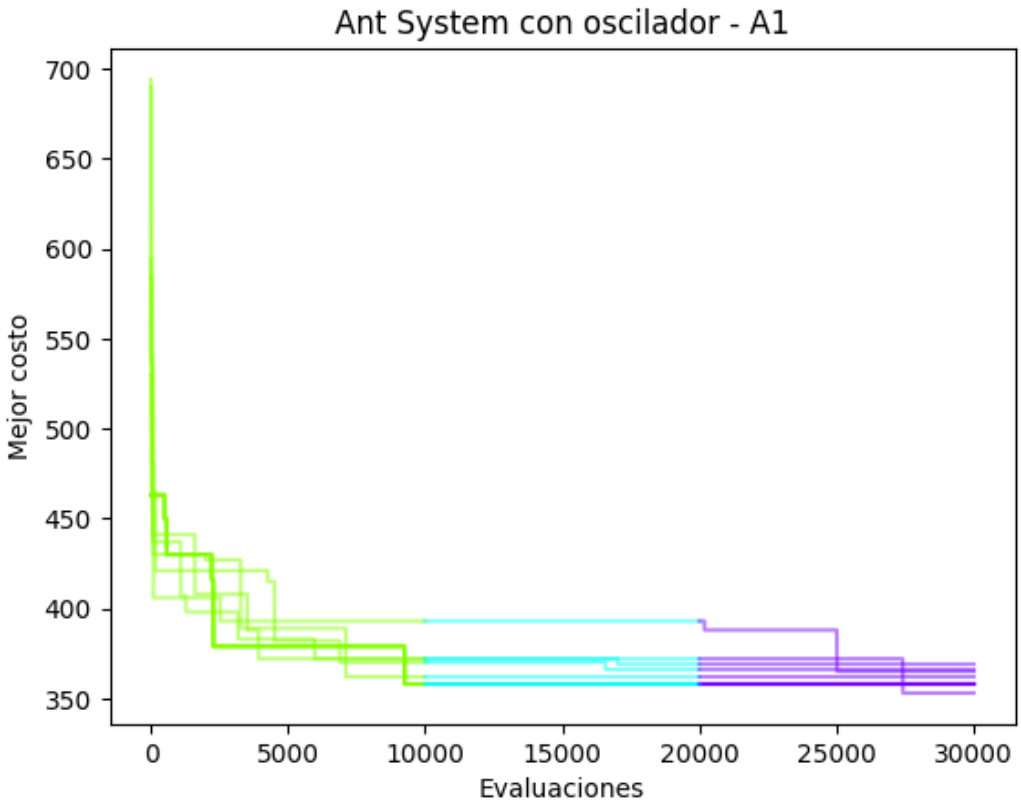


Figura 6.6: Gráfico de convergencia - A1 - Ant System con oscilador

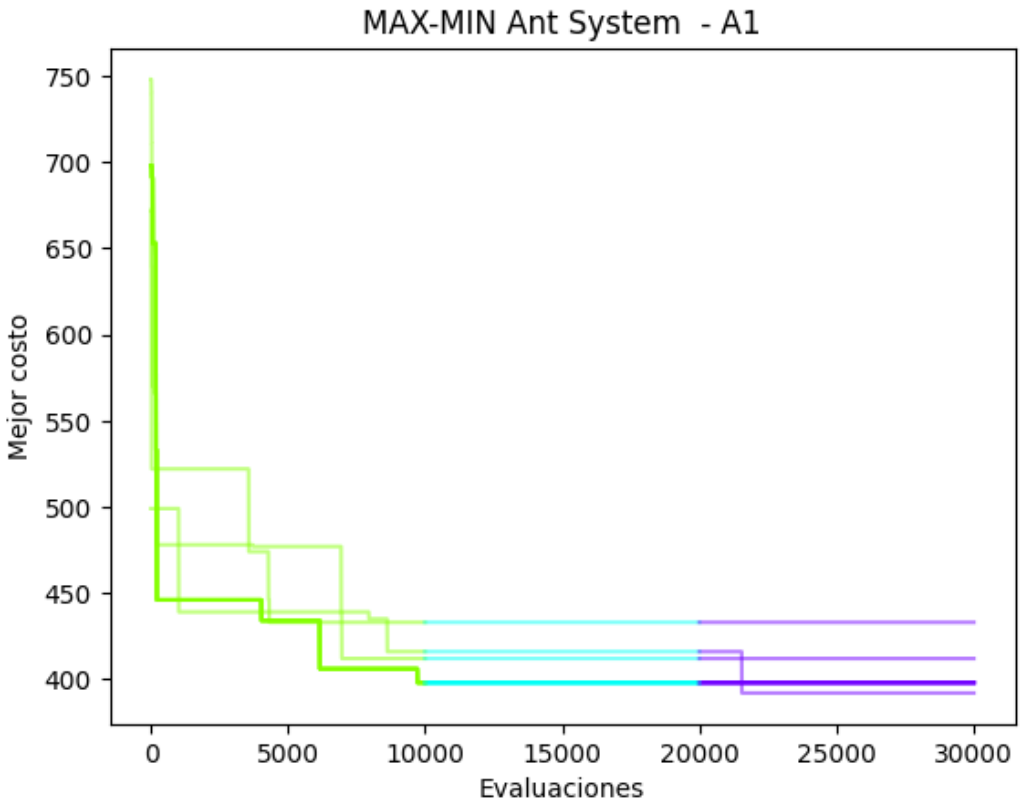


Figura 6.7: Gráfico de convergencia - A1 - MAX-MIN Ant System

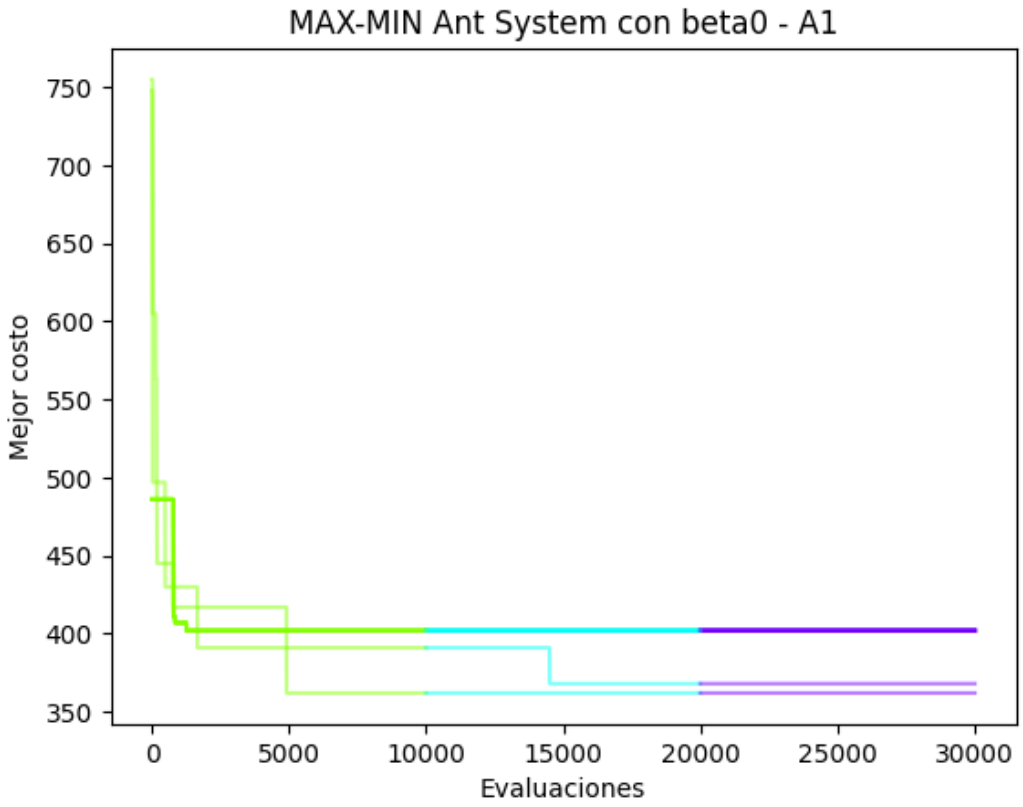


Figura 6.8: Gráfico de convergencia - A1 - MAX-MIN Ant System con beta0

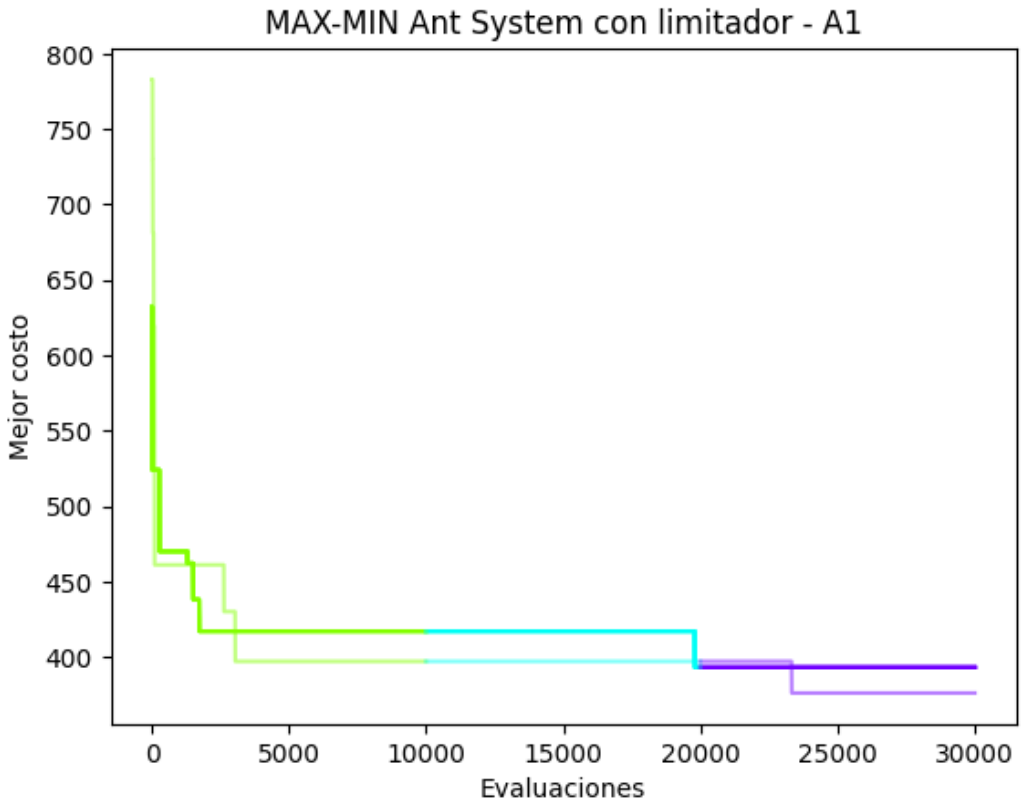


Figura 6.9: Gráfico de convergencia - A1 - MAX-MIN Ant System con limitador

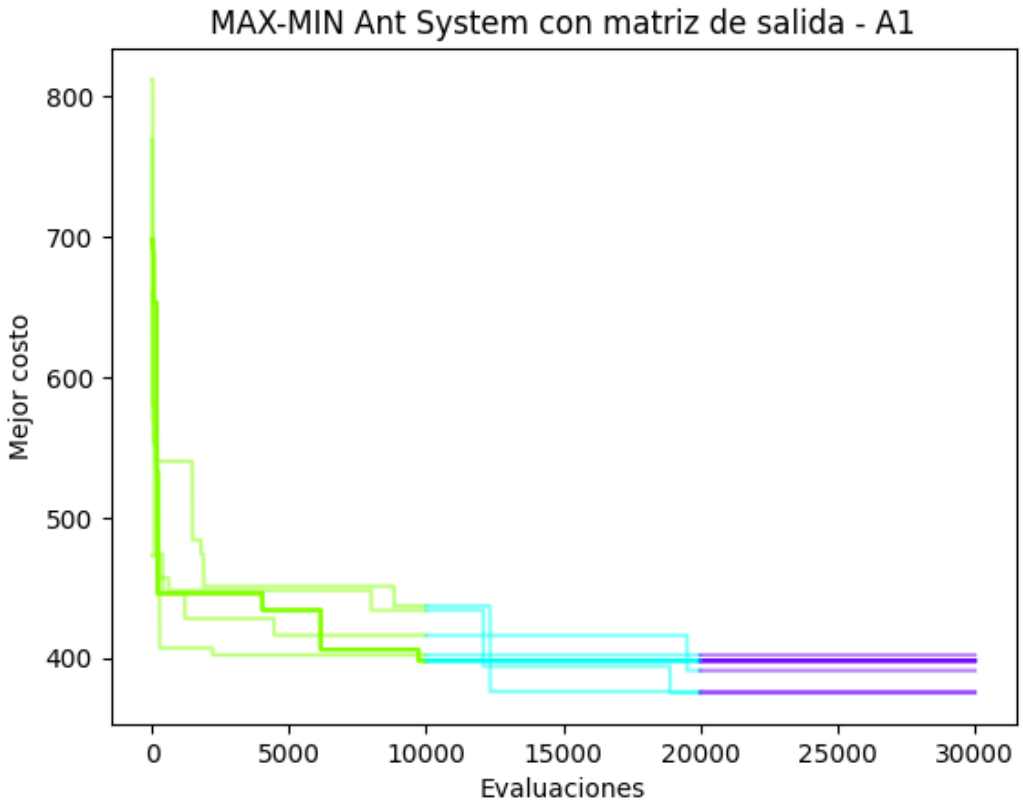


Figura 6.10: Gráfico de convergencia - A1 - MAX-MIN Ant System con matriz de salida

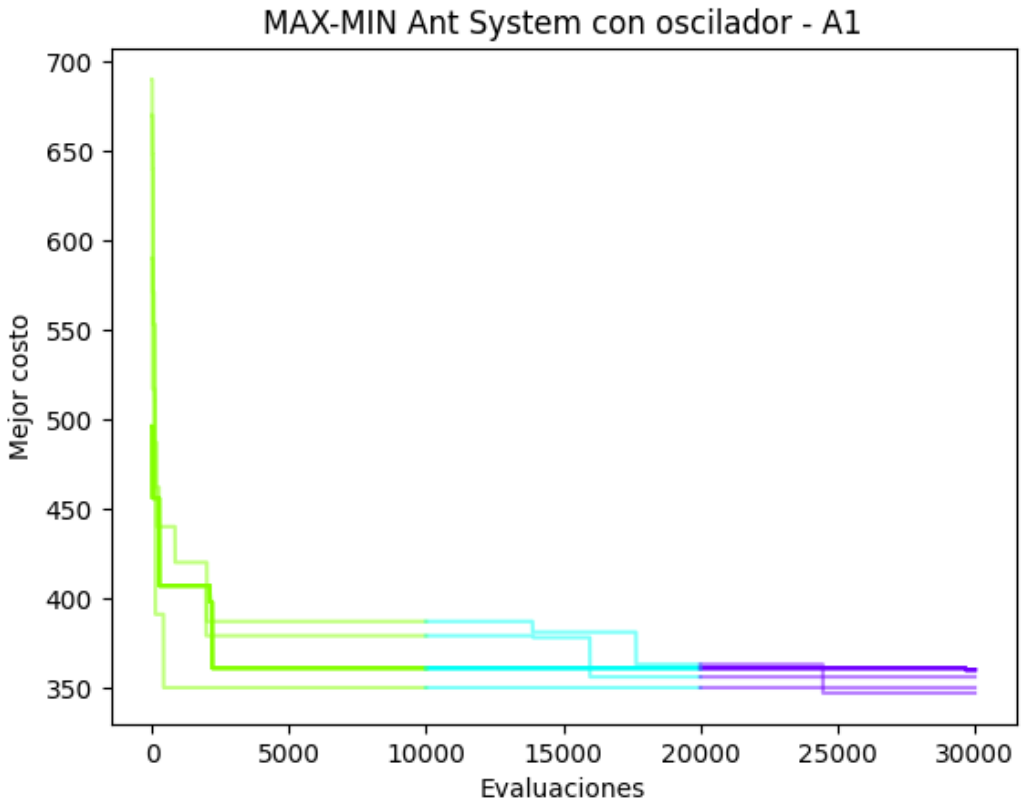


Figura 6.11: Gráfico de convergencia - A1 - MAX-MIN Ant System con oscilador

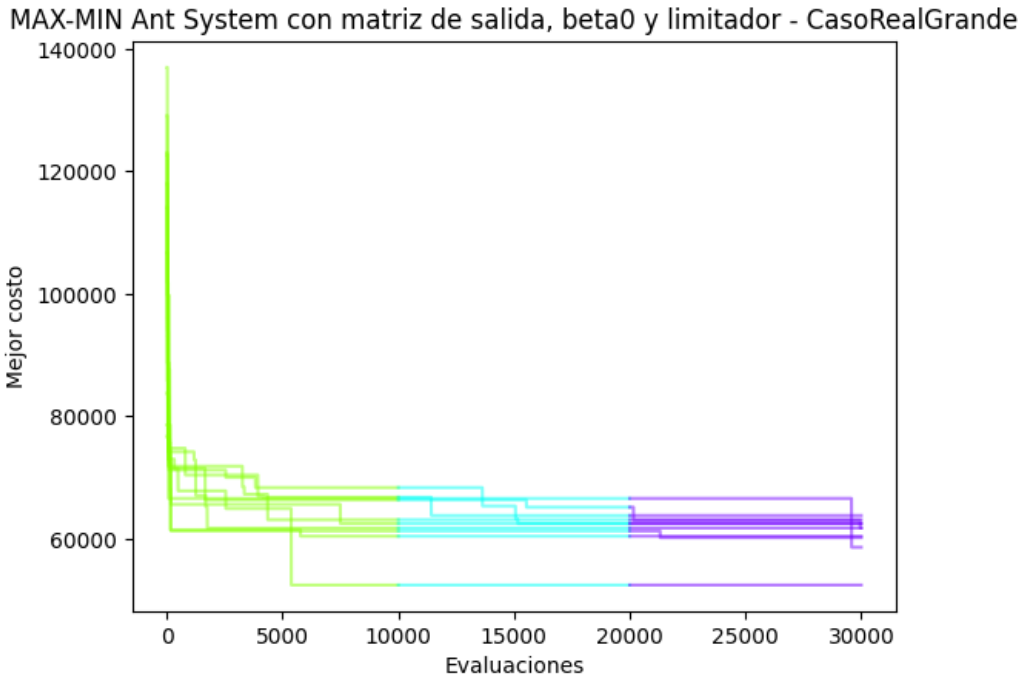


Figura 6.12: Gráfico de convergencia - CasoRealGrande - MAX-MIN Ant System con matriz de salida, beta0 y limitador

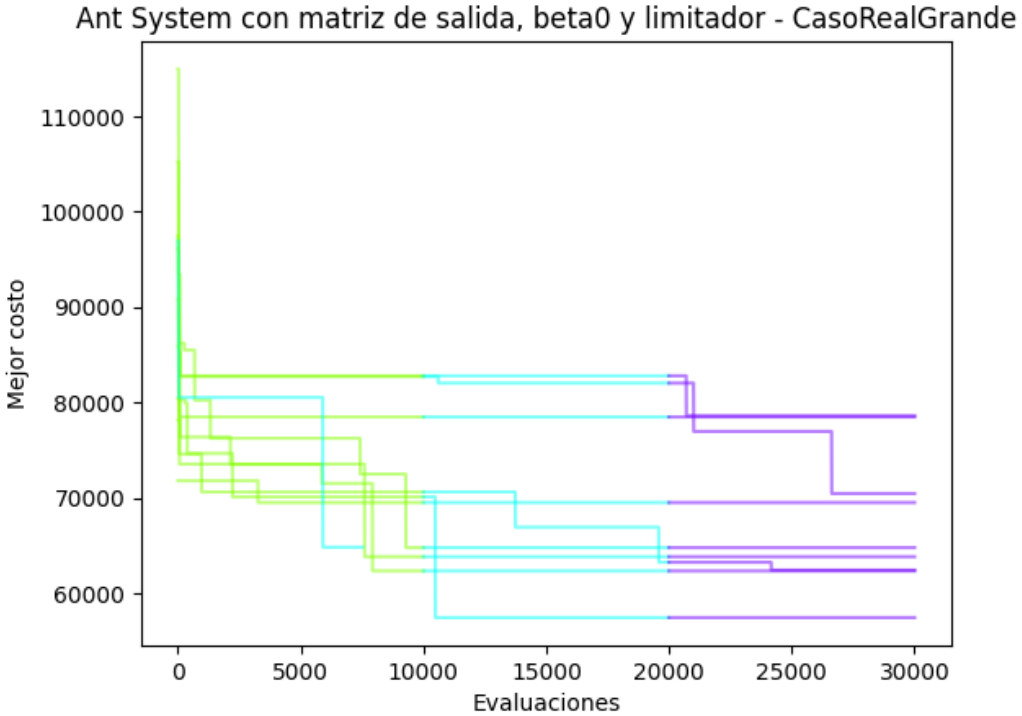


Figura 6.13: Gráfico de convergencia - CasoRealGrande - Ant System con matriz de salida, beta0 y limitador

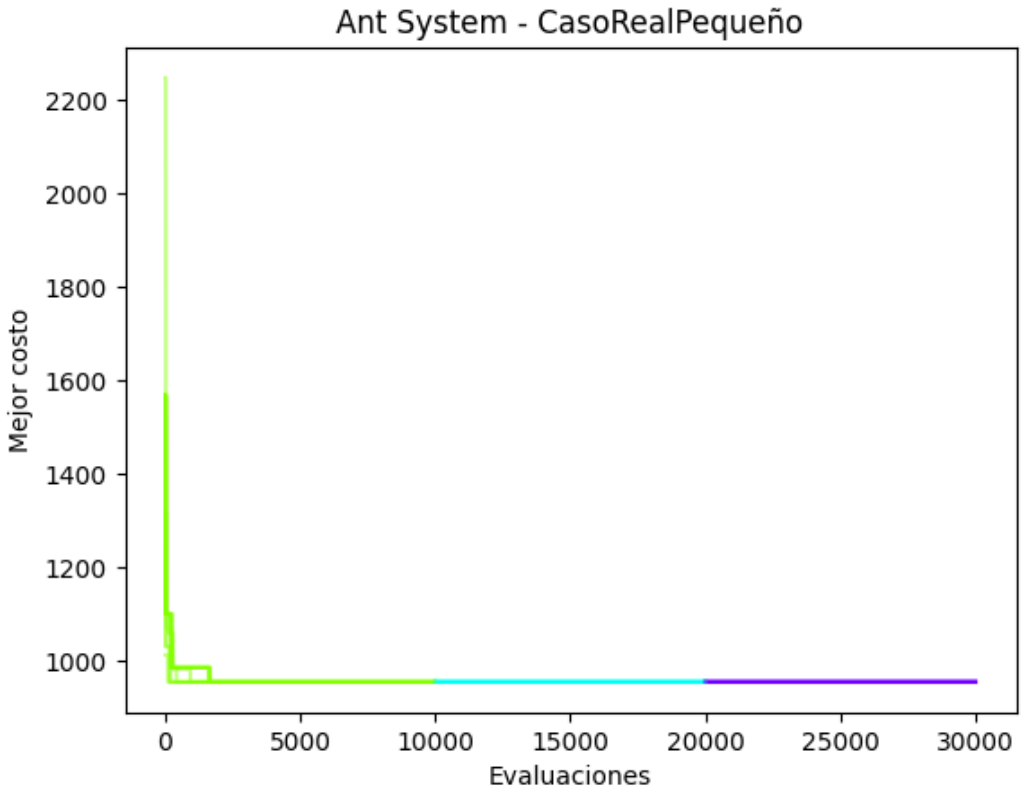


Figura 6.14: Gráfico de convergencia - CasoRealPequeño - Ant System

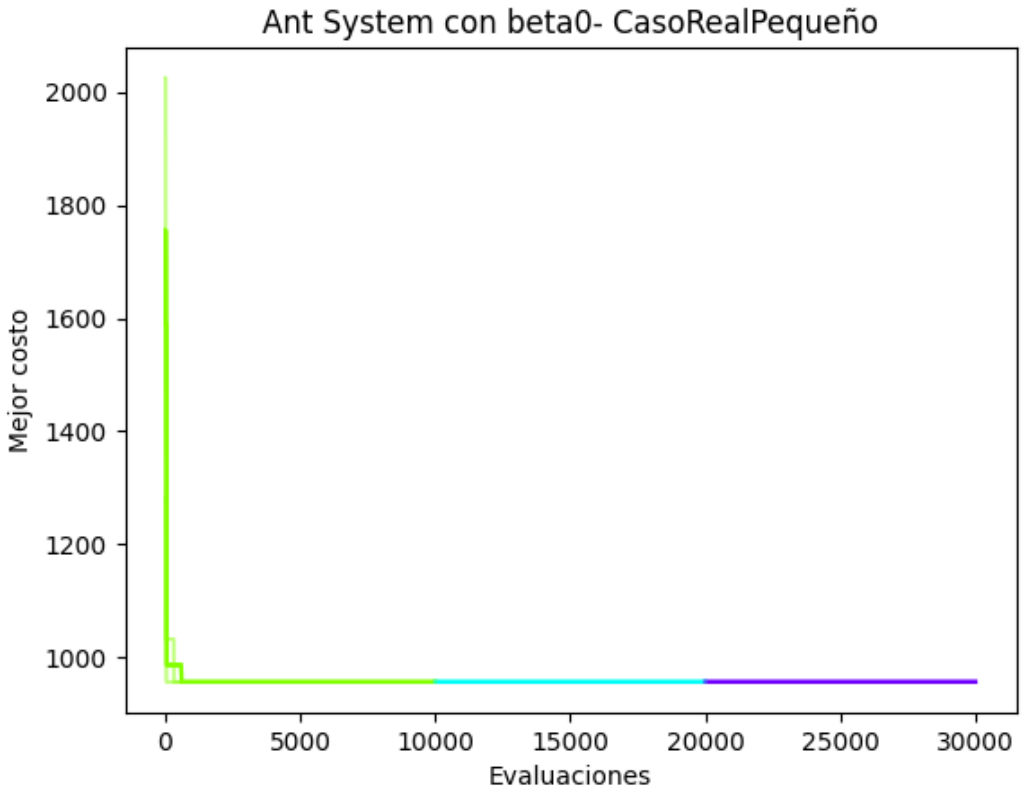


Figura 6.15: Gráfico de convergencia - CasoRealPequeño - Ant System con beta0

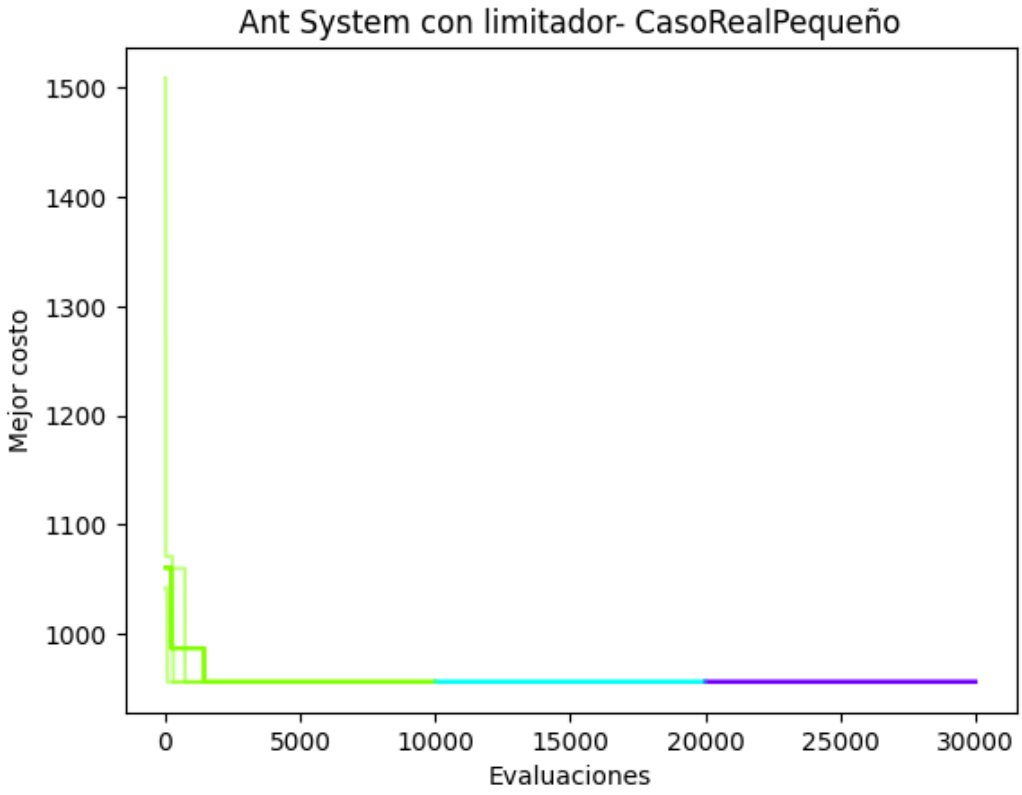


Figura 6.16: Gráfico de convergencia - CasoRealPequeño - Ant System con limitador

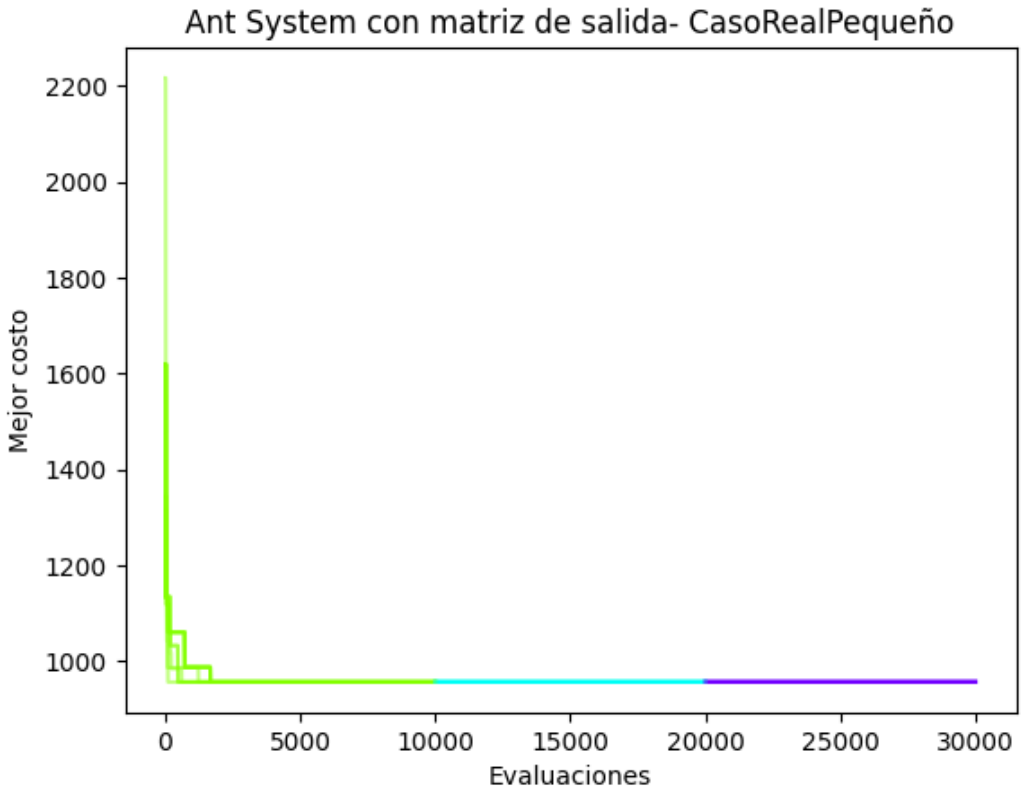


Figura 6.17: Gráfico de convergencia - CasoRealPequeño - Ant System con matriz de salida

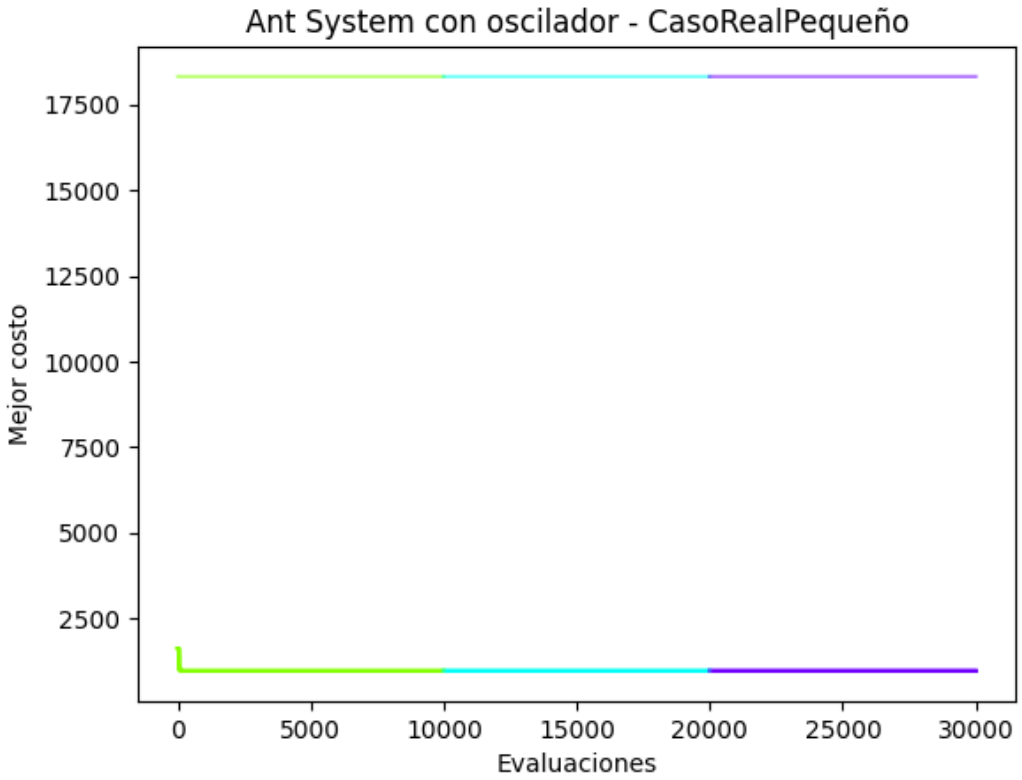


Figura 6.18: Gráfico de convergencia - CasoRealPequeño - Ant System con oscilador

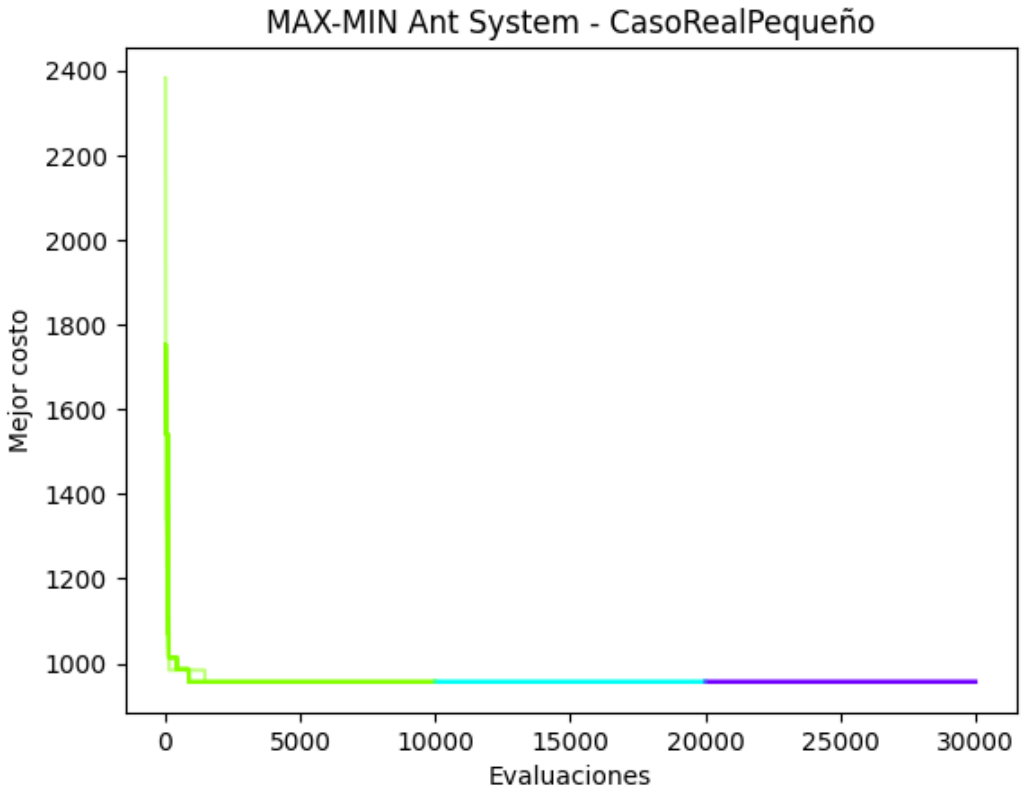


Figura 6.19: Gráfico de convergencia - CasoRealPequeño - MAX-MIN Ant System

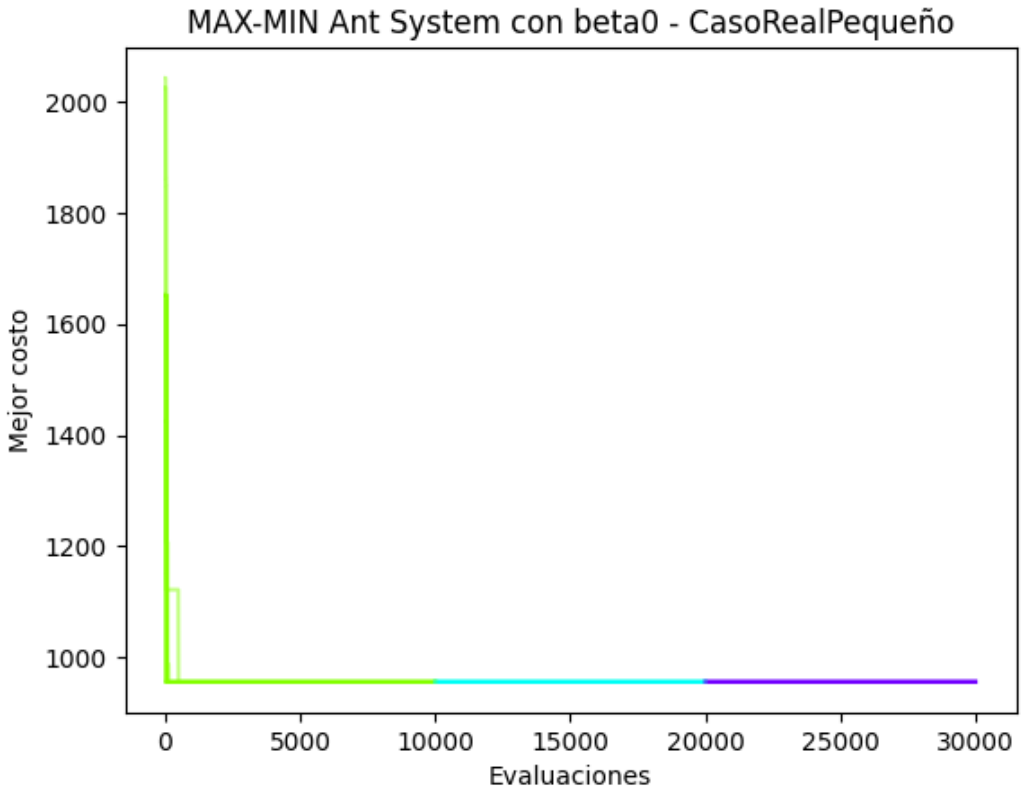


Figura 6.20: Gráfico de convergencia - CasoRealPequeño - MAX-MIN Ant System con beta0

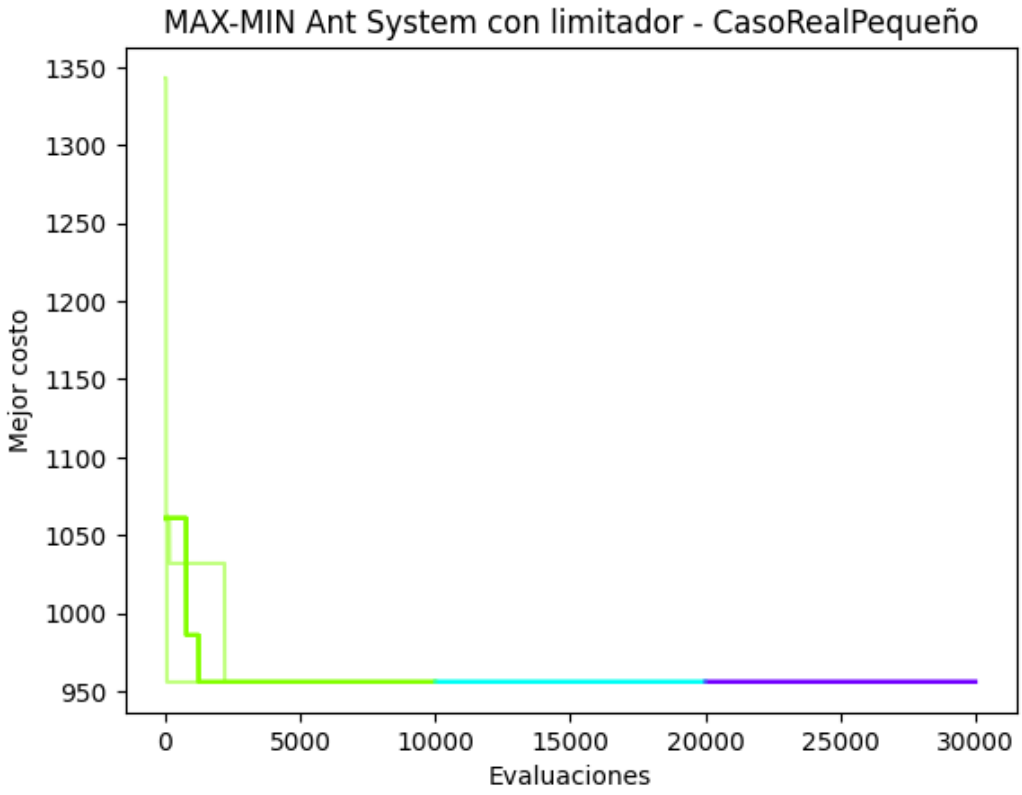


Figura 6.21: Gráfico de convergencia - CasoRealPequeño - MAX-MIN Ant System con limitador

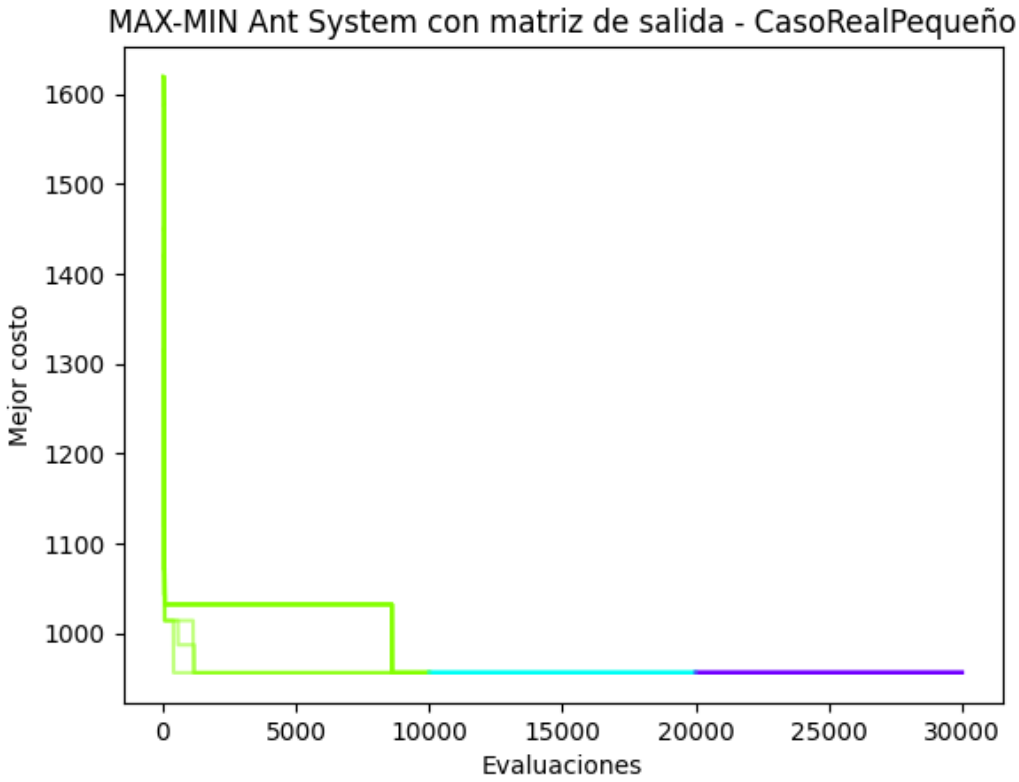


Figura 6.22: Gráfico de convergencia - CasoRealPequeño - MAX-MIN Ant System con matriz de salida

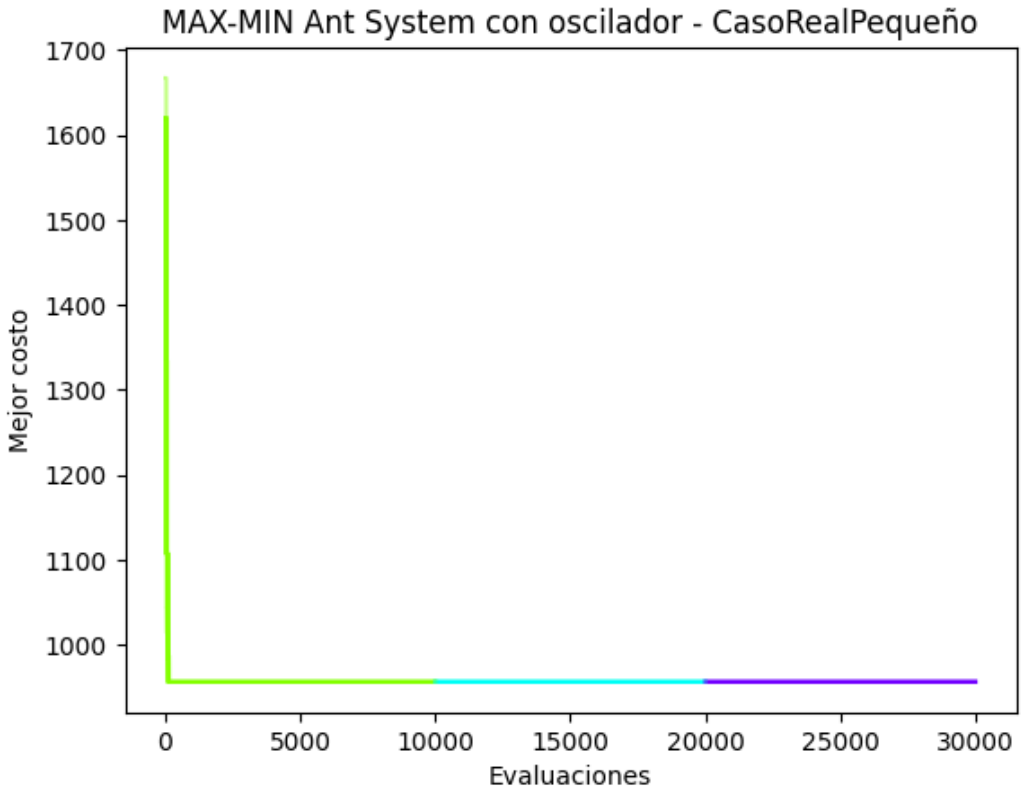


Figura 6.23: Gráfico de convergencia - CasoRealPequeño - MAX-MIN Ant System con oscilador

Anexo C: Coordenadas paralelas

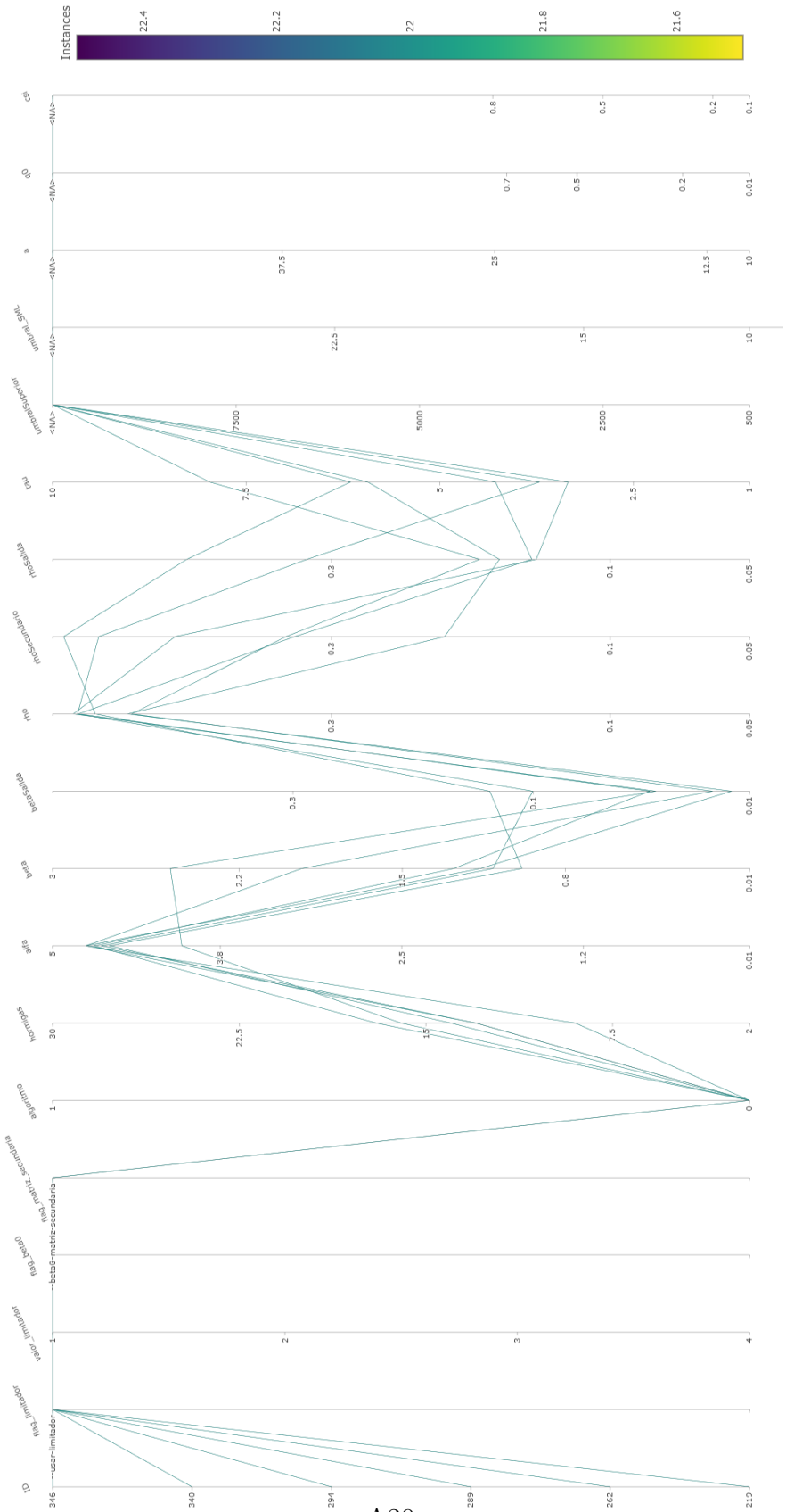


Figura 6.24: Visualización de coordenadas paralelas - Mejores soluciones

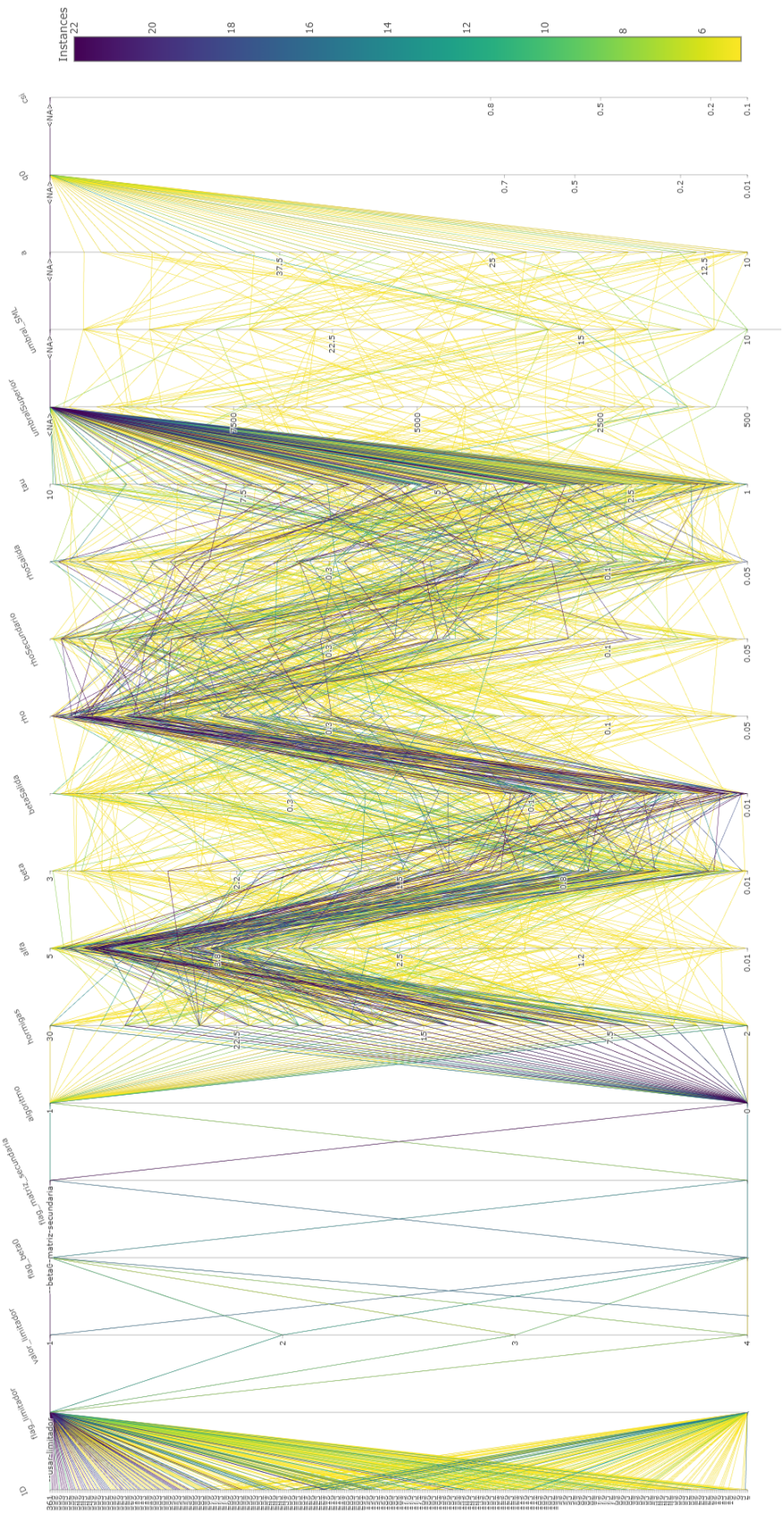


Figura 6.25: Visualización de coordenadas paralelas - Todo el espacio de búsqueda

Anexo D: Carta Gantt

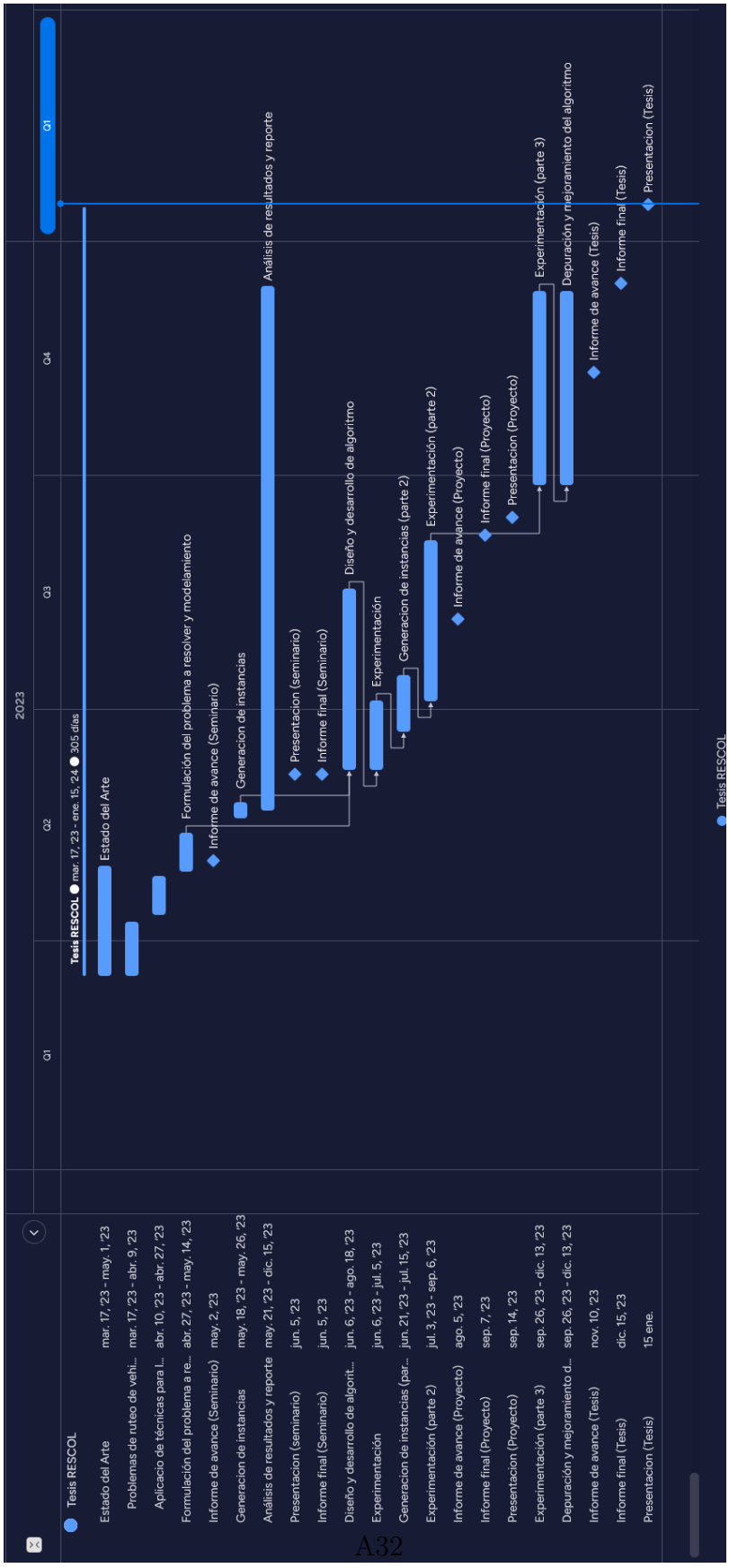


Figura 6.26: Carta Gantt